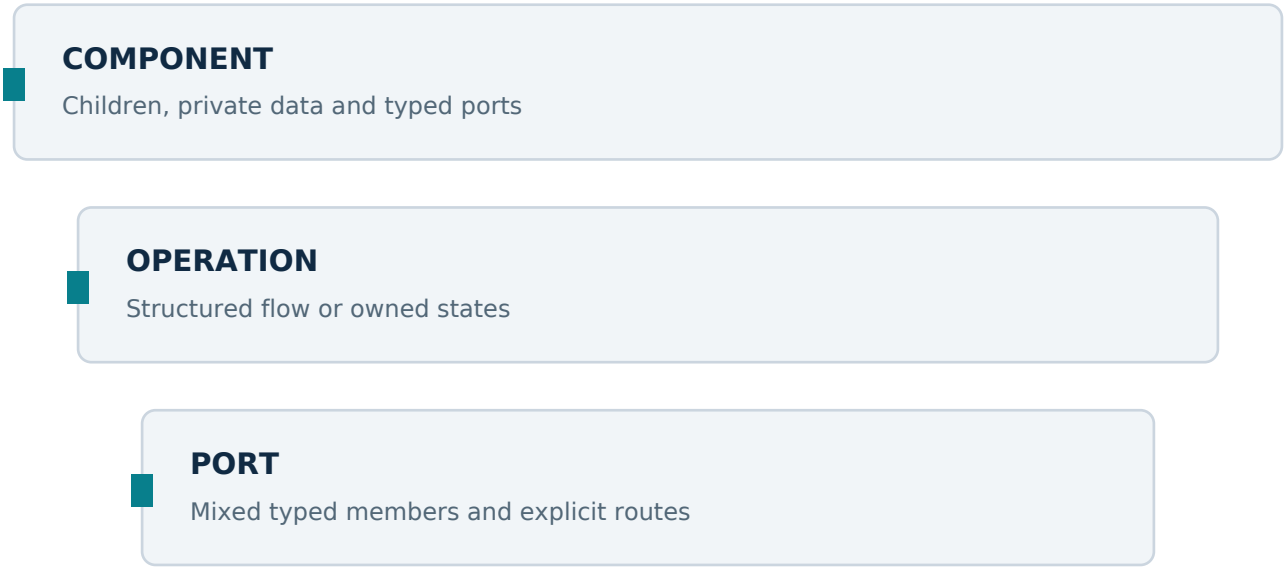
SOFTWARE ARCHITECTURE MODELING LANGUAGE

SWAML

Complete language specification

Specification 2.0.0 / Language 2.0 / Reference simulation swaml-reference/2.0



Typed interfaces at every software boundary.

Embedded realtime systems, services and event-driven architectures.

22 October 2026 / Version 2.0.0 / SWAML contributors

Independent language design on a proposed KerML semantic foundation.

github.com/voidaliot/swaml

About this edition

VERSION 2.0.0 / REFERENCE EDITION

This book contains the complete SWAML 2.0 language specification, every keyword in alphabetical order, formal execution rules, flat notation examples, an ontology and view guide, a reference simulation binding, the complete grammar and runnable model examples. The specification and package revision is 2.0.0. Source files begin with packages; hosts select and pin the 2.0 grammar. Explicit type definitions retain def; component usages form execution roots and operations own their states directly. Reusable QoS definitions reduce repeated connector policy without weakening explicit routes.

Normative chapters define syntax, validity, interface isolation, operation/state behavior and mapping obligations. Tutorials and figures illustrate these rules. This independently authored language is not an OMG UML release or an AUTOSAR implementation. KerML export remains a proposed semantic mapping contract.

The reference parser, validator and deterministic logical-time simulator provide executable evidence. Production timing, runtime and platform mappings require the target binding and evidence defined in the normative chapters.

Navigation	How to use this PDF
Contents and bookmarks	Select a contents entry or use the PDF outline to open a chapter or section.
Cross-references	Links to included documents jump inside this book. Other repository links target the pinned v2.0.0 release.
Code and tables	Code is selectable, numbered and wrapped without omitting source text. Tables repeat their headers across pages.
Source record	The build manifest lists every included source and its SHA-256 digest. Original source files are embedded as PDF attachments. The release includes a PDF checksum.

Original text and code: MIT license, copyright 2026 Catalin Ilie. Referenced standards retain their own terms; no standards PDFs are redistributed. Full license text appears at the end.

Contents

About this edition 2

SWAML 2.0 keyword reference 9

Keyword: after . . . 9
Keyword: and . . . 9
Keyword: assert . . . 9
Keyword: attribute . . . 9
Keyword: bool . . . 9
Keyword: byte . . . 10
Keyword: call . . . 10
Keyword: component . . . 10
Keyword: concurrent . . . 10
Keyword: connect . . . 10
Keyword: const . . . 10
Keyword: datatype . . . 10
Keyword: deadline . . . 10
Keyword: def . . . 11
Keyword: deploy . . . 11
Keyword: deployment . . . 11
Keyword: else . . . 11
Keyword: entry . . . 11
Keyword: enum . . . 11
Keyword: event . . . 11
Keyword: every . . . 11
Keyword: exit . . . 12
Keyword: fail . . . 12
Keyword: false . . . 12
Keyword: final . . . 12
Keyword: for . . . 12
Keyword: from . . . 12
Keyword: idempotent . . . 12
Keyword: if . . . 12
Keyword: import . . . 13
Keyword: in . . . 13
Keyword: include . . . 13
Keyword: initial . . . 13
Keyword: int . . . 13
Keyword: interruptible . . . 13
Keyword: kind . . . 13
Keyword: ms . . . 13
Keyword: name . . . 14
Keyword: not . . . 14
Keyword: ns . . . 14

Keyword: of . . . 14
Keyword: on . . . 14
Keyword: operation . . . 14
Keyword: optional . . . 14
Keyword: or . . . 15
Keyword: out . . . 15
Keyword: package . . . 15
Keyword: path . . . 15
Keyword: port . . . 15
Keyword: provides . . . 15
Keyword: publish . . . 15
Keyword: qos . . . 15
Keyword: real . . . 16
Keyword: receive . . . 16
Keyword: record . . . 16
Keyword: recursive . . . 16
Keyword: redefines . . . 16
Keyword: reply . . . 16
Keyword: request . . . 16
Keyword: requirement . . . 17
Keyword: requires . . . 17
Keyword: return . . . 17
Keyword: returns . . . 17
Keyword: reversed . . . 17
Keyword: s . . . 17
Keyword: satisfy . . . 17
Keyword: schedule . . . 17
Keyword: send . . . 18
Keyword: specializes . . . 18
Keyword: start . . . 18
Keyword: state . . . 18
Keyword: string . . . 18
Keyword: times . . . 18
Keyword: to . . . 18
Keyword: true . . . 18
Keyword: try . . . 19
Keyword: uint . . . 19
Keyword: until . . . 19
Keyword: us . . . 19
Keyword: useCase . . . 19
Keyword: var . . . 19
Keyword: verificationCase . . . 19
Keyword: verify . . . 19
Keyword: via . . . 20
Keyword: view . . . 20
Keyword: void . . . 20

Keyword: wait . . . 20
Keyword: when . . . 20
Keyword: where . . . 20
Keyword: while . . . 20
Symbols and precedence . . . 20

SWAML 2.0 language specification **21**

1. Normative set . . . 21
2. Precedence and scope . . . 21
3. Invariants . . . 21
4. Inventory . . . 22

1. Structure, names and types **23**

1.1 Names and documentation . . . 23
1.2 Components and occurrences . . . 23
1.3 State and lexical values . . . 23
1.4 Value domains . . . 24
1.5 Reuse and library boundaries . . . 24

2. Ports and communication **25**

2.1 Directed port members . . . 25
2.2 Structural compatibility . . . 25
2.3 Connectors and closure . . . 25
2.4 Versioned policies and queues . . . 26
2.5 Requests, replies and failure . . . 26

3. Operations, schedules and states **27**

3.1 Operations and effects . . . 27
3.2 Failure . . . 27
3.3 Statements and scopes . . . 27
3.4 Suspension and locks . . . 28
3.5 Schedule . . . 28
3.6 States . . . 29

4. Deployment, cases, views and conformance **30**

4.1 Deployment . . . 30
4.2 Requirements and cases . . . 30
4.3 Views . . . 30
4.4 Validation groups . . . 31
4.5 Evidence boundaries . . . 31

5. Proposed KerML semantic mapping **32**

5.1 Representation . . . 32
5.2 Required semantic library . . . 32
5.3 Traceability and claims . . . 32

6. Reference execution contract **33**

6.1 Lifecycle and time . . . 33
6.2 Lock and snapshot . . . 33
6.3 Values, flow and failure . . . 33
6.4 Communication . . . 34
6.5 Input ownership and interruption . . . 34
6.6 Evidence and resources . . . 34

7. Normalized model contract 35

7.1 Identity and provenance . . . 35
7.2 Families . . . 35
7.3 Routes and queues . . . 36
7.4 Executable lowering . . . 36
7.5 Presentation and equivalence . . . 36

8. Conformance obligations and evidence 37

8.1 Static obligations . . . 37
8.2 Runtime obligations . . . 38
8.3 Analysis, kernel and graphical claims . . . 39

9. Formal semantics 40

9.1 Domains . . . 40
9.2 Typing schemas . . . 40
9.3 Transition schemas . . . 41
9.4 Projection and adequacy . . . 42

SWAML 2.0 graphical notation 43

SWAML 2.0 canonical graphical catalog 44

Packages and imports . . . 44
Typed copied values . . . 45
Component definitions and usages . . . 45
Indexed occurrence identity . . . 46
Attributes and lexical locals . . . 46
Mixed directed port members . . . 47
Policy-bearing connection . . . 47
Queue-free boundary forwarding . . . 48
Operations and nested ownership . . . 48
Expression calls and copied out values . . . 49
Runtime failure and lazy fallback . . . 49
Statements and decisions . . . 50
Loops and copied iteration . . . 50
Schedule entry and successful chaining . . . 51
Suspending wait and receive . . . 51
Armed interrupt and cancellation . . . 52
Ordered state transitions . . . 52
Platform Nodes and placement . . . 53
Requirements and intent links . . . 53

Named routed case steps . . . 54
Seven library view definitions . . . 54
Declared and observed timing . . . 55
Owned documentation and lexical notes . . . 55
Typed expressions and operators . . . 56
Lexical and path spelling . . . 56

A. Complete concrete grammar **57**

B. Grammar-to-semantics inventory **60**

C. Model and library sources **67**

01 hello . . . 67
02 realtime controller . . . 68
03 service pubsub . . . 69
04 stateful function . . . 70
05 values and flow . . . 71
06 mixed gateway . . . 72
07 client server . . . 73
08 service field . . . 74
09 component roots . . . 75
10 reusable qos . . . 76
11 autosar classic . . . 77
12 autosar adaptive . . . 78
13 web application . . . 79
14 cloud service . . . 80
15 component specialization . . . 81
16 requirements and views . . . 82
17 function timing . . . 83
18 control loops . . . 84
19 interruptible function . . . 85
20 classes and operations . . . 86
21 receive and try . . . 87
22 platform deployment . . . 88
23 multiplicity . . . 89
24 operation usage . . . 90
25 function completion . . . 91
26 boundary forwarding . . . 92
27 mixed port . . . 93
swaml 2.0 . . . 94

Quick start: SWAML 2.0 **95**

SWAML 2.0 usage guide **96**

A first model . . . 96
Reuse structure without sharing state . . . 96
Choose the value's lifetime . . . 97

Compute with operations . . . 97
Connect components . . . 97
Serve and request a value . . . 98
Schedule and suspend deliberately . . . 98
Receive, interrupt and model state . . . 98
Describe deployment, intent and views . . . 98
Validate and repair . . . 99

AI authoring with SWAML 2.0 **100**

Build a model from intent . . . 100
Validate-fix loop . . . 100
Common diagnostics . . . 101

Architecture patterns **102**

Ontology and views **103**

SWAML 2.0 reference simulation **104**

Requirements, documentation and intent **105**

SWAML 2.0 worked diagrams **106**

Directed ports and explicit connections . . . 106
Forwarding preserves boundary direction . . . 106
Schedule and success chaining . . . 107
Operation-owned ordered states . . . 107
Indexed placement on platform Nodes . . . 108
Declared contracts and observed intervals . . . 108

SWAML 2.0 implementation status **109**

Reference implementation . . . 109
Deliberate implementation limits . . . 110

Tool integration **111**

SWAML 2.0 review resolution **112**

SWAML 2.0.0 release **114**

ADR 0016: SWAML 2.0 restart **115**

D. License **116**

USER REFERENCE / CONCERN AND KEYWORD NAVIGATION

SWAML 2.0 keyword reference

Generated from spec/keywords.json and the normative grammar. Examples here are grammar fragments; complete runnable models are in the usage guide and examples. Five contextual words remain legal names outside their positions. Call appears only in the on call trigger.

after · and · assert · attribute · bool · byte · call · component · concurrent · connect · const · datatype · deadline · def · deploy · deployment · else · entry · enum · event · every · exit · fail · false · final · for · from · idempotent · if · import · in · include · initial · int · interruptible · kind · ms · name · not · ns · of · on · operation · optional · or · out · package · path · port · provides · publish · qos · real · receive · record · recursive · redefines · reply · request · requirement · requires · return · returns · reversed · s · satisfy · schedule · send · specializes · start · state · string · times · to · true · try · uint · until · us · useCase · var · verificationCase · verify · via · view · void · wait · when · where · while

Keyword: after

Introduces a state timer, case dependency or success schedule dependency.

Context: Ordered state transitions. **Grammar:** ScheduleEntry, Receive, Transition, ScenarioStep. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: and

Used in typed expressions and operators; see its grammar contexts and normative constraints.

Context: Typed expressions and operators. **Grammar:** FilterAnd, AndExpression. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: assert

Used in runtime failure and lazy fallback; see its grammar contexts and normative constraints.

Context: Runtime failure and lazy fallback. **Grammar:** Assert. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: attribute

Declares typed occurrence state or an immutable requirement value.

Context: Attributes and lexical locals. **Grammar:** Attribute, PortMember. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: bool

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: byte

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: call

Used only as the on call schedule trigger.

Context: Schedule entry and successful chaining. **Grammar:** ScheduleEntry. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: component

Declares a reusable definition or a distinct owned occurrence.

Context: Component definitions and usages. **Grammar:** ComponentDef, ComponentUsage. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: concurrent

Permits overlapping non-writing snapshot activations.

Context: Schedule entry and successful chaining. **Grammar:** ScheduleEntry. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: connect

Joins compatible ports; boundary-to-child derives forwarding.

Context: Policy-bearing connection. **Grammar:** Connect. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: const

Makes an attribute immutable for its occurrence.

Context: Attributes and lexical locals. **Grammar:** Attribute. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: datatype

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** DataType. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: deadline

Used in schedule entry and successful chaining; see its grammar contexts and normative constraints.

Context: Schedule entry and successful chaining. **Grammar:** ScheduleEntry. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: def

Used in component definitions and usages; see its grammar contexts and normative constraints.

Context: Component definitions and usages. **Grammar:** ComponentDef, PortDef, QosPolicy, Operation, ViewDef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: deploy

Used in platform nodes and placement; see its grammar contexts and normative constraints.

Context: Platform Nodes and placement. **Grammar:** Placement. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: deployment

Places software root occurrences on platform Node occurrences.

Context: Platform Nodes and placement. **Grammar:** Deployment. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: else

Used in statements and decisions; see its grammar contexts and normative constraints.

Context: Statements and decisions. **Grammar:** If. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: entry

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** StateAction. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: enum

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** EnumType. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: event

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** PortMember. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: every

Used in schedule entry and successful chaining; see its grammar contexts and normative constraints.

Context: Schedule entry and successful chaining. **Grammar:** ScheduleEntry. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: exit

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** StateAction. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: fail

Used in runtime failure and lazy fallback; see its grammar contexts and normative constraints.

Context: Runtime failure and lazy fallback. **Grammar:** Fail. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: false

Used in typed expressions and operators; see its grammar contexts and normative constraints.

Context: Typed expressions and operators. **Grammar:** Literal. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: final

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** State. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: for

Used in loops and copied iteration; see its grammar contexts and normative constraints.

Context: Loops and copied iteration. **Grammar:** For, ForEach. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: from

Introduces the source endpoint of a named case step.

Context: Named routed case steps. **Grammar:** ScenarioStep, Name. **Name use:** contextual.

Normative rule · Graphical catalog.

Keyword: idempotent

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** PortMember. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: if

Used in statements and decisions; see its grammar contexts and normative constraints.

Context: Statements and decisions. **Grammar:** If. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: import

Used in packages and imports; see its grammar contexts and normative constraints.

Context: Packages and imports. **Grammar:** Import. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: in

Used in attributes and lexical locals; see its grammar contexts and normative constraints.

Context: Attributes and lexical locals. **Grammar:** Argument, ForEach. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: include

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** ViewSelection. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: initial

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** OperationBody. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: int

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: interruptible

Used in armed interrupt and cancellation; see its grammar contexts and normative constraints.

Context: Armed interrupt and cancellation. **Grammar:** Interruptible. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: kind

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** FilterComparison. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: ms

Used in lexical and path spelling; see its grammar contexts and normative constraints.

Context: Lexical and path spelling. **Grammar:** Duration. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: name

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** FilterComparison. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: not

Used in typed expressions and operators; see its grammar contexts and normative constraints.

Context: Typed expressions and operators. **Grammar:** FilterPrimary, UnaryExpression. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: ns

Used in lexical and path spelling; see its grammar contexts and normative constraints.

Context: Lexical and path spelling. **Grammar:** Duration. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: of

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** Deployment, Satisfaction, Scenario, View. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: on

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** ScheduleEntry, Receive, Interruptible, StateAction, Transition, Deployment, Placement. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: operation

Declares executable behavior; def supplies a reusable signature/body.

Context: Operations and nested ownership. **Grammar:** PortMember, Operation, OperationUsage. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: optional

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** PortUsage. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: or

Used in suspending wait and receive; see its grammar contexts and normative constraints.

Context: Suspending wait and receive. **Grammar:** Receive, FilterOr, OrExpression. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: out

Used in attributes and lexical locals; see its grammar contexts and normative constraints.

Context: Attributes and lexical locals. **Grammar:** Argument. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: package

Used in packages and imports; see its grammar contexts and normative constraints.

Context: Packages and imports. **Grammar:** Package. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: path

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** FilterComparison. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: port

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** PortDef, PortUsage. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: provides

Offers an attribute/event or implements an operation member.

Context: Directed port members. **Grammar:** PortMember. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: publish

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** Send, ScenarioStep. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: qos

Used in policy-bearing connection; see its grammar contexts and normative constraints.

Context: Policy-bearing connection. **Grammar:** QosPolicy. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: real

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: receive

Used in suspending wait and receive; see its grammar contexts and normative constraints.

Context: Suspending wait and receive. **Grammar:** ScheduleEntry, ReceiveBranch, Interruptible, Transition. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: record

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** RecordLiteral. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: recursive

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** ViewSelection. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: redefines

Used in component definitions and usages; see its grammar contexts and normative constraints.

Context: Component definitions and usages. **Grammar:** ComponentUsage, Attribute, PortUsage, Connect, Operation, OperationUsage. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: reply

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** ScenarioStep. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: request

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** ScenarioStep, Name. **Name use:** contextual.

Normative rule · Graphical catalog.

Keyword: requirement

Used in requirements and intent links; see its grammar contexts and normative constraints.

Context: Requirements and intent links. **Grammar:** Requirement. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: requires

Receives an attribute/event or requests an operation member.

Context: Directed port members. **Grammar:** PortMember. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: return

Used in expression calls and copied out values; see its grammar contexts and normative constraints.

Context: Expression calls and copied out values. **Grammar:** Return. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: returns

Declares the result type; omitted result is void.

Context: Operations and nested ownership. **Grammar:** PortMember, Operation. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: reversed

Flips each direction of a port usage.

Context: Directed port members. **Grammar:** PortUsage, Name. **Name use:** contextual.

Normative rule · Graphical catalog.

Keyword: s

Used in lexical and path spelling; see its grammar contexts and normative constraints.

Context: Lexical and path spelling. **Grammar:** Duration. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: satisfy

Used in requirements and intent links; see its grammar contexts and normative constraints.

Context: Requirements and intent links. **Grammar:** Satisfaction. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: schedule

Owns trigger-to-operation bindings.

Context: Schedule entry and successful chaining. **Grammar:** Schedule. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: send

Used in directed port members; see its grammar contexts and normative constraints.

Context: Directed port members. **Grammar:** Send, ScenarioStep. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: specializes

Used in component definitions and usages; see its grammar contexts and normative constraints.

Context: Component definitions and usages. **Grammar:** ComponentDef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: start

Used in schedule entry and successful chaining; see its grammar contexts and normative constraints.

Context: Schedule entry and successful chaining. **Grammar:** ScheduleEntry. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: state

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** State. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: string

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: times

Introduces a once-evaluated uint loop limit.

Context: Loops and copied iteration. **Grammar:** While, Until, Name. **Name use:** contextual.

Normative rule · Graphical catalog.

Keyword: to

Introduces a case endpoint or state transition target.

Context: Ordered state transitions. **Grammar:** Transition, ScenarioStep, Name. **Name use:** contextual.

Normative rule · Graphical catalog.

Keyword: true

Used in typed expressions and operators; see its grammar contexts and normative constraints.

Context: Typed expressions and operators. **Grammar:** Literal. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: try

Catches runtime expression failures with a lazy compatible fallback.

Context: Runtime failure and lazy fallback. **Grammar:** TryExpression. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: uint

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: until

Runs body then tests the stopping condition.

Context: Loops and copied iteration. **Grammar:** Until. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: us

Used in lexical and path spelling; see its grammar contexts and normative constraints.

Context: Lexical and path spelling. **Grammar:** Duration. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: useCase

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** Scenario. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: var

Used in attributes and lexical locals; see its grammar contexts and normative constraints.

Context: Attributes and lexical locals. **Grammar:** LocalVariable. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: verificationCase

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** Scenario. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: verify

Used in requirements and intent links; see its grammar contexts and normative constraints.

Context: Requirements and intent links. **Grammar:** Verification. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: via

Used in named routed case steps; see its grammar contexts and normative constraints.

Context: Named routed case steps. **Grammar:** ScenarioStep. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: view

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** ViewDef, View. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: void

Used in typed copied values; see its grammar contexts and normative constraints.

Context: Typed copied values. **Grammar:** TypeRef. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: wait

Suspends for a duration and releases the serial lock.

Context: Suspending wait and receive. **Grammar:** Wait. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: when

Used in ordered state transitions; see its grammar contexts and normative constraints.

Context: Ordered state transitions. **Grammar:** Transition. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: where

Used in seven library view definitions; see its grammar contexts and normative constraints.

Context: Seven library view definitions. **Grammar:** ViewSelection. **Name use:** reserved.

Normative rule · Graphical catalog.

Keyword: while

Used in loops and copied iteration; see its grammar contexts and normative constraints.

Context: Loops and copied iteration. **Grammar:** While. **Name use:** reserved.

Normative rule · Graphical catalog.

Symbols and precedence

Colon means typed by; returns names results. Dot is containment/member access; :: qualifies packages. Brackets express fixed extents, multiplicity or indexing. Parentheses group conditions/arguments and connector endpoints. There are no source arrows. Operators in ascending precedence are or, and, equality, numeric comparison, addition/subtraction, multiplication/division/remainder and unary not/minus. Only and/or short-circuit. Formal docs use /** */; ordinary comments remain lexical.

NORMATIVE / LANGUAGE CONTRACT

SWAML 2.0 language specification

Edition: 2.0.0 · **Parser:** 2.0 · **Reference profile:** swaml-reference/2.0 · **Status:** release edition, 22 October 2026.

SWAML is the Software Architecture Modeling Language: a textual language for explicit software structure, communication and executable behavior. ADR 0016 accepts the clean restart. The language has one component concept and one operation concept. It does not accept 1.x source. Published books and PDFs retain their original bytes.

1. Normative set

The concrete grammar defines tokens and parse structure. These chapters define validity and meaning:

1. Structure and types
2. Ports and communication
3. Operations, schedules and states
4. Deployment, cases, views and conformance
5. Proposed KerML mapping
6. Execution contract
7. Normalized model
8. Conformance obligations
9. Formal semantics

Graphical notation, its catalog and the mapping inventory complete the contract. Usage, quick start, AI authoring and examples are informative. Implementation status distinguishes implemented checks from contracts and future evidence.

2. Precedence and scope

MUST and MUST NOT state requirements; SHOULD requires a documented reason for deviation; MAY grants permission. Grammar acceptance is insufficient when prose constraints fail. Unknown syntax and unsupported editions MUST be rejected. The host selects parser 2.0 and pins its package revision; files are headerless. Formatting never changes validity.

The language is closed. No generic UML/SysML constructs, host code, dynamic topology, generics, overloading, multiple inheritance, recursion or global mutable values are implicit. Platform Nodes model placement identities rather than hardware execution. A simulator run does not prove schedulability, absence of races, universal verification, KerML conformance or graphical editability.

3. Invariants

- **I1 Boundary:** every exchange between different component occurrences crosses owned typed port members and an explicit connection/forwarding route.
- **I2 Ownership:** components own private attributes, children and operations. Operations own nested operations and flow or states. Nested operations never capture enclosing locals/parameters.
- **I3 Provenance:** every provided event/attribute has an emitting operation; every required event/attribute and provided operation has exactly one input owner. Forwarded boundaries have no local handler/emitter.

- **I4 Values:** payloads, arguments, results, out values and array iterations are copied. Definition identity never merges occurrence stores. Records/enums retain nominal identity.
- **I5 Policy:** each connection occurrence has a complete effective pinned policy and independent queues. Forwarding has neither. Attributes keep latest values regardless of event/request capacity.
- **I6 Presentation:** views select existing semantic identities; drawings do not create behavior or routes. Layout stays separate; declared timing and observed evidence remain distinguishable.

Local calls, startup, periodic releases, waits, state timers and same-owner after dependencies are local lifecycle/computation. Operator and test inputs use ordinary source components, emitting operations, ports and routes; no destination injection or simulator function-release API exists.

4. Inventory

Family	Ownership and meaning
Package, import	Flat multi-segment namespace and explicit visibility
datatype, enum, arrays	Finite nominal records/enums and copied fixed arrays
Component definition/usage	Reusable structure or distinct occurrence; usage body is anonymous specialization
Attribute/local/parameter	Occurrence state, lexical local, copied in or success-only out
Port definition/usage/member	Directed typed boundary; optional and reversed prefixes
Connection/forwarding	Policy-bearing opposite directions or queue-free boundary alias
Operation definition/declaration/usage	Reusable signature/body or owned executable, nested without captures
Schedule/entry	Trigger binding, dispatch arguments, deadline and concurrency
State/transition	Ordered nested receive/timer reactions; final completion
Deployment/placement	Software occurrences placed on platform Node occurrences
Requirement/case/view	Immutable intent, routed scenario and stable selection union

Definitions have qualified declaration identity. Occurrences have root plus indexed child paths. Activations add unique run identity. All transformations preserve these distinctions and source provenance.

NORMATIVE / LANGUAGE CONTRACT

1. Structure, names and types

Normative chapter of SWAML 2.0.

1.1 Names and documentation

Files contain packages without a version header. Package names may contain several :: segments; packages do not nest. import P; exposes P's top-level declarations and requires that package in the loaded workspace. Qualified references are unambiguous; ambiguous short imports are errors independent of load order. Names are unique in their scope. No overloads, implicit imports or escaped identifiers exist. The contextual words reversed, from, to, times and request remain legal names outside their syntactic positions. Other literal words in the grammar are reserved.

Identifiers use ASCII letter/underscore followed by word characters. Strings use double quotes and the portable escapes for quote, backslash, newline, carriage return and tab. Ordinary // and /* */ comments are lexical annotations. Each declaration owns at most one formal /** */ doc, first inside its body or immediately before the semicolon of a leaf. Documentation and comments do not execute.

1.2 Components and occurrences

component def declares a reusable type. component name: Type; creates a usage; a package-owned usage is an execution root. An untyped package usage may own inline topology. Child usages require definitions. Each root is checked independently for closure. Definitions may be reusable open libraries; an unused definition is not an execution root.

A usage body is an anonymous specialization with explicit whole-member replacement and added members, including typed roots. Single specializes inheritance is acyclic. An inherited member with the same name requires redefines; a nonexistent inherited target is an error. Replacement preserves category, types, operation signature and body category. Child component definitions may specialize the replaced type. Port replacement preserves directed compatibility and optionality. Attributes preserve constness and type. No partial operation-body merge, super dispatch or implicit shadowing exists.

A literal positive multiplicity [n] on a child creates n separate occurrences. Paths index every array usage with a literal in range, as in wheels[2].svc. Connector loops and port arrays do not exist. Composition is finite and acyclic. Definition identity does not identify an occurrence or a store.

1.3 State and lexical values

attribute declares component-owned mutable state; const attribute fixes its value per occurrence. Both are explicitly typed and may be redefined with a different compatible initializer in a usage. Initializers are pure and cannot read mutable state. Requirements own immutable typed attributes regardless of the const spelling.

var declares an activation/block local. in is the default parameter direction and is immutable; out parameters are mutable with success-only copy-back. Live names cannot shadow another live local, parameter or attribute. A nested operation sees its own parameters, its nested operations and the owning component's attributes, never enclosing parameters or locals. Local data access never traverses child components or ports.

1.4 Value domains

bool, byte (0-255), uint (0-9007199254740991), int (-9007199254740991 through 9007199254740991), finite real and string are primitive domains. void has no values and is only an operation result. Records and enums are nominal by qualified declaration identity. A datatype declares every field explicitly; a record construction supplies each once, with no extra fields. Enum literals use Type.literal; a live value named Type takes precedence, so a qualified enum reference avoids ambiguity.

Fixed arrays have positive safe extents, copied elements and invariant element type/extent. Array literals are checked elementwise against their destination type. Index expressions are integral; literal out-of-range indexes fail statically and dynamic bounds fail at runtime. In-place field/index assignment and reference values are absent.

Scalar widening is byte to uint to int to real. Known integral literals may initialize smaller integral domains if in range. General computed integral assignments are range-checked by the backend where permitted by the type rule; real never implicitly narrows. Integral arithmetic yields int; slash or a real operand yields real. Integer operations are exact within the safe domain and overflow fails. Division/remainder by zero and non-finite reals fail. Equality is recursive and type-compatible; ordering is numeric. Only and/or short-circuit. The grammar defines precedence.

1.5 Reuse and library boundaries

Package operation definitions use only parameters and nested operations. Their usages inherit the signature; an optional body replaces the whole body. Calls in an inherited component body resolve by name in the concrete occurrence's effective members, with signature preservation checked on redefinition. Call graphs are acyclic. No other component's operation or calculation is a fallback name resolution target.

The versioned source libraries are Swaml::Math, Swaml::Calls, Swaml::Qos, Swaml::Views and Swaml::Platform. They are always loaded but imported explicitly. Math provides abs, min, max, checked floor/round/trunc conversion to int, Unicode scalar length and concat. floor/round/trunc/length are explicitly identified host intrinsics in the reference library; round ties toward positive infinity. Their backend semantics and range failures remain required.

NORMATIVE / LANGUAGE CONTRACT

2. Ports and communication

Normative chapter of SWAML 2.0.

2.1 Directed port members

A port def owns named provides/requires attribute, event and operation members. An attribute carries one typed value, an event has copied in parameters, and an operation has in/out parameters and an optional returns result. Only operation members may be idempotent. Interfaces, interface kinds, versions, roles and separate signals do not exist.

A component uses port p: Definition; or an inline port body. reversed flips every member direction. optional is a prefix that waives that port's closure requirement; it does not waive required ports elsewhere on a forwarding chain. A provided attribute/event is emitted by this side; a provided operation is implemented by this side. Requires means the opposite.

Every provided attribute/event MUST have syntactic operation emission provenance. Every required attribute/event and provided operation MUST have exactly one owner: a schedule entry, one stateful operation's transition set, one receive statement or one interrupt region. Multiple alternative transitions in that one stateful operation are one owner. A forwarded boundary is never also locally handled or emitted.

All input stimuli, including operator/test drivers, are ordinary typed emissions from validated source operations through routes. Hidden harness releases and destination injection are forbidden. An unconnected optional request fails unavailable. An optional unconnected event/attribute emission has no recipient and remains observable.

2.2 Structural compatibility

Connection compares names independently of declaration order or port-definition identity. Every requires member on either side has a provides member of the same name and kind on the peer. Extra provides are permitted. For attribute/event delivery the provided payload is assignable to the required payload. For operation in parameters the required type is assignable to the provided type; out parameters reverse this variance. The provided result is assignable to the required result. Directions and parameter arity agree. Records and enums are nominal; numeric widening is permitted. Every missing/mismatched member produces an individual diagnostic.

Forwarding compares a boundary and immediate child's port in the same direction. The child supplies every boundary member with preserved direction and compatible payload/signature. Child requirements cannot be lost at the boundary. This aliases identities rather than generating messages.

2.3 Connectors and closure

connect (a, b); joins owned ports or ports of immediate children. A child with multiplicity requires a literal index. Name and policy are optional: connect link: Reliable (a, b);. Endpoints are unordered.

Boundary-to-child endpoints derive forwarding when same-direction compatibility holds. Forwarding has no queue or policy; a policy is an error. Each boundary and child port is forwarded at most once. Other endpoints derive a connection using opposite-direction compatibility. Invalid endpoints report the applicable compatibility failures. Self connections and duplicate endpoint pairs, including duplicates after forwarding resolution, are errors.

Every nonoptional port of each concrete root MUST resolve through any forwarding aliases to a connection. A required attribute or operation has exactly one peer after forwarding. Event-only required ports permit multiple peers. A connector owns its queue identities per expanded occurrence, direction and member; sharing a named policy never shares queues. Distinct connections remain independent under fanout. Partial fanout effects committed before a failed offer remain committed.

2.4 Versioned policies and queues

A named qos def is typed by Swaml::Qos::Policy and supplies complete valid delivery, ordering, capacity, overflow and requestTimeout fields, plus optional maxLatency/maxAge. Values use the typed Delivery, Ordering and Overflow enums. Inline policies and per-use overrides do not exist. Each effective use is checked, including inherited connections and idempotency obligations.

Default is atMostOnce, fifo, capacity 16, reject, requestTimeout 1s, without latency/age bounds. Latest is atMostOnce/fifo/1/dropOldest/1s. Reliable is atLeastOnce/fifo/16/reject/1s. BestEffort is bestEffort/unordered/16/dropNewest/1s. These are pinned library values, always inspectable. atLeastOnce requests require an idempotent provided operation declaration; the declaration alone does not prove actual idempotence.

Policy buffering applies to events and requests. Attribute members keep only the latest pending value independently of capacity and overflow. fifo preserves order within its queue. Unordered allows other orders, although the deterministic reference interpreter chooses insertion order. Overflow rejects the offer with a runtime failure or discards the selected oldest/newest message observably. Age discards are observable; latency assumptions are contract obligations. A backend identifies delivery and failure assumptions explicitly.

2.5 Requests, replies and failure

A required operation is called as p.member(args). Its timeout comes from the route policy. Each request has a unique correlation and its response returns on the same member/route in reverse. Successful result and out values are copied back together; failure copies neither. Timeout, unavailable, rejected, transport and remoteFailure are failure reasons recorded as Swaml::Calls::CallFailure values or local runtime kinds in evidence. The model cannot inspect the reason.

A request suspends and releases the caller's serial lock. Timeout and reply at the same timestamp resolve as timeout; terminal correlations never reopen. Late replies are suppressed observably. Interruption cancels the local request continuation and correlation; the server continues and its committed effects remain. The interrupted terminal is not a catchable failure. try catches expression failures and supplies a compatible lazy fallback; bare calls propagate failure to the activation.

NORMATIVE / LANGUAGE CONTRACT

3. Operations, schedules and states

Normative chapter of SWAML 2.0.

3.1 Operations and effects

An operation declaration always has parentheses and an optional returns T result; omission means void. operation def is package-owned reusable behavior. An operation usage has a type after a colon, no parentheses/result, and optionally replaces the whole inherited body. Names are unique and call graphs acyclic. Nested operations precede executable statements/states and never capture enclosing locals/parameters. A body owns flow or states directly; the body grouping has no executable identity.

Calls are expressions. A void call is a statement; a non-void result cannot be silently discarded. Calls with out arguments are statements; each out argument is a mutable compatible local, out parameter or component attribute. In/out values are copied and out copy-back occurs only after successful whole return. A caller cannot invoke a different component by a structural path; it must use its own required port operation. Stateful operations are activated by their schedule rather than called locally.

Effects are inferred transitively through the acyclic call graph: pure, reads state, writes state, communicates, suspends. The implementation MUST also retain a writes flag when a higher effect dominates, so communicating/suspending callees cannot conceal a write. Passing an attribute as out is a write. Attribute initializers are pure. State guards may only be pure or read state. Concurrent schedule entries cannot write state through any reachable callee or out argument.

Suspending calls may be the whole var initializer, assignment right side or expression statement, optionally wrapped in try; they cannot be embedded in arithmetic, call arguments, return values or predicates. Other expressions can have nonsuspending effects. Arguments evaluate left to right; only and/or short-circuit. Declarations and source order do not select overloads or change resolution.

3.2 Failure

Runtime failures unwind the current expression/frame/activation without rolling back committed effects. try(e, fallback) catches all runtime expression failures from valid source, including arithmetic, ranges, indexes, assert/fail in called operations and remote failures. Fallback is assignable to the expression type and evaluated only on failure. Nested try is valid. Void try(e) explicitly ignores failure; a void fallback can run recovery behavior.

Static errors are never caught. Interrupts and deadline/activation/resource violations bypass try. The failure reason is recorded in evidence and unavailable as an implicit binding. Void has no value; completion is not a sentinel value. Non-void flow must structurally return or fail on every path; a positive-extent copied for-in body or an until body with a proven positive/no limit can establish this, while a dynamic possibly-zero limit cannot; bounded/unbounded loops that can execute zero times are insufficient to establish a return.

3.3 Statements and scopes

var declares a typed local; assignment replaces a copied mutable cell. if requires bool. while tests first; until executes at least once unless its explicit limit is zero, and stops when its bool condition becomes true. Optional n times takes a uint-compatible expression evaluated once at entry; reaching the limit exits normally. for uses an initializer-local scope, bool condition, body and assignment update. for (x: T in values) iterates one copied fixed-array snapshot with a fresh element/body scope each time. Locals do not leak from blocks. There is no break, continue, do, bound or forEach syntax.

wait D suspends for a positive duration. on receive p.m(bindings) { ... } or receive ... or after D { ... } waits for one input or optional timeout. Each payload is explicitly bound, each member has that statement as exclusive owner, and input received before the statement waits stays in the connector queue. Without

timeout the wait is unbounded, subject to an entry deadline and host limits. Selecting a branch consumes one message once.

interruptible { body } on receive p.m(bindings) { handler } arms an unnamed region only while the protected body, including its called frames, is running or suspended. Input while inactive is discarded and traced; the validator warns. Input while armed transfers within the same activation, cancels protected continuation/local request, preserves outer locals/deadline and committed effects, and then executes the handler. Protected block locals do not escape. Nested regions unwind to the selected owning region. A server handling a cancelled request is not cancelled. Regions are prohibited in stateful operations; concurrent regions are permitted only when transitively non-writing and consume input once in activation order in the reference backend.

send p.attribute(v) updates a provided attribute; publish p.event(args) emits a provided event. Return completes a frame, fail fails it, assert requires a bool and fails when false. Private state writes, emissions and successful local call effects remain committed when a later statement fails.

3.4 Suspension and locks

Every component occurrence has one serial lock. Serial dispatch evaluates/copies schedule arguments under this lock. wait, requests, receive and idle state-machine waits release it. Resumption reacquires it and reads live attributes. Other activations may run and modify state while a serial activation is suspended. Activation locals remain its own copies. Concurrent entries overlap and read the dispatch snapshot throughout, including after resumption, and never write state.

A stale-read warning identifies a pre-suspension state read that may affect a later write. Recheck live state after resuming. This warning is conservative and is not a race-freedom proof. A successful static check does not establish scheduling, termination or deadlock freedom.

3.5 Schedule

A component owns at most one schedule block. Effective entries inherit and are replaced by repeating their (trigger, operation) identity. Duplicate identities in one authored block are errors. Trigger identity includes a receive/call member, an every period, or an after predecessor. One operation can have several entries.

Trigger	Release and signature
on start	Once after the initialization barrier
on every D	At root epoch + kD, k at least 1; deadline defaults to D
on receive p.m(bindings)	One activation per received required event/attribute
on call p.m(bindings)	Serve a provided operation; typed result/out values form the reply
after predecessor	Fresh activation after successful whole same-owner predecessor completion and unlock

Entry arguments evaluate and copy at dispatch. Non-call entries bind void flow with no out parameters; after targets are parameterless void flow. Predecessors are scheduled same-owner operation names, never paths/qualified names/port members; the effective after graph is acyclic. Failure releases nothing. A fresh activation has its own release/deadline/locals/snapshot/backlog accounting; results and parameters are not forwarded. Stateful success means reaching final, not becoming idle after a reaction.

An explicit deadline is positive. An every entry's deadline cannot exceed its period. Deadline omission elsewhere is permitted; a host may impose additional documented validation requirements. concurrent is entry-local and requires a non-writing, non-stateful operation. Source WCET, timing blocks and root profiles do not exist; target bindings own execution-cost evidence.

3.6 States

A stateful operation is parameterless void and has exactly one on start entry. initial names one owned state. States own at most one on entry and one on exit block plus ordered receive/timer transitions. A receive transition binds payload, optionally tests a pure/read-state bool guard, and names its target with to. after D is relative to the state update, before entry actions begin. Entry actions finish before a transition can run; if entry suspends past a timer expiry, the earliest elapsed timer is selected on resumption before queued input. Timers with equal expiry use declaration order. Evaluate competing enabled transitions in declaration order and take the first. There are no transition names, priorities or completion transitions.

A reaction runs exit, transition effect, state update and entry in that order. Timers are invalidated by state exit and stale firings are observable discards. Unhandled input does not reset the current state timer. Every state is graph-reachable from initial; a final state has no outgoing transitions or exit action. Reaching final runs entry and completes the whole operation once. Hierarchical/orthogonal states and physical preemption are outside the language.

NORMATIVE / LANGUAGE CONTRACT

4. Deployment, cases, views and conformance

Normative chapter of SWAML 2.0.

4.1 Deployment

Swaml::Platform supplies Node, Platform, Partition and Process component definitions. Node specialization owns only const attributes and child Nodes, with no operations, ports, connectors or schedule. Optional ref: string retains an external model identity for the host; it is not required for SWAML validity and is never silently resolved as a software path. Buses, hardware timing, core/task allocation and operating-system scheduling are target-translation concerns.

A package deployment Name of Root on PlatformRoot owns placements. deploy on nodePath places the software root; deploy childPath on nodePath places that occurrence and descendants. A more specific path overrides an ancestor; an indexed path overrides a multiplicity family path. Exact paths are unique within one deployment. Every software path and literal index resolves in that root; every target resolves to a Node occurrence of that platform root. Only components are deployed. Several deployments may refer to one root and a host selects one explicitly; absence remains unallocated.

4.2 Requirements and cases

A requirement owns immutable typed attributes and at most one doc. satisfy Requirement of designPath records design intent. Only verificationCase owns verify Requirement. Neither relation proves fulfillment. A useCase or verificationCase selects one root with of; it owns routed steps and optional documentation, never behavior or hidden actors that bypass ports.

A step has kind name from source to target, optional via connector and optional after step. Kinds are send, publish, request and reply. Source/target are occurrence port-member paths in the case root. Direction, kind and payload/signature compatibility must match a real resolved connection. via names a connector; it may be omitted only when exactly one connection joins the endpoints. Unnamed connectors cannot be selected by name. Forwarding aliases are resolved and do not create a case message.

Names are unique. An after predecessor names an earlier step; source order otherwise contributes display order only. A reply identifies one unanswered causal request ancestor with reversed endpoints, same member and route. A reply on an independent branch cannot answer another branch's request. Multiple replies to one request or ambiguous causal candidates are errors. Partial scenarios may leave a request unanswered. Route-valid scenarios are intent until an identified witness execution supplies payloads, assumptions and trace evidence.

4.3 Views

Swaml::Views provides General, Interconnect, Case, Sequence, Grid, Flow and Timing. A view is typed by a definition, uses compact of subject or one or more include clauses, and owns optional documentation. Include may expand recursive descendants and apply a restricted Boolean where filter over name, kind and path, string comparisons, and/or/not. Filters never call operations or execute behavior. Selection is a union of occurrence identities in first-selection order; duplicates are removed stably.

General and Grid select named semantic elements except views. Interconnect selects components. Case/Sequence select roots or cases. Flow/Timing select operations. Paths preserve definition/occurrence identity and explicit multiplicity indexes. Browser is navigation rather than an eighth view kind. General Box in box and Tree layouts retain exactly the same selected identities and topology; layout/collapse/geometry stay in presentation sidecars.

Flow covers statements, nested operation calls, loops, receive, interruption and function-owned states. Timing has declared and observed modes. Declared shows schedule release grid, causes, deadlines, static wait intervals and permitted concurrent overlap. Observed adds an identified trace, serial lock gaps, message latency, activation phases, terminal markers and open intervals; WCET/budget bars identify their target-binding/evidence source. A missing trace never produces fabricated observation. Views introduce no execution semantics.

4.4 Validation groups

Code	Static group
SW2000	Syntax and recovery diagnostics in machine-readable CLI output
SW2001	Names, imports, uniqueness and forbidden live-name shadowing
SW2002	Value types, nominal shapes, domains, arguments/results and indexes
SW2003	Definition/usage, specialization, containment and whole-member redefinition
SW2004	Port shape, structural compatibility, endpoints, aliases, closure/cardinality
SW2005	Single input owner and emission provenance
SW2006	Calls, nested scope, effects, returns, out arguments and suspension placement
SW2007	Schedule identity, signatures, deadlines and after graph
SW2008	Stateful signatures, scheduling, ordered states and reachability/finality
SW2009	Node content, deployment paths, indexes and specificity
SW2010	Requirement/case/view references, direction, correlation and subject rules
SW2011	Conservative stale-read warning
SW2012	Inactive interrupt-input discard warning
SW2013	Complete typed policies and representable positive durations
SW2014	Canonical formatting differs, only in explicit formatting-check mode

Every diagnostic includes a source range and stable code. Machine-readable diagnostics retain declaration identity and occurrence context when resolved. A suggested edit is supplied only for a safe unambiguous correction; tools must revalidate the resulting complete workspace before applying it. Parser recovery may suspend semantic checks on invalid syntax, but the document remains invalid and must not crash the service.

4.5 Evidence boundaries

S is syntax, V is static validity, E is reference execution behavior, A is an explicitly scoped analysis, K is validated kernel export and G is validated graphical source editing. Implementations report supported groups and evidence rather than promoting one level to another. Chapter 8 defines the obligations and status reports actual support. The reference simulator uses logical zero-cost execution and bounded resources; target deployment timing requires separate evidence.

NORMATIVE / LANGUAGE CONTRACT

5. Proposed KerML semantic mapping

Normative mapping contract of SWAML 2.0, conditional on a backend claiming K conformance. No exporter or validated KerML semantic library is supplied by the current reference implementation. Langium AST generation and exhaustive inventory coverage do not establish KerML conformance.

5.1 Representation

Packages map to Package/namespace membership and imports. Datatypes/enums preserve nominal identity; arrays preserve copied shape. Components map to software Structure definitions and owned typed features. A usage creates an occurrence, including each multiplicity index; no shared private state is introduced. Specialization/redefinition retains the effective member set and base/replacement provenance.

A result-bearing operation maps to a Function; a void operation maps to a Behavior. This mapping does not make a result-bearing SWAML operation pure. Effects remain explicit SWAML constraints. Operation usages retain their shared signature and effective body. Nested operations are owned executables without lexical captures; states contribute to their owning operation rather than becoming an independent executable.

Ports map to owned typed features with per-member directional projections. Reversal flips each projection. Structural compatibility is retained as per-member typing constraints, not replaced by nominal port identity. Connections map to Connector instances with source occurrence/member route and independent policy queues. Forwarding maps to queue-free boundary aliases with preserved chains, not an extra message hop. Messages, transfer values, correlation and terminal failures require SWAML semantics over suitable KerML Step/transfer structures.

5.2 Required semantic library

A K-conforming backend MUST implement a pinned SWAML semantic library specializing the selected KerML 1.0 library. It defines Component, Operation, Port, Connector, Message, Queue, Activation, Schedule, state-machine reactions, copied values, failure and interruption. It MUST preserve suspension/unlock/relock, snapshots, deadlines, queue identity, success-only out copy-back, after success releases, state transition ordering and cancellation/late-reply behavior. Merely attaching uninterpreted metadata is preservation rather than proof that a kernel tool understands the execution semantics.

No arbitrary metaclass name is invented as a kernel class. Abstract bases require concrete backend choices validated by that kernel. Unsupported meaning yields an explicit partial-export result. The inventory covers every grammar production/type as element, grouping, reference, value, lowering or lexical, with its normative home. Groupings/lexical nodes contribute to owners and must not create duplicate semantic elements.

5.3 Traceability and claims

Every lowered node/constraint retains source declaration identity, URI/range, occurrence context and a generation role. Inherited declarations and replacements retain both provenances. Requirements, satisfaction, verification, deployment, cases and views retain typed metadata/references, without proof claims. Timing observations reference their run and binding; layout stays separate.

A conforming backend identifies the exact source/package/library revisions, selected KerML implementation, checked constraints and evidence. Round trips preserve the normalized model and cannot erase effects, merge occurrences/queues, reorder state transitions, invent routes, weaken contracts or convert copied values into references.

NORMATIVE / LANGUAGE CONTRACT

6. Reference execution contract

Normative contract of SWAML 2.0, profile **swaml-reference/2.0**. A run records parser/package identity, source bytes, root, selected deployment, options, limits, fault assumptions and an ordinal logical trace. Host adapters load files and hash sources; the language core performs neither filesystem access nor hashing.

6.1 Lifecycle and time

Expand the finite occurrence tree and initialize separate private stores before releasing startup work. An uncaught initializer runtime failure ends initialization, records the occurrence, attribute, source range and reason, retains partial stores and source evidence, and releases no startup work. A caught initializer failure may recover and remains observable. All roots are independent execution choices. The reference epoch is 0ns. Every D releases at kD for k at least 1. Schedule arguments evaluate and copy at dispatch, under the serial lock or the concurrent dispatch snapshot. Entry deadline is measured from release, including backlog, request transport, wait and receive. Periodic deadline defaults to period.

Time uses nonnegative safe integer nanoseconds. Durations are positive and must remain representable after conversion/addition. Reference CPU steps cost zero physical time; a step budget prevents runaway computation. Ordering is deterministic by timestamp, phase and insertion ordinal. Deadline/request timeout wins a tie with reply or ordinary work. Receive timeout wins a tie with delivery in the reference selection. A run explicitly records its horizon; live/waiting activations and future events yield horizon status with open intervals rather than invented completion.

6.2 Lock and snapshot

One serial activation runs at a time per component occurrence. Nonsuspending local calls borrow its context and lock. wait, requests, receive and idle state waits release the lock. On readiness, serial work reacquires it before continuing and uses live attributes; locals remain copied cells. Concurrent entries may overlap, read one dispatch snapshot throughout and cannot write state even through callees or out arguments. Target backends must state how they realize the same snapshot semantics.

After a successful whole operation completes and unlocks, each effective after entry releases one fresh activation. No release occurs on failure, idle state reaction or intermediate local return. Each dependent has fresh deadline, locals and snapshot/backlog accounting. Correlated service reply transport and local completion dependencies are distinct observations. Duplicate dependencies are statically invalid and the graph is acyclic.

6.3 Values, flow and failure

Evaluation is left to right with and/or short-circuit. Payloads, records, arrays, parameters/results and out copy-back are copied, with nominal identities and domain checks. for in uses one copied collection. A loop limit evaluates once, is checked in the uint domain, and exits normally at the count; termination beyond declared limits is not statically proved. Integer overflow, non-finite reals, zero division, invalid indexes and failed assert/fail are runtime failures.

A local failure unwinds frames to the nearest try or fails the activation. Try fallback is lazy and type-compatible; prior state writes/emissions remain committed. Out values copy back only on successful return. Interrupt transfers and deadline/resource/activation violations are outside try. Resource failure is explicit evidence; no silently dropped activation is reported as successful execution.

6.4 Communication

Resolve boundary aliases before routing; forwarding adds neither queue nor latency. Each connection occurrence/direction/member owns a queue. Bare connections use the pinned Default. Check outgoing arguments against the emitting/requesting port member before routing, even when the peer accepts a wider domain. Request out cells initialize in the provider domain; successful replies convert at caller copy-back. Attribute queues replace the pending value; event/request queues obey capacity, delivery, ordering, overflow and optional age/latency assumptions. A reference run chooses insertion order for unordered queues. Fanout proceeds in route order and may partially commit before rejection.

A required request creates a unique pending correlation and suspends the caller. Result/out values and failure return through the same route in reverse. Timeout uses requestTimeout. First terminal transition is immutable; timeout wins equal-time delivery; late replies are traced and suppressed. AtLeastOnce may redeliver and requires an idempotent declaration. The reference fault model records offer-indexed drop, duplicate, transport and additional latency; retries/redelivery assumptions must be inspectable. Each duplicate copy independently obeys queue capacity and overflow; attribute copies preserve the single latest pending value. Accepted input comes only from validated source emissions; no destination injection exists.

6.5 Input ownership and interruption

An event is consumed by its one owner. Input for a receive statement remains queued before that statement waits. Inputs for inactive interrupt regions are discarded and observed. An armed region includes suspended local callees; matching input cancels protected frames/continuation and any pending local request, but not a remote server. Transfer stays in the same activation/deadline, preserves outer locals and committed effects, and runs the handler. Terminal interrupted correlations suppress late replies. No physical scheduling/preemption guarantee is implied.

A state machine waits without the component lock. Each reaction selects the first enabled nested transition for the input/timer, runs exit/effect/state-update/entry, and releases the lock again when idle. The timer epoch is the state update before entry actions, including time spent suspended in entry. Entry completes before timer selection; elapsed timers precede queued input, with earliest expiry first and equal-expiry transitions in source order. Deadline ties still win. State entry owns timer generations; stale timers cannot fire after exit. Unhandled input does not restart a timer. Final entry completes the whole operation once; it can release after dependents.

6.6 Evidence and resources

Trace ordinals distinguish equal-time observations. Record initialize, release, dispatch, ready, unlock, wait/requestWait/receiveWait, messages/offers/delivery, writes, states, interruption, completion/failure, caught failures, timeout, overflow/discards, late replies and violations. Retain activation/correlation/route identities where applicable. Final stores are occurrence-indexed and timing projections leave live intervals open. No-failure-observed means only that finite run under its recorded assumptions. Step, backlog and concurrent limits are finite, explicit options. Production timing, real transport guarantees and universal verification remain separate claims.

NORMATIVE / LANGUAGE CONTRACT

7. Normalized model contract

Normative information-preservation contract of SWAML 2.0. It defines what any exporter/target adapter must retain; it does not claim a kernel exporter exists.

7.1 Identity and provenance

A declaration identity is package plus owned-name path. An occurrence identity is root plus indexed containment path; an operation occurrence additionally identifies its effective declaration. Activations/correlations/messages add unique run identities. Anonymous syntax/lowering nodes use an owner plus structural slot/generation role, not an invented user declaration. Source ranges are provenance rather than persistent identities. Every semantic node except a root has one owner; reference edges never imply containment.

Each normalized node/constraint retains source declaration URI/range and any inherited/replacement provenance. A persistent editor identity map is separate, preserving renames/moves atomically with source references and sidecars. Same-type multiplicity occurrences never share stores, queues or activation identity.

7.2 Families

Family	Required content
Workspace/package	Edition, package revision, sources, imports, unique declaration namespaces
Record/enum/type use	Nominal identity, fields/literals, primitive domain, fixed array shape
Component definition/usage	Base, effective members, explicit replacements, anonymous usage specialization, multiplicity
Attribute/local/parameter	Owner/scope, explicit type, const/in/out, initializer, copied-cell lifetime
Operation	Signature, effective whole body, nested ownership without captures, inferred effect flags/class
Port/member	Definition or inline members, direction, reversal, optionality, payload/signature, idempotency
Connection/forwarding	Endpoint occurrence references, alias provenance, structural member checks, effective policy
Policy	Pinned complete values and declaration/use provenance; no shared runtime queue
Schedule/entry	Trigger identity, effective target, dispatch arguments/bindings, period/deadline, concurrent
State/transition	Initial/final, entry/exit/effects, receive bindings/guard or timer, ordered selection
Deployment	Software/platform roots, Node paths, family/index placements, effective specificity
Requirement/case/view	Immutable attributes/docs, intent links, route/dependency/correlation graph, ordered selection union

Expansion creates every literal multiplicity occurrence independently and keeps the definition reference. Inheritance produces an ordered effective set; replacement points to its inherited declaration. Late-bound

local calls use that occurrence's effective operation name with the preserved signature. An operation usage retains the definition signature and either inherited or replacement body; no partial merge is inferred.

7.3 Routes and queues

Each port occurrence has per-member projections with effective provides/requires direction. Resolve forwarding to leaf port identities, retaining the full boundary chain without queue/storage/hop. Every resolved input projection references its unique owner and every provided attribute/event its emission sites. Outgoing operation projection is a request rather than an emission declaration.

A normalized connection records the two leaf ports, per-member directed routes and complete policy. Queue identity is connection occurrence, source/destination direction and member; requests/replies retain correlation separately. Attribute pending values use latest replacement, not event overflow rules. Extra provides remain contract members without fabricated peer requirements. Structural checks preserve each failed/matched member and nominal payload identity.

7.4 Executable lowering

Statements retain ordered operands, scopes, effects and success/failure continuations. Calls are expression nodes with a resolved local/port target, copied in/out/result types and effect flags. Try is a failure-catching node with a lazy fallback, never a rollback boundary. Wait/request/receive creates a suspended continuation plus unlock/readiness/relock; concurrent reads keep the captured dispatch snapshot. Loop limits evaluate once; for in owns one copied collection. Interrupt regions retain armed extent, input ownership, same-activation unwind, request cancellation and inactive/late-input observations.

A schedule entry lowers to a release mechanism and dispatch-time argument evaluation. After edges retain same-owner predecessor operation and completed activation identities, success-only release after unlock, fresh deadline/context and no implicit data forwarding/join. Stateful reactions retain source-order transitions, guards, timer generations and exit/effect/update/entry phases. State bodies do not gain separate executable identities. Final entry completes the operation once.

7.5 Presentation and equivalence

Views normalize to library definition plus ordered selected occurrences, retaining original include/filter clauses. General layouts preserve the same graph. Timing observations retain exact source/run/root/activation/correlation identities and open intervals; declared contracts remain independently identifiable. Geometry, collapse, routing bends and camera stay in sidecars.

Equivalence requires an identity-respecting correspondence preserving ownership, types/shapes, effective members, routes/policies, evaluation/effect order, schedule releases, suspension, failures, cancellation, states, deployment, intent links and selected occurrence order. Whitespace/ordinary comments are outside execution equivalence but remain required for authoring round trips. Exporters cannot drop unsupported semantics silently or claim that carrying metadata supplies execution/kernel evidence.

NORMATIVE / LANGUAGE CONTRACT

8. Conformance obligations and evidence

Normative chapter of SWAML 2.0. Acceptance levels are separate; implementations report the tested subset and limitations in their support matrix.

8.1 Static obligations

ID	Required validation	Diagnostic group
V01	Headerless pinned edition, portable tokens, malformed recovery and literal domains	syntax, SW2000/2002
V02	Unique names, explicit imports, unambiguous qualified/short references, no live-name shadowing	SW2001
V03	Finite nominal records/enums/fixed arrays, complete typed values, no void data, index ranges	SW2002
V04	Component-only definitions/usages, finite containment, acyclic single base, explicit compatible whole replacement, typed root additions, positive multiplicity	SW2003
V05	Directed port shape, reversal, nominal payloads and structural variance with per-member diagnostics	SW2004
V06	Immediate-child indexed routes, derived forwarding/connection, policy-free unique aliases, no duplicate resolved routes, root closure/cardinality	SW2004
V07	Every required event/attribute and provided operation has one input owner; provided event/attribute emission provenance	SW2005
V08	Operation signatures, no nested captures, typed calls/out/result, acyclic graph, return paths, pure initializers	SW2002/2006
V09	Transitive effect inference, read-only guards, non-writing concurrent entries, suspension only in allowed whole expressions	SW2006

ID	Required validation	Diagnostic group
V10	One schedule, unique effective identities, matching trigger/binding signatures, same-owner acyclic after, fresh void targets and period/deadline rules	SW2007
V11	Stateful parameterless void signature, exactly one start entry, reachable states, ordered transitions and final restrictions	SW2008
V12	Typed complete policies, effective-use idempotency, positive safe time/capacity domains	SW2013
V13	Node-only target content, root/occurrence/index placement resolution and unique exact paths	SW2009
V14	Immutable requirements, legal satisfy/verify, routed cases, causal unique replies, seven library view kinds/subjects and stable selection union	SW2010
V15	Stale-read and inactive interrupt-input warnings without a race-freedom claim	SW2011/2012

Validation MUST rebuild semantic string paths as well as indexed references after workspace changes/additions/deletions. Negative lookups matter: a newly loaded declaration may make a previously resolved short name ambiguous. Malformed source remains editable and invalid; recovery cannot silently accept a different program. Definite-return analysis may be conservative rather than solving arbitrary conditions. Dynamic numeric/index/termination failures remain runtime obligations.

8.2 Runtime obligations

ID	Required observable evidence
E01	Distinct occurrence/multiplicity stores, copied values, activation-local scopes and successful out copy-back
E02	Initialization barrier, dispatch arguments, serial lock release/reacquisition and concurrent dispatch snapshots
E03	Separate occurrence/direction/member queues, Default policy, latest attributes, overflow/order/age/delivery behavior
E04	Unique routed correlations, result/out reply, failure propagation, timeout tie and terminal late-reply suppression
E05	Ordered expression/flow execution, loop limits, copied iteration, checked arithmetic/indexes, lazy try without rollback
E06	Ordered state reactions, live guards, timer generations, inactive input and final whole completion
E07	Entry release/deadline accounting, finite step/backlog/concurrent resources and explicit violations outside try

ID	Required observable evidence
E08	Armed interrupt extent across callees/suspensions, same-activation unwind, terminal cancellation and no server rollback
E09	One fresh after activation per successful whole completion after unlock, failure suppression and new context/deadline
E10	Run/source/root/profile/option provenance, trace ordinals, occurrence stores and open observed timing intervals

A deterministic reference run may choose one permitted order/delivery outcome. Its tests do not establish all possible traces or real transport reliability. A target implementation identifies additional scheduling/transport assumptions and evidence. Idempotency annotations are checked declarations, not proofs of side-effect behavior.

8.3 Analysis, kernel and graphical claims

Case realization needs a witness execution with payloads/assumptions. Requirement fulfillment needs identified test/analysis/review evidence beyond satisfy/verify links. Schedulability needs target scheduler, cost and transport assumptions plus measurement/proof. Race/deadlock freedom requires analysis of suspension/interleaving and requests. A finite search without failure is not universal proof.

K requires the complete normalized model, a pinned semantic library/exporter, kernel validation and round trips. Inventory coverage is not K. G requires Browser plus seven source views, occurrence-aware identity, notation, source-validated edits, sidecar layout and atomic native undo/redo. LSP syntax/formatting alone is not G. Semantic paths need path-aware rename/reference/navigation; generic name references are insufficient.

Conformance reports identify edition, exact implementation revision, groups checked, unsupported capabilities, run limits and evidence links. Implementation status is the reference report; it must change when behavior or evidence changes.

NORMATIVE / LANGUAGE CONTRACT

9. Formal semantics

Normative schemas of SWAML 2.0. These constrain implementations without claiming a mechanically proved metatheory.

9.1 Domains

Let D be the resolved declaration/effective-member graph, O the finite indexed occurrence tree, Γ a typed lexical environment, σ occurrence stores, ρ activation locals, Q per-connection/direction/member buffers, A activations, M state/timer generations, R correlations, L serial locks, B pending releases and t integer nanoseconds.

```text
C = (D, O, σ, Q, A, M, R, L, B, t)
D ⊢ O valid
D; Γ ⊢ e : T ! ε
D; Γ ⊢ s ⇒ Γ'
C  - observation→ C'
```

Values are bounded primitive scalars, nominal enums/records and fixed copied arrays; void has no inhabitant. Effect ε retains read/write/communication/suspension flags, with pure as empty. Transitive union over an acyclic call graph computes effects. The maximum class is a presentation of these flags rather than permission to erase a write beneath suspension.

9.2 Typing schemas

Construct	Premises and result
x / x.field / x[i]	x in Γ; nominal record field or integral array index; result declared type; dynamic index must be in extent
Enum.literal	No live first-segment value shadows the enum; unambiguous accessible nominal enum and literal
operation(args)	Resolved same-owner/nested or imported pure definition; copied typed in arguments; mutable compatible out targets; result declared type; effect is union of arguments and callee
p.member(args)	p is owned, member requires operation; arguments satisfy signature; effect communicates and suspends
try(e, f)	e:T and f:U assignable to T; lazy f; effect union; no fallback only when T is void
var x:T=e	Compatible value, fresh live name; Γ extends with mutable x; suspension allowed only when e is whole call/try
x=e	Mutable local/out/attribute, compatible type; attribute write adds write effect; same whole-expression suspension rule
return e	e assignable to owning operation result; expression does not suspend; success copy-back
if/while/until/assert	bool condition; optional loop limit uint-compatible and evaluated once
for	Fresh initializer scope; bool condition; mutable typed update; fresh body scope each iteration

Construct	Premises and result
for in	Collection T[n], explicit compatible element type; copied collection; fresh element/body scope
wait/receive	Suspended continuation; owned input with explicit payload binding for receive; no implicit failure/payload name
interruptible	Non-stateful operation, exclusive required input; protected and handler scopes share only outer locals
state guard	bool, effects pure or reads state; transitions selected in authored order
concurrent entry	Callee transitive effect has no write and body is not stateful
attribute initializer	Explicit destination type; empty state environment; pure effect

Scalar type assignability uses widening, with checked integral value conversion in assignments/parameter initialization. Port compatibility is structural by member name with contravariant operation inputs and covariant output/result payloads. Nominal records/enums do not become structurally equivalent. Array values preserve invariant element type/extent; literals are checked elementwise.

9.3 Transition schemas

Rule	Configuration change and observation
initialize	Expand O, create separate σ[o], initialize all before startup release
release	Add fresh activation to B with release time/cause/deadline; bounded admission or explicit violation
dispatch	Acquire L[o] for serial work; copy entry arguments under lock; concurrent work captures σ[o] snapshot
local step	Evaluate in ρ plus live serial σ or captured concurrent snapshot; update copied cells and emit source-provenanced observations
suspend	Save continuation/ρ; remove serial lock; register wait timer, receive alternatives or pending request
ready	Mark continuation ready; serial continuation must reacquire lock before reading current σ
offer	Resolve alias chain, select connection/member queue, copy payload, apply latest attribute replacement or policy overflow/delivery
request	Create unique pending R[r] with expiry t + requestTimeout; offer request; suspend caller
reply	Before expiry and while pending, set immutable succeeded/failed terminal, copy success result/out, ready caller
timeout	At expiry, pending becomes failed; ready failure continuation; tie beats reply
try success/failure	Return e value on success; on RuntimeFault evaluate fallback lazily with existing σ/Q effects retained
interrupt	Match active region; disarm/unwind protected continuation, cancel pending correlation as interrupted, retain activation/deadline/outer ρ and committed effects

Rule	Configuration change and observation
state reaction	Select first enabled transition, exit/effect/update/entry, invalidate old timer generation; idle releases lock
complete	Finish activation and unlock; correlated reply is offered separately; successful whole completion releases after targets fresh
violation	Deadline, activation overflow, step/concurrent resource or clock failure stops the reference run and bypasses try

At terminal correlation, later replies cannot change R. Inactive interrupt input is discarded observably. Receive statements preserve queued early input. A state's unhandled input leaves its entry time/timer generation intact. Set entry time at the state update, before entry actions. Select an already elapsed timer after entry completes before queued input; earliest expiry then declaration order determines competing timers. Deadline/timeout measurements include suspension and backlog; zero-cost interpreter steps do not supply physical cost evidence.

9.4 Projection and adequacy

A conforming reference trace records enough identity and order to reconstruct observable writes, messages, state changes, waits and terminals under its chosen assumptions. A view is a pure identity selection function over D/O; Timing additionally projects a separately identified trace. No view transition changes C. A bounded run ending at a horizon retains live/open intervals. Source hashing belongs to hosts. Kernel/target refinement must preserve these observations or explicitly report unsupported semantics; syntax acceptance alone is insufficient.

NORMATIVE / LANGUAGE CONTRACT

SWAML 2.0 graphical notation

Original monochrome SWAML notation uses thin rules, explicit labels and owned compartments. It does not claim OMG notation conformance. The canonical catalog pairs every concept with context, source subject, drawing and constraints. Worked diagrams retain source-backed model subjects. spec/notation.json covers every grammar/type family and every keyword, including a text-only rationale for lexical spelling.

Components show definition versus occurrence identity and indexed multiplicity. Private attributes stay in their component; operations show parameters/returns and nested ownership without closure captures. Schedule compartments show triggers, deadlines and concurrency separately from signatures. Do not draw performers or action allocations.

Port boundary squares expose direction per member: provides/requires attribute, event or operation. The square is a neutral container that may mix all three member kinds. Attribute/event arrows point outward for provides and inward for requires; a compact double arrow summarizes both directions only when both exist. A ball marks each provided operation and an open socket each required operation. Expanded labels preserve member names, kinds and signatures; these glyphs never assign a single kind or role to the whole port. Reversal flips every member arrow and swaps each operation ball/socket. Connection lines join opposite-compatible members and identify their effective policy; forwarding connects boundary to immediate child with the same member directions and is visibly labeled as a queue-free alias. Do not restore interface kinds, versions or whole-port roles.

Flow shows expressions/calls, decisions, loop forms, lazy try fallback, receive/wait suspension and armed interrupt transfer. States are inside an operation; transitions are nested/ordered and labeled by input/guard or after timer and target. State bodies have no separate executable identity. Final completion is distinguishable from idle reactions.

Platform Nodes own const attributes and child Nodes. Deployment edges refer to the separate placement declaration and preserve indexed occurrence identity. Requirement/case links show intent rather than proof. Cases show named from/to/via/after steps on real routes.

Browser remains navigation. General, Interconnect, Case, Sequence, Grid, Flow and Timing are imported view definitions. General Box in box/Tree preserve the same selected identities/topology. Timing declared mode shows schedules/deadlines/static waits; observed mode adds an identified run, lock gaps, message latency, terminals and open intervals. Missing evidence never becomes measured WCET. Geometry, collapse and routes stay in sidecars.

Regenerate with python scripts/build-notation.py and python scripts/build-graphical-examples.py, then run both with --check. SVG, PNG and scene JSON share the same original geometry. New artifacts live under versioned 2.0 directories; historical publication assets remain untouched.

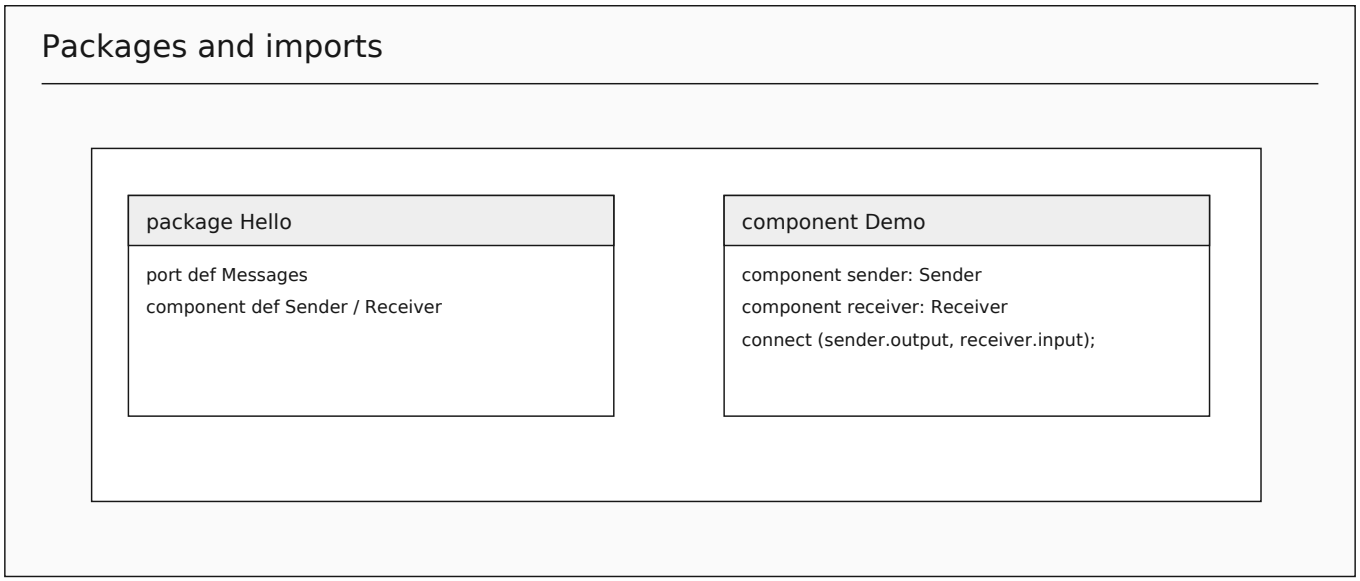
NORMATIVE / LANGUAGE CONTRACT

SWAML 2.0 canonical graphical catalog

Generated from spec/notation.json. Original monochrome notation; no OMG notation-conformance claim. Every drawing preserves source ownership and identity.

Packages and imports

SWAML 2.0; Hello::Demo.



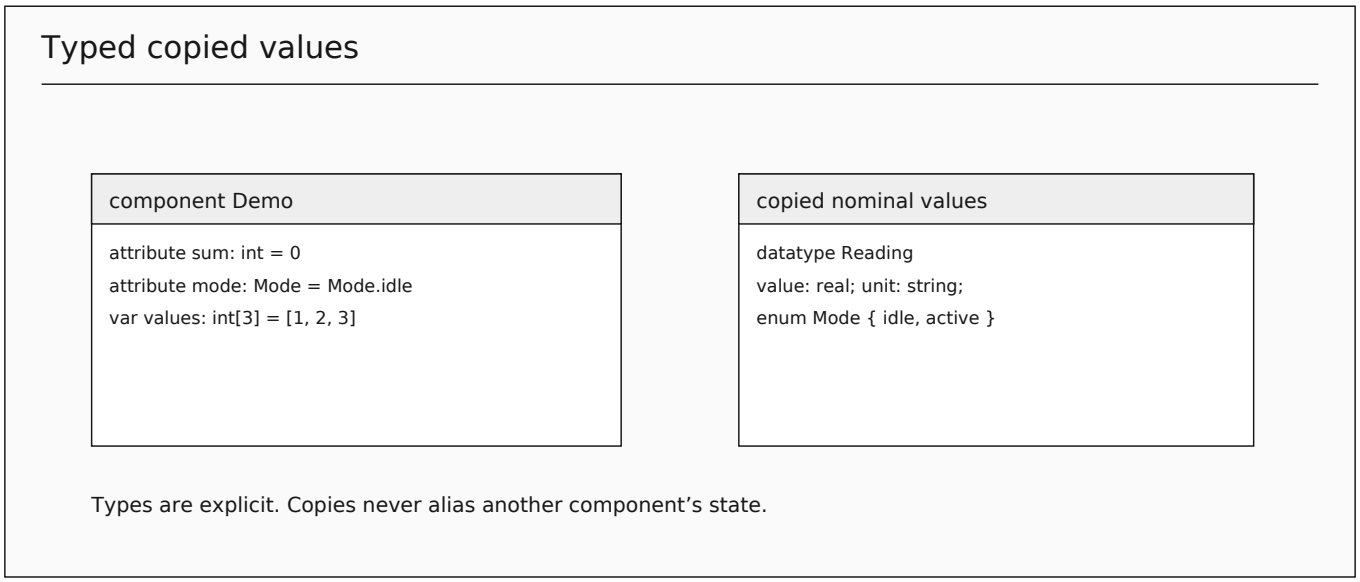
Packages and imports

Source: Hello::Demo. **Grammar:** Model, Package, Import, QualifiedName.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Typed copied values

SWAML 2.0; Values::Demo.



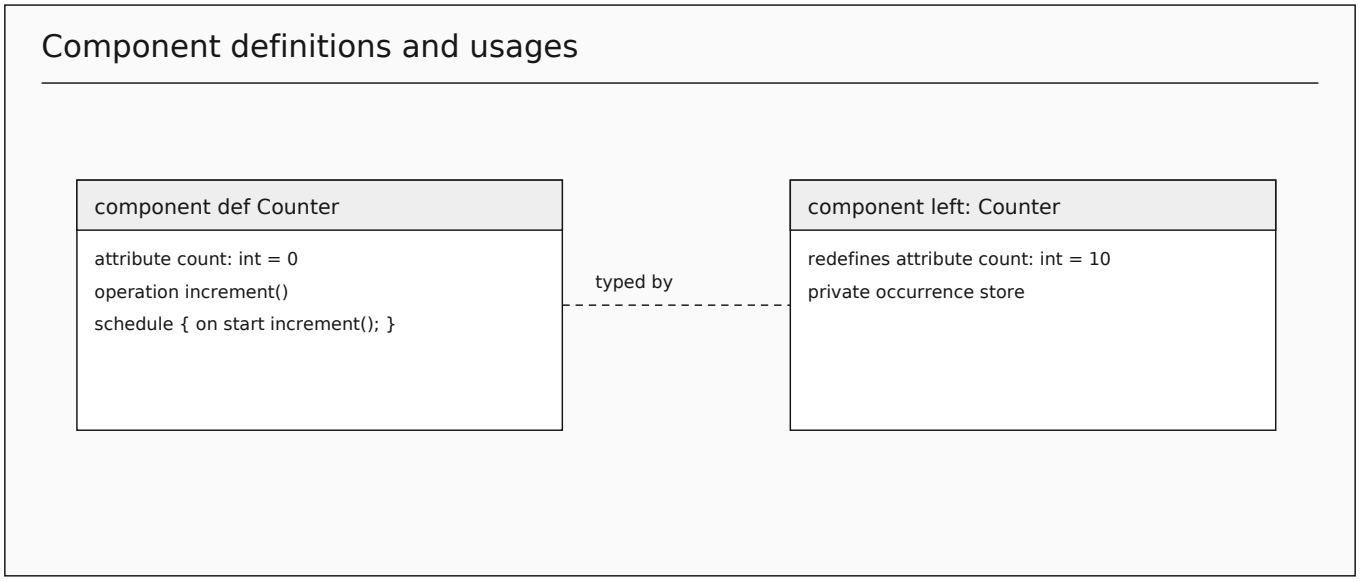
Typed copied values

Source: Values::Demo. **Grammar:** DataType, Field, EnumType, TypeRef, ValueType, RecordLiteral, FieldValue, ArrayLiteral.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Component definitions and usages

SWAML 2.0; Reuse::Demo.



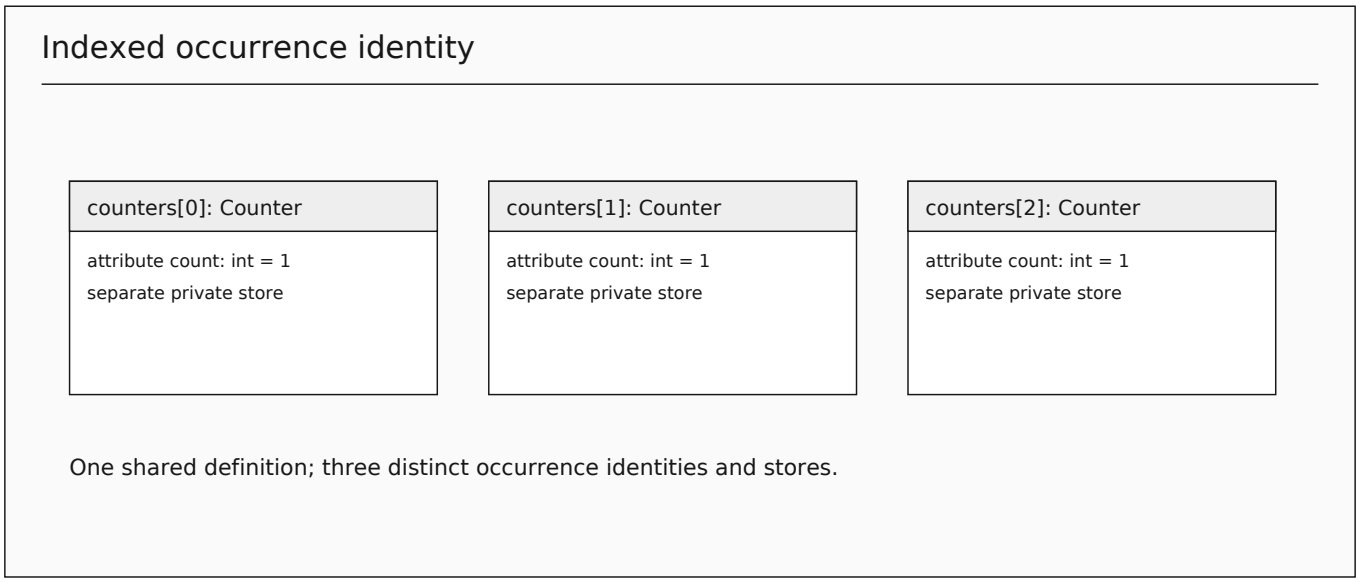
Component definitions and usages

Source: Reuse::Demo. **Grammar:** ComponentDef, ComponentUsage, ComponentMember, Element.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Indexed occurrence identity

SWAML 2.0; Multiplicity::Demo.



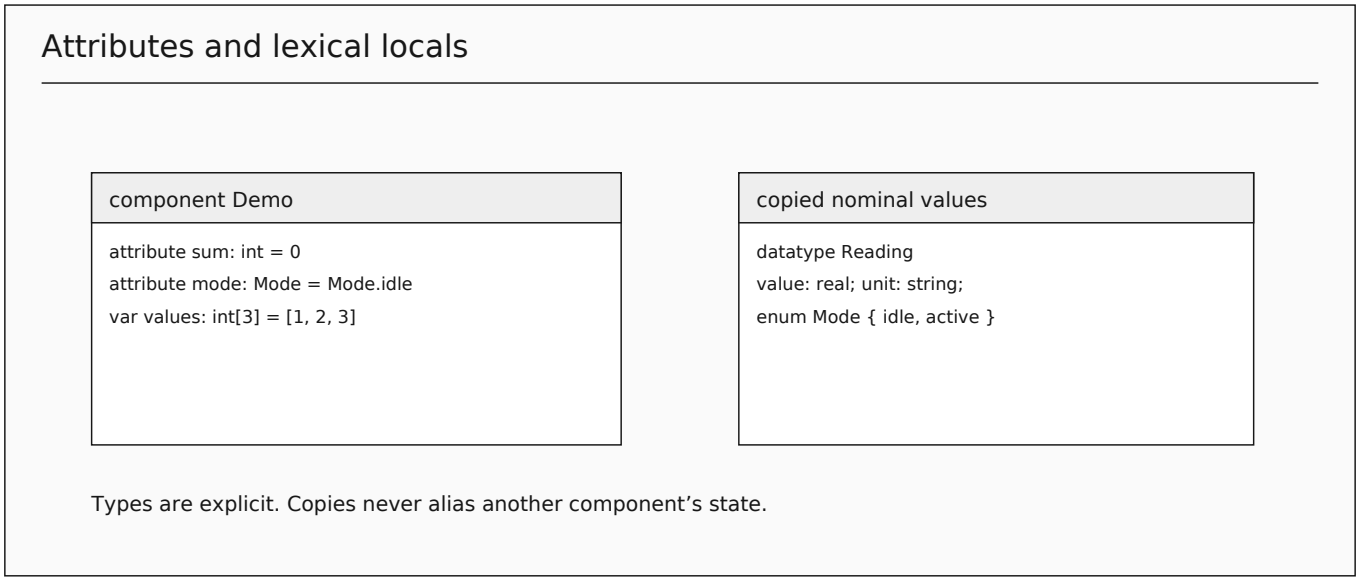
Indexed occurrence identity

Source: Multiplicity::Demo. **Grammar:** .

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Attributes and lexical locals

SWAML 2.0; Values::Demo.



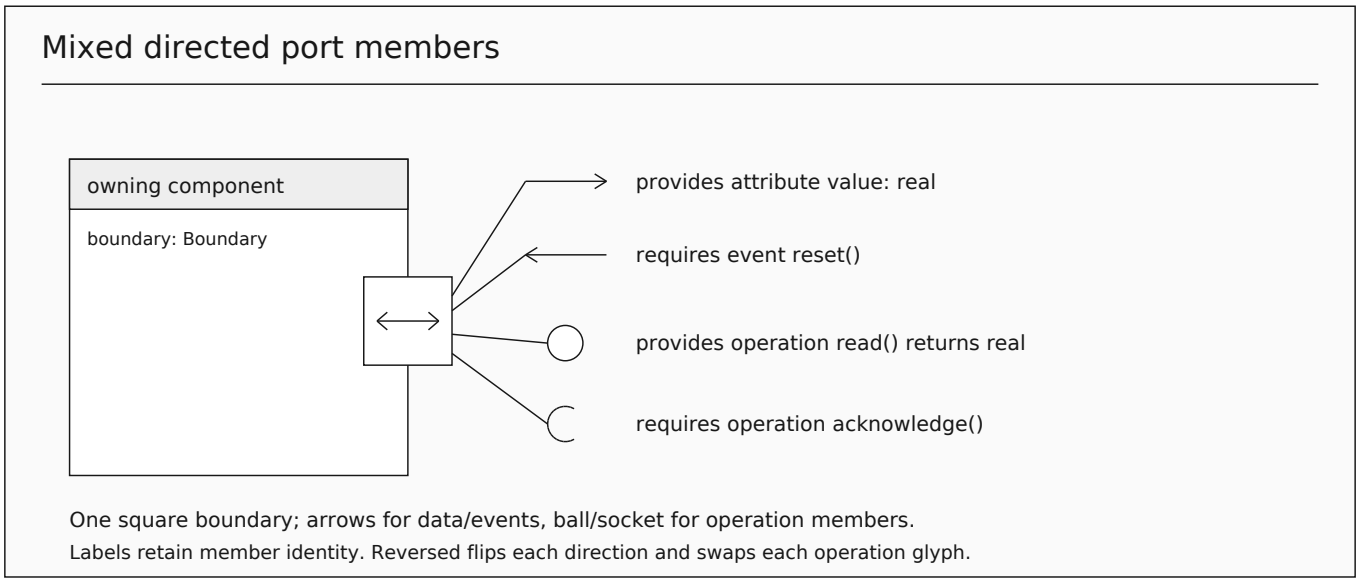
Attributes and lexical locals

Source: Values::Demo. **Grammar:** Attribute, LocalVariable, Argument.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Mixed directed port members

SWAML 2.0; Mixed::Boundary.



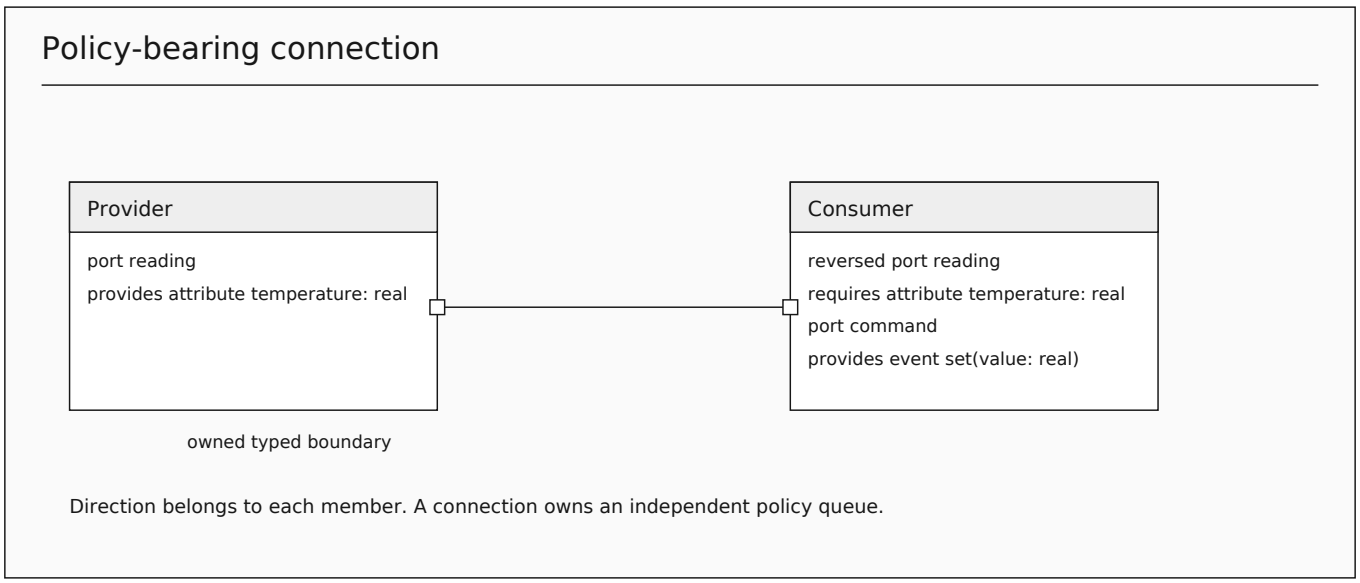
Mixed directed port members

Source: Mixed::Boundary. **Grammar:** PortDef, PortUsage, PortMember, MemberPath, Send.

One square boundary can contain attribute, event and operation members together. Arrowheads show data/event direction; an outward arrow means provides and inward means requires. A ball marks a provided operation; an open socket marks a required operation. These are member glyphs, not port types. A compact double arrow summarizes opposite data/event directions only; expanded labels retain each member name and type. Reversed flips every direction, arrow and operation glyph; geometry never adds members or routes. A canonical source-backed projection; geometry is separate from semantics.

Policy-bearing connection

SWAML 2.0; Policies::Demo.



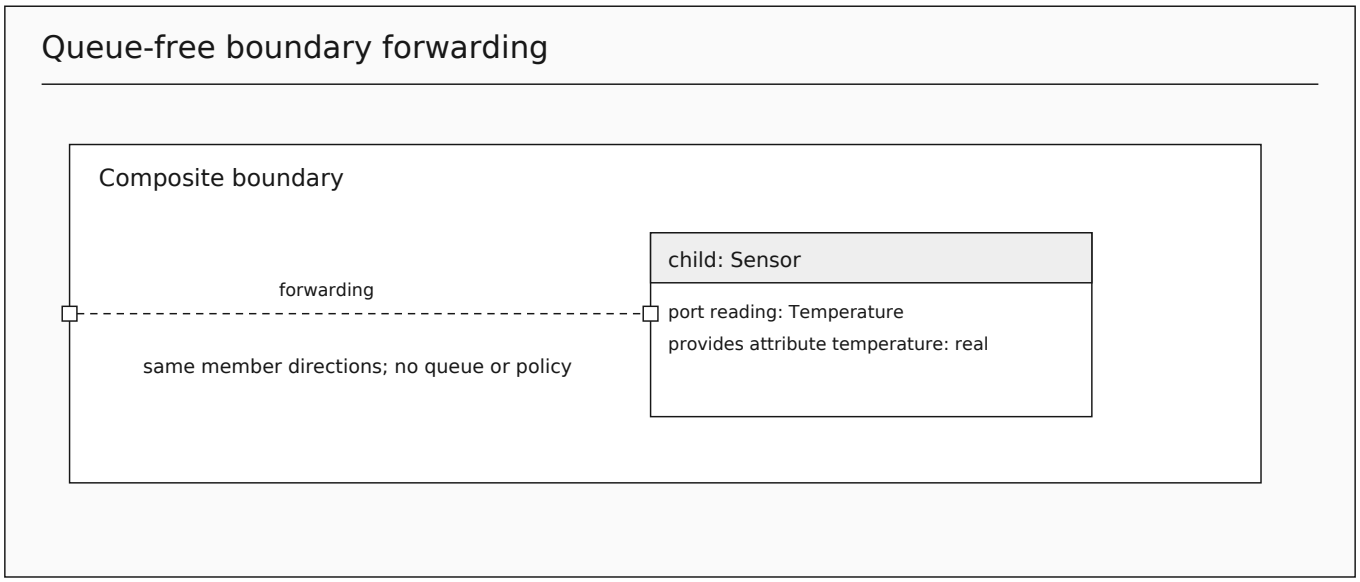
Policy-bearing connection

Source: Policies::Demo. **Grammar:** Connect, QosPolicy, PolicySetting.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Queue-free boundary forwarding

SWAML 2.0; Boundary::Demo.



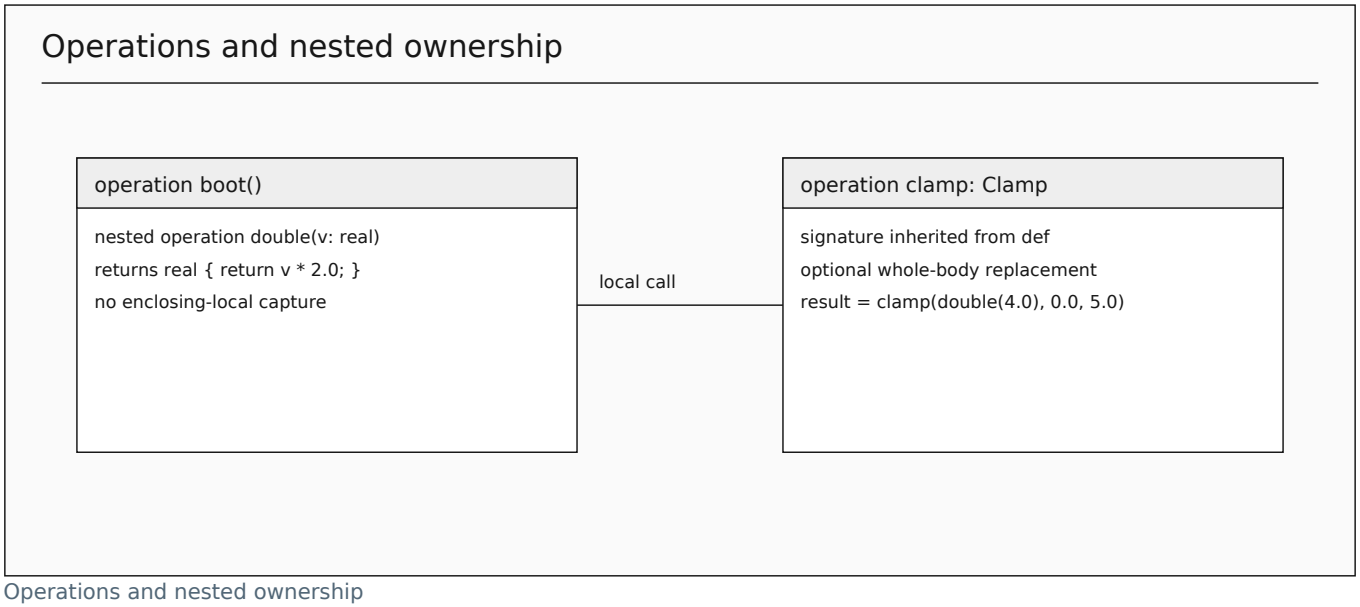
Queue-free boundary forwarding

Source: Boundary::Demo. **Grammar:** .

Preserve owned source and distinct definition/occurrence identities. Boundary alias adds no queue, policy or message. A canonical source-backed projection; geometry is separate from semantics.

Operations and nested ownership

SWAML 2.0; OperationUsage::Demo.



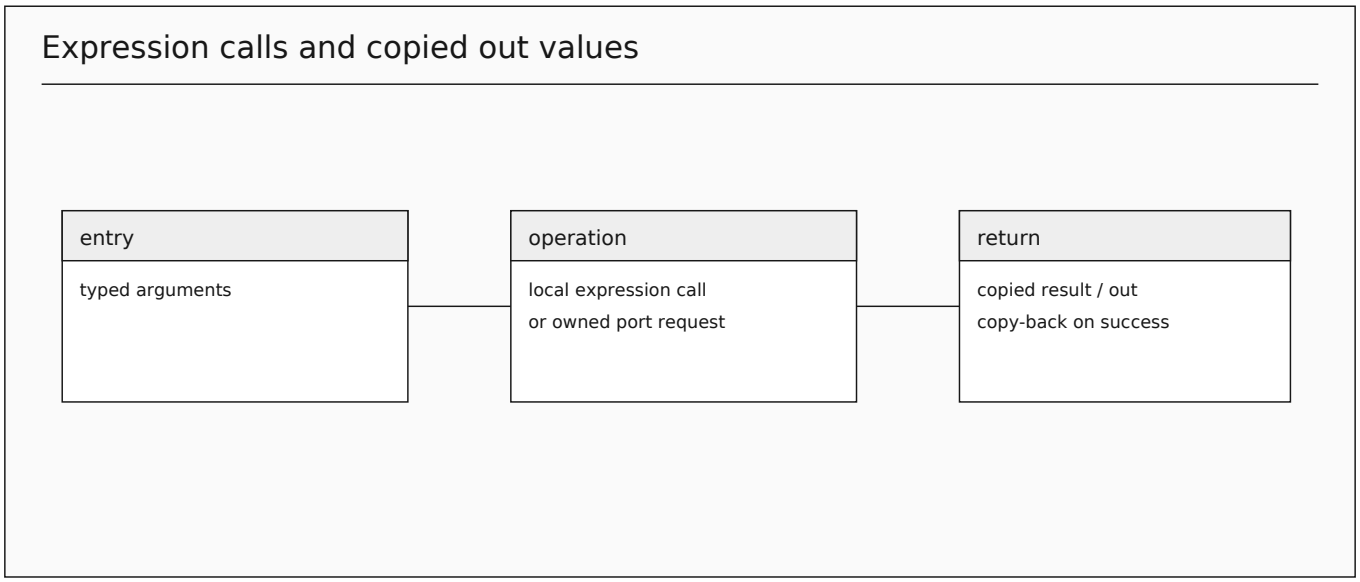
Operations and nested ownership

Source: OperationUsage::Demo. **Grammar:** Operation, OperationUsage, Executable, OperationBody.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Expression calls and copied out values

SWAML 2.0; ClientServer::Client.



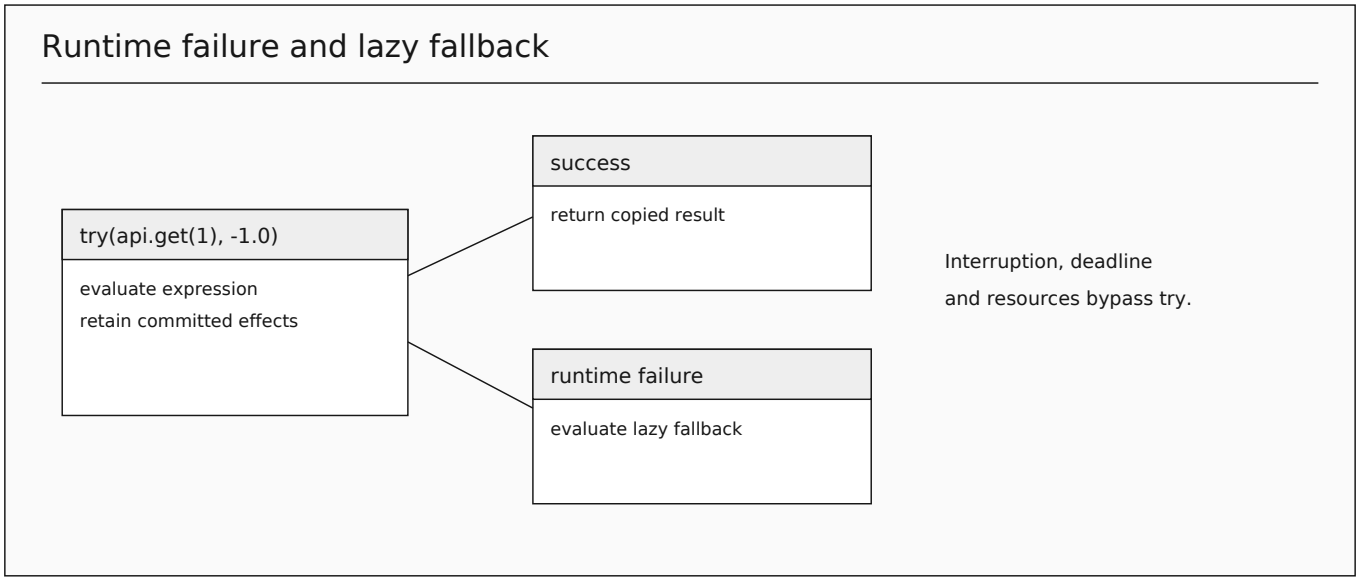
Expression calls and copied out values

Source: ClientServer::Client. **Grammar:** CallExpression, ExpressionStatement, Return.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Runtime failure and lazy fallback

SWAML 2.0; ClientServer::Client.



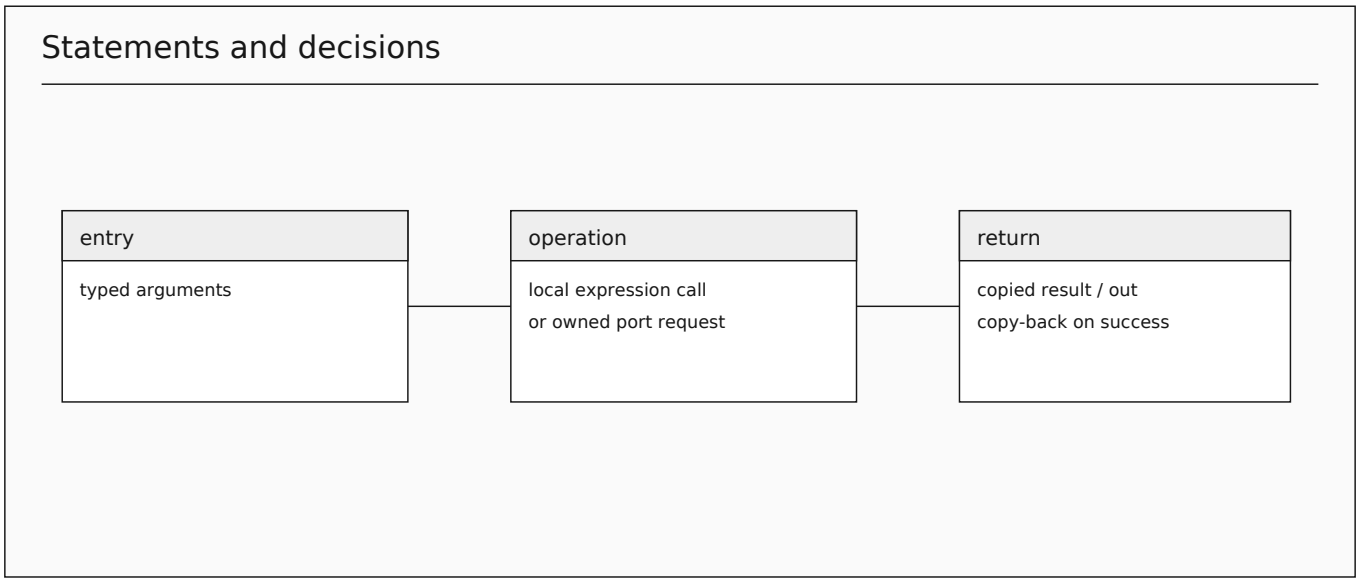
Runtime failure and lazy fallback

Source: ClientServer::Client. **Grammar:** TryExpression, Fail, Assert.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Statements and decisions

SWAML 2.0; Values::Demo.



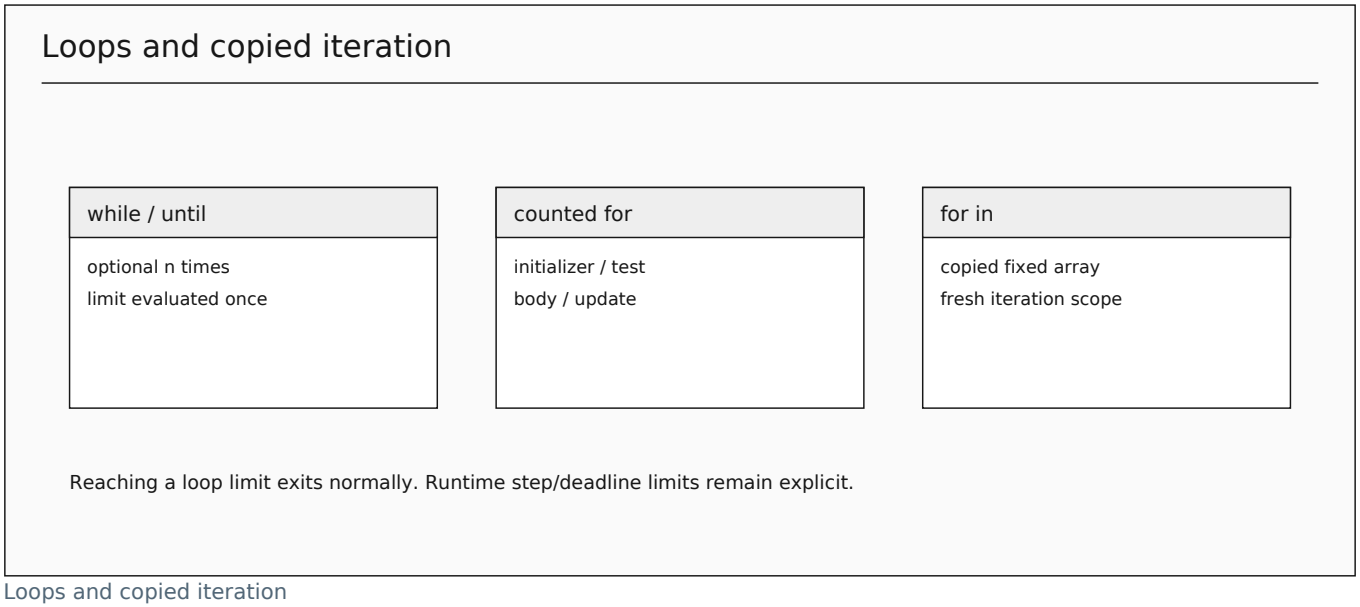
Statements and decisions

Source: Values::Demo. **Grammar:** Block, Statement, Assign, If.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Loops and copied iteration

SWAML 2.0; Loops::Demo.



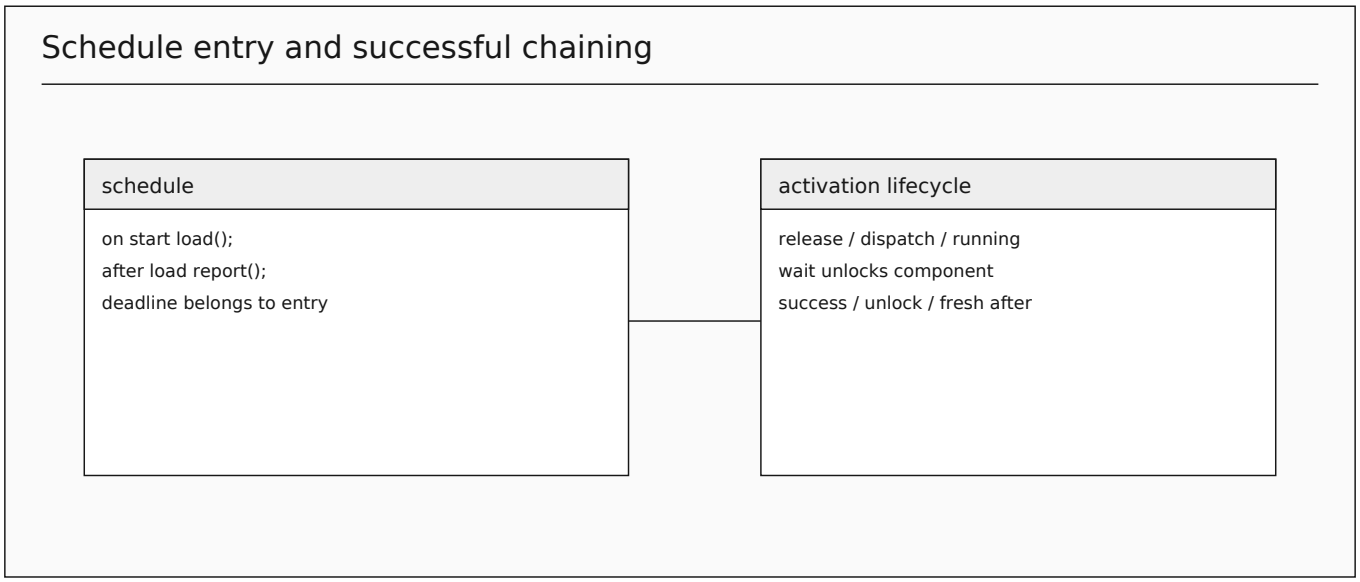
Loops and copied iteration

Source: Loops::Demo. **Grammar:** While, Until, For, ForUpdate, ForEach.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Schedule entry and successful chaining

SWAML 2.0; Chaining::Demo.



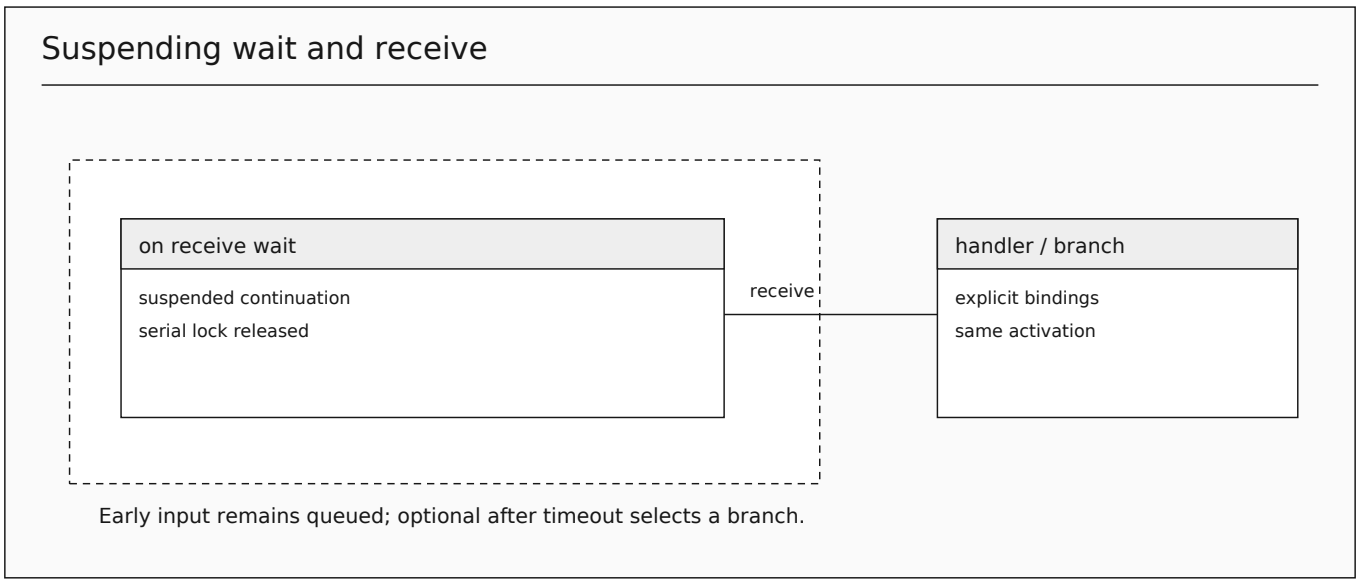
Schedule entry and successful chaining

Source: Chaining::Demo. **Grammar:** Schedule, ScheduleEntry.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Suspending wait and receive

SWAML 2.0; Receiving::Receiver.



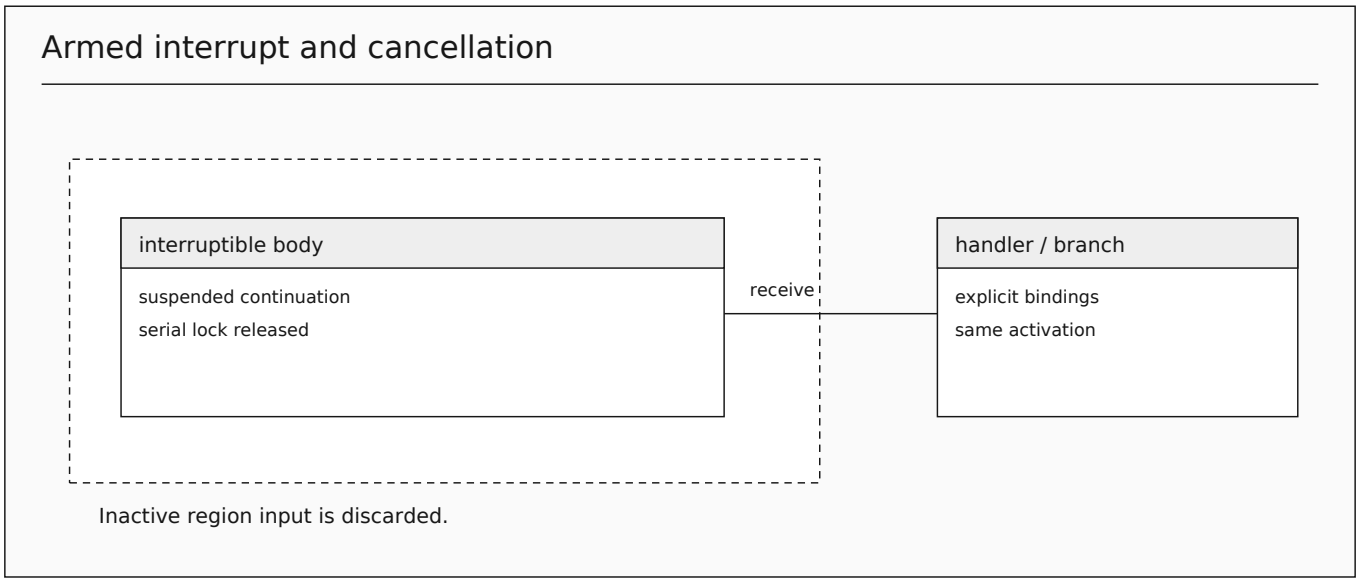
Suspending wait and receive

Source: Receiving::Receiver. **Grammar:** Wait, Receive, ReceiveBranch.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Armed interrupt and cancellation

SWAML 2.0; Interrupts::Receiver.



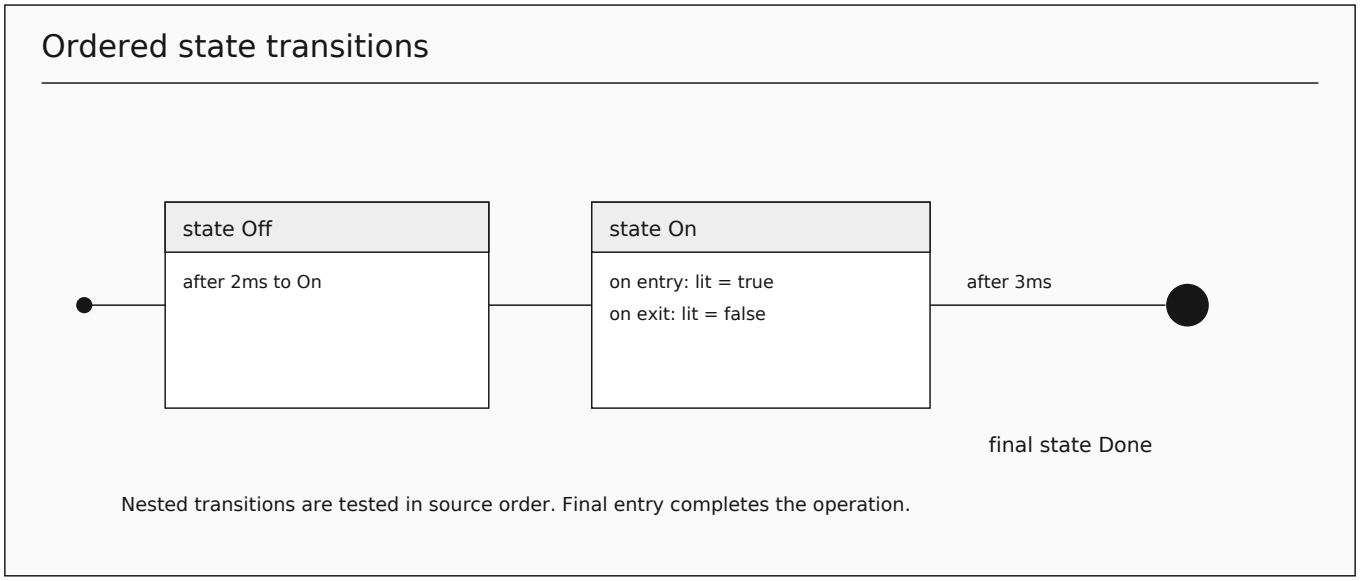
Armed interrupt and cancellation

Source: Interrupts::Receiver. **Grammar:** Interruptible.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Ordered state transitions

SWAML 2.0; States::Demo.lamp.



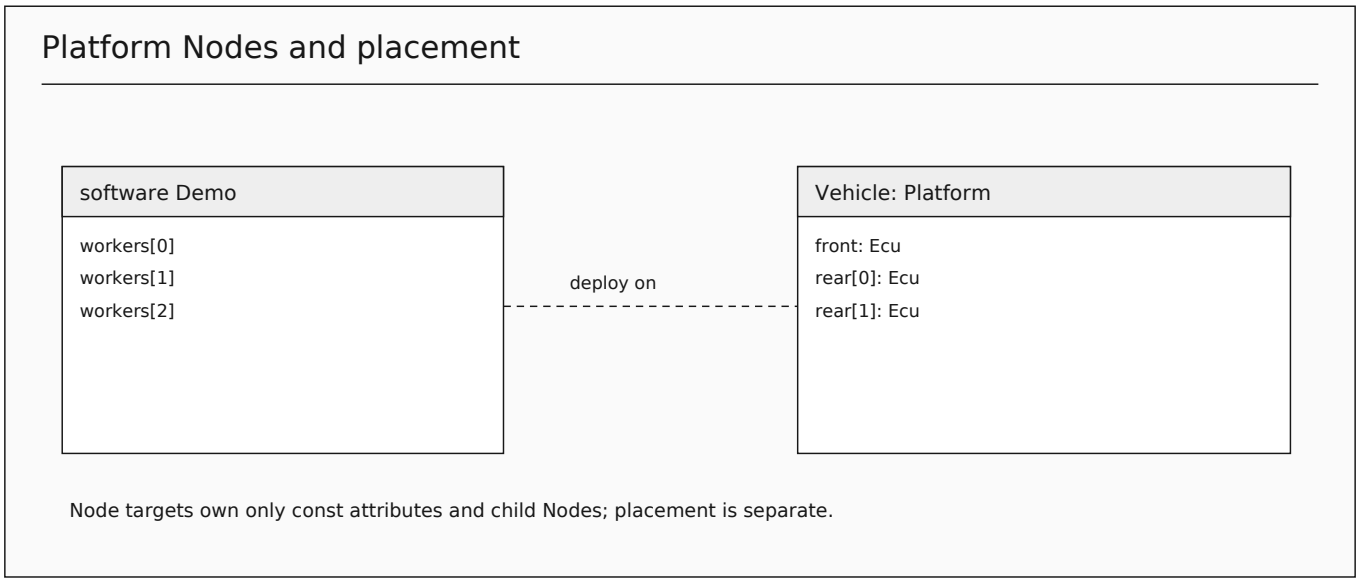
Ordered state transitions

Source: States::Demo.lamp. **Grammar:** State, StateAction, Transition.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Platform Nodes and placement

SWAML 2.0; Placement::Demo.



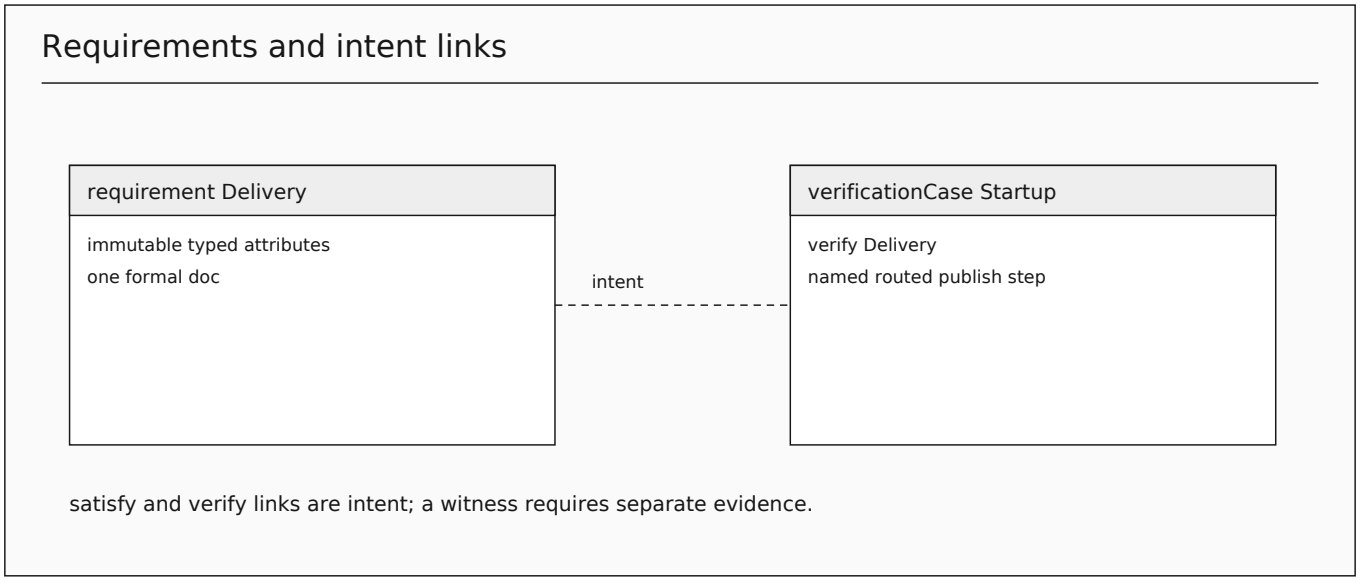
Platform Nodes and placement

Source: Placement::Demo. **Grammar:** Deployment, Placement.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Requirements and intent links

SWAML 2.0; Traceability::Delivery.



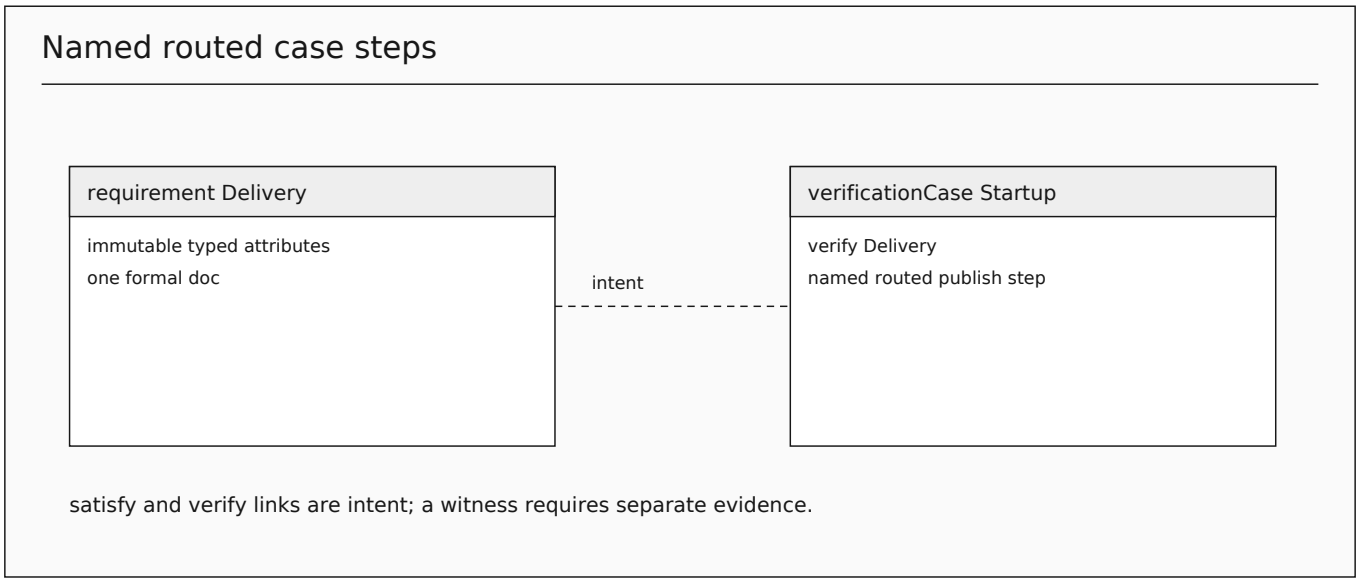
Requirements and intent links

Source: Traceability::Delivery. **Grammar:** Requirement, Satisfaction, Verification.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Named routed case steps

SWAML 2.0; Traceability::Startup.



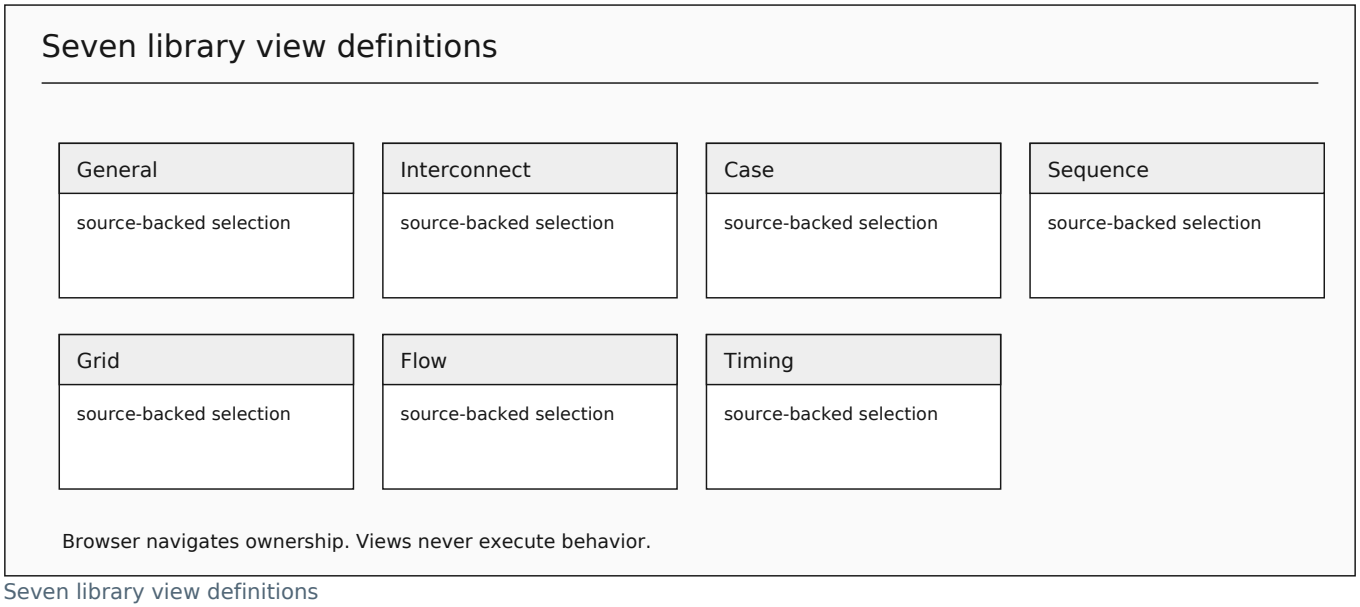
Named routed case steps

Source: Traceability::Startup. **Grammar:** Scenario, ScenarioStep.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Seven library view definitions

SWAML 2.0; Traceability::Demo.



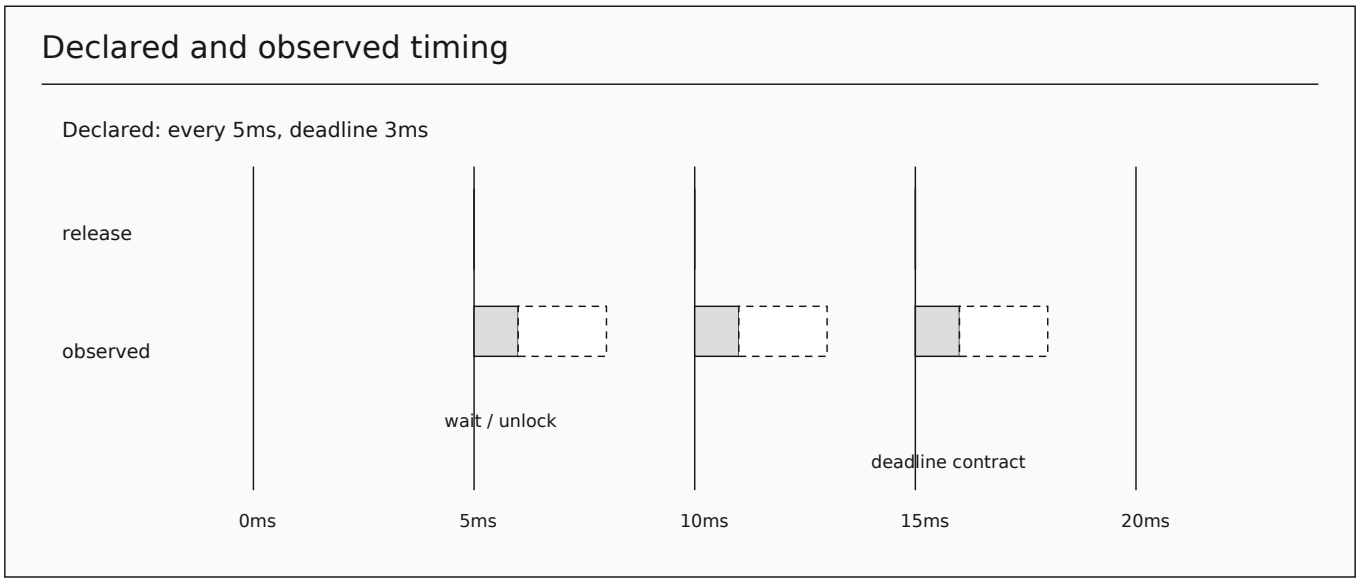
Seven library view definitions

Source: Traceability::Demo. **Grammar:** ViewDef, View, ViewSelection, FilterExpression, FilterOr, FilterAnd, FilterPrimary, FilterBinary, FilterNot, FilterComparison.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Declared and observed timing

SWAML 2.0; Timing::Demo.sample.



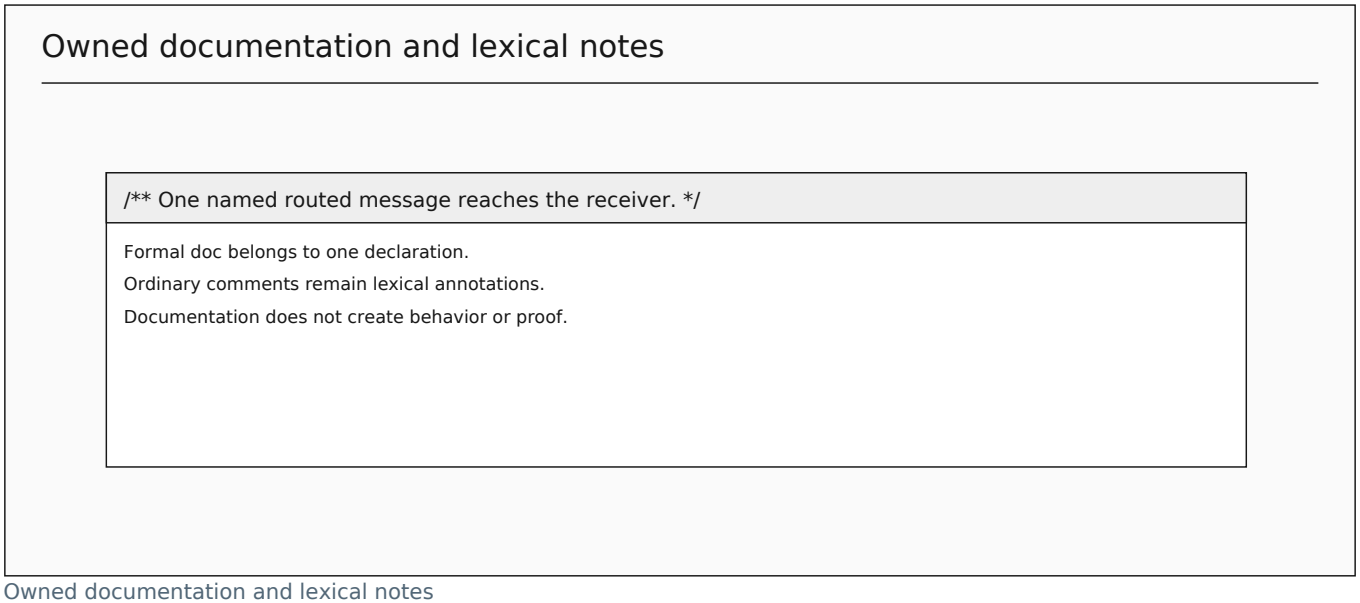
Declared and observed timing

Source: Timing::Demo.sample. **Grammar:** .

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Owned documentation and lexical notes

SWAML 2.0; Traceability::Delivery.



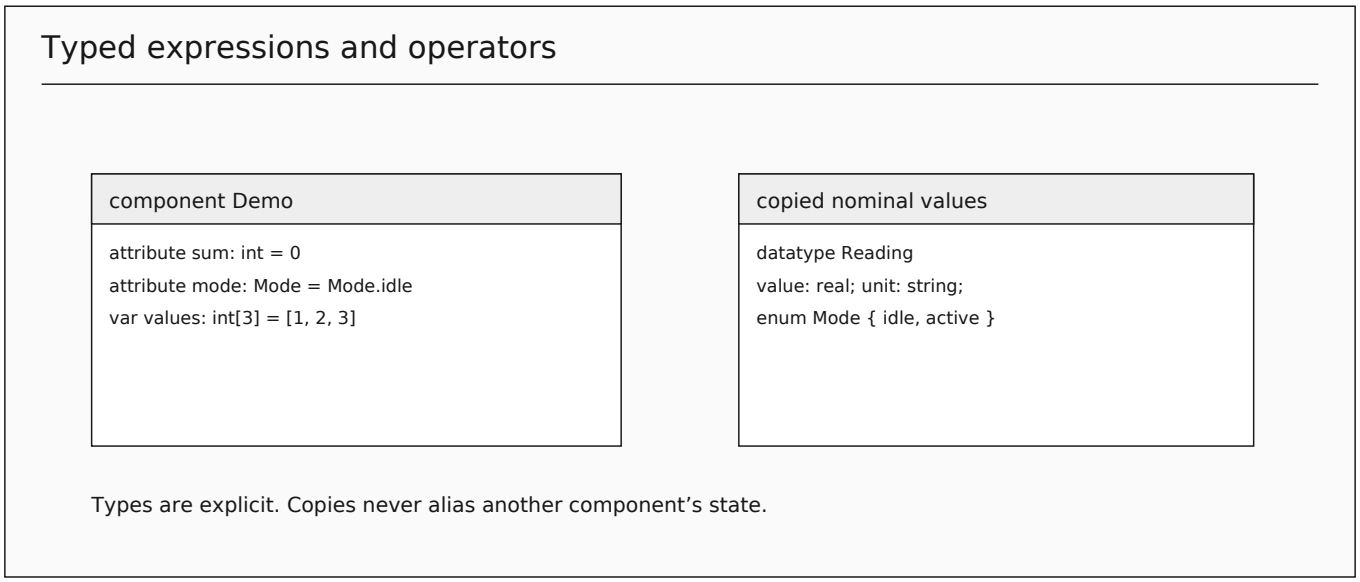
Owned documentation and lexical notes

Source: Traceability::Delivery. **Grammar:** Documentation, DOC_COMMENT.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Typed expressions and operators

SWAML 2.0; Values::Demo.



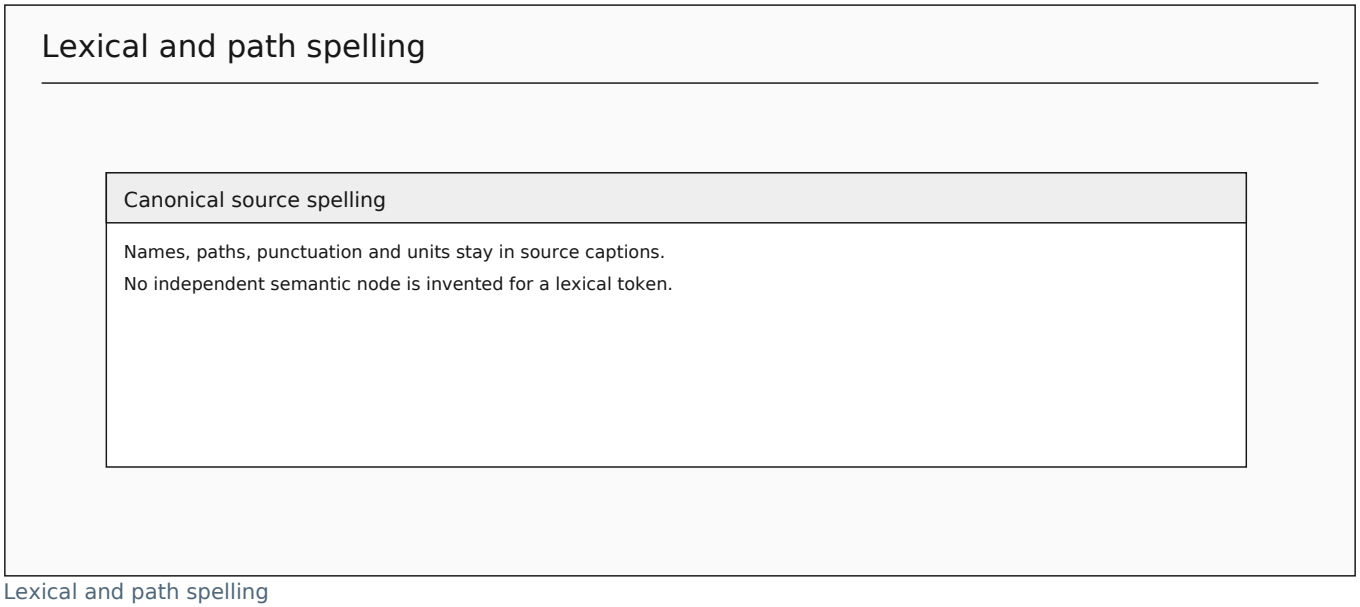
Typed expressions and operators

Source: Values::Demo. **Grammar:** Expression, OrExpression, AndExpression, EqualityExpression, RelationalExpression, AdditiveExpression, MultiplicativeExpression, UnaryExpression, PrimaryExpression, BinaryExpression, ReferenceExpression, ValuePath, Literal, Numeric.

Preserve owned source and distinct definition/occurrence identities. Drawing is a projection; it does not add execution semantics. A canonical source-backed projection; geometry is separate from semantics.

Lexical and path spelling

SWAML 2.0; Hello::Demo.



Lexical and path spelling

Source: Hello::Demo. **Grammar:** Duration, Name, Path, WS, ID, NUMBER, INT, STRING, ML_COMMENT, SL_COMMENT.

Text-only spelling; show as a source caption without invented semantic nodes. Drawing is a projection; it does not add execution semantics. Lexical spelling has no independent graphical identity.

NORMATIVE APPENDIX

A. Complete concrete grammar

Authoritative Langium grammar, reproduced in full. Line wraps are typographic continuations; only numbered rows begin original source lines.

LANGIUM GRAMMAR

```
grammar Swaml

entry Model: packages+=Package*;
Package: 'package' name=QualifiedName '{' documentation=Documentation? (imports+=Import |
elements+=Element | satisfactions+=Satisfaction)* '}';
Import: 'import' namespace=QualifiedName ';';
Element: DataType | EnumType | ComponentDef | ComponentUsage | PortDef | Operation | QosPolicy | ViewDef
 | Requirement | Scenario | View | Deployment;
DataType: 'datatype' name=Name '{' documentation=Documentation? fields+=Field* '}';
Field: name=Name ':' type=TypeRef documentation=Documentation? ';';
EnumType: 'enum' name=Name '{' documentation=Documentation? literals+=Name (',' literals+=Name)* '}';
TypeRef: (primitive=('bool'|'byte'|'uint'|'int'|'real'|'string'|'void') |
type=[ValueType:QualifiedName]) ('[' size=INT ']')?;
type ValueType = DataType | EnumType;
Argument: (direction=('in'|'out'))? name=Name ':' type=TypeRef;
ComponentDef: 'component' 'def' name=Name ('specializes' specializes=[ComponentDef:QualifiedName])? '{'
documentation=Documentation? members+=ComponentMember* '}';
ComponentUsage: (redefines?='redefines')? 'component' name=Name (':'
definition=[ComponentDef:QualifiedName] ('[' size=INT ']')?)? (';' | '{' documentation=Documentation?
members+=ComponentMember* '}');
ComponentMember: ComponentUsage | PortUsage | Attribute | Operation | OperationUsage | Connect |
Schedule;
Attribute: (redefines?='redefines')? (constant?='const')? 'attribute' name=Name ':' type=TypeRef '='
value=Expression documentation=Documentation? ';';
PortDef: 'port' 'def' name=Name '{' documentation=Documentation? members+=PortMember* '}';
PortUsage: (redefines?='redefines')? (optional?='optional')? (reversed?='reversed')? 'port' name=Name
(':' definition=[PortDef:QualifiedName] documentation=Documentation? ';' | '{'
documentation=Documentation? members+=PortMember* '}');
PortMember: direction=('provides'|'requires') (idempotent?='idempotent')?
kind=('attribute'|'event'|'operation') name=Name (':' type=TypeRef | '(' (parameters+=Argument (','
parameters+=Argument)*)? ')' ('returns' result=TypeRef)?) documentation=Documentation? ';';
Connect: (redefines?='redefines')? 'connect' (name=Name)? (':' policy=[QosPolicy:QualifiedName])? '('
source=Path ',' target=Path ')' documentation=Documentation? ';';
QosPolicy: 'qos' 'def' name=Name (':' base=QualifiedName)? '{' documentation=Documentation?
settings+=PolicySetting* '}';
PolicySetting: name=Name '=' (value=Expression | duration=Duration) ';';
Operation: (redefines?='redefines')? 'operation' (definition?='def')? name=Name '('
(parameters+=Argument (',' parameters+=Argument)*)? ')' ('returns' result=TypeRef)? body=OperationBody;
OperationUsage: (redefines?='redefines')? 'operation' name=Name ':' definition=[Operation:QualifiedName]
 (documentation=Documentation? ';' | body=OperationBody);
Executable: Operation | OperationUsage;
OperationBody: '{' documentation=Documentation? operations+=Executable* ('initial' initial=Name ';'
states+=State+ | statements+=Statement*) '}';
Schedule: 'schedule' '{' entries+=ScheduleEntry* '}';
ScheduleEntry: ('on' (kind='start' | kind='every' period=Duration | kind=('receive'|'call')
endpoint=MemberPath '(' (bindings+=Name (',' bindings+=Name)*)? ')') | kind='after' predecessor=Name)
operation=Name '(' (args+=Expression (',' args+=Expression)*)? ')' ('deadline' deadline=Duration)?
(concurrent?='concurrent')? ';';
Block: '{' statements+=Statement* '}';
Statement: LocalVariable | Assign | Send | ExpressionStatement | Return | If | While | Until | For |
ForEach | Wait | Receive | Interruptible | Fail | Assert;
LocalVariable: 'var' name=Name ':' type=TypeRef '=' value=Expression ';';
Assign: target=Name '=' value=Expression ';';
ExpressionStatement: expression=Expression ';';
Send: kind=('send'|'publish') endpoint=MemberPath '(' (args+=Expression (',' args+=Expression)*)? ')'
';';
Return: 'return' value=Expression? ';';
If: 'if' '(' condition=Expression ')' then=Block ('else' otherwise=Block)?;
```

LANGIUM GRAMMAR / CONTINUED

```
While: 'while' '(' condition=Expression (',' limit=Expression 'times')? ')' body=Block;
Until: 'until' '(' condition=Expression (',' limit=Expression 'times')? ')' body=Block;
For: 'for' '(' initializer=LocalVariable condition=Expression ';' update=ForUpdate ')' body=Block;
ForUpdate infers Assign: target=Name '=' value=Expression;
ForEach: 'for' '(' variable=Argument 'in' collection=Expression ')' body=Block;
Wait: 'wait' duration=Duration ';';
Receive: 'on' branches+=ReceiveBranch ('or' branches+=ReceiveBranch)* ('or' 'after' timeout=Duration
timeoutBody=Block)?;
ReceiveBranch: 'receive' endpoint=MemberPath '(' (bindings+=Name (',' bindings+=Name)*)? ')' body=Block;
Interruptible: 'interruptible' body=Block 'on' 'receive' endpoint=MemberPath '(' (bindings+=Name (','
bindings+=Name)*)? ')' handler=Block;
Fail: 'fail' reason=STRING ';';
Assert: 'assert' condition=Expression ';';
State: (final?='final')? 'state' name=Name '{' documentation=Documentation? (actions+=StateAction |
transitions+=Transition)* '}';
StateAction: 'on' kind=('entry'|'exit') body=Block;
Transition: ('on' 'receive' endpoint=MemberPath '(' (bindings+=Name (',' bindings+=Name)*)? ')' ('when'
'(' guard=Expression ')')? | 'after' delay=Duration) 'to' target=Name effect=Block;
Deployment: 'deployment' name=Name 'of' root=[ComponentUsage:QualifiedName] 'on'
platform=[ComponentUsage:QualifiedName] '{' documentation=Documentation? placements+=Placement* '}';
Placement: 'deploy' target=Path? 'on' node=Path ';';
Requirement: 'requirement' name=Name '{' documentation=Documentation? attributes+=Attribute* '}';
Documentation: text=DOC_COMMENT;
Satisfaction: 'satisfy' requirement=[Requirement:QualifiedName] 'of' source=Path ';';
Verification: 'verify' requirement=[Requirement:QualifiedName] ';';
Scenario: kind=('useCase'|'verificationCase') name=Name 'of' root=[ComponentUsage:QualifiedName] '{'
documentation=Documentation? verifications+=Verification* steps+=ScenarioStep* '}';
ScenarioStep: kind=('send'|'publish'|'request'|'reply') name=Name 'from' source=Path 'to' target=Path
('via' connector=Name)? ('after' predecessor=Name)? ';';
ViewDef: 'view' 'def' name=Name documentation=Documentation? ';';
View: 'view' name=Name ':' definition=[ViewDef:QualifiedName] ('of' subject=Path
documentation=Documentation? ';' | '{' documentation=Documentation? selections+=ViewSelection+ '}');
ViewSelection: 'include' subject=Path (recursive?='recursive')? ('where' filter=FilterExpression)? ';';
type FilterExpression = FilterBinary | FilterNot | FilterComparison;
FilterExpression returns FilterExpression: FilterOr;
FilterOr returns FilterExpression: FilterAnd ({infer FilterBinary.left=current} operator='or'
right=FilterAnd)*;
FilterAnd returns FilterExpression: FilterPrimary ({infer FilterBinary.left=current} operator='and'
right=FilterPrimary)*;
FilterPrimary returns FilterExpression: {infer FilterNot} 'not' operand=FilterPrimary | FilterComparison
 | '(' FilterExpression ')';
FilterComparison: field=('name'|'kind'|'path') operator=('=='|'!=') value=STRING;
Duration: value=INT unit=('ns'|'us'|'ms'|'s');
Name returns string: ID | 'reversed' | 'from' | 'to' | 'times' | 'request';
MemberPath returns string: Name '.' Name;
Path returns string: QualifiedName ('[' INT ']')? ('.' Name ('[' INT ']')?)*;
QualifiedName returns string: Name ('::' Name)*;
ValuePath returns string: QualifiedName ('.' Name)*;
type Expression = BinaryExpression | UnaryExpression | Literal | CallExpression | TryExpression |
ReferenceExpression | RecordLiteral | ArrayLiteral;
Expression returns Expression: OrExpression;
OrExpression returns Expression: AndExpression ({infer BinaryExpression.left=current} operator='or'
right=AndExpression)*;
AndExpression returns Expression: EqualityExpression ({infer BinaryExpression.left=current}
operator='and' right=EqualityExpression)*;
EqualityExpression returns Expression: RelationalExpression ({infer BinaryExpression.left=current}
operator=('=='|'!=') right=RelationalExpression)*;
RelationalExpression returns Expression: AdditiveExpression ({infer BinaryExpression.left=current}
operator=('<'|'<='|'>'|'>=') right=AdditiveExpression)*;
AdditiveExpression returns Expression: MultiplicativeExpression ({infer BinaryExpression.left=current}
operator=('+'|'-') right=MultiplicativeExpression)*;
MultiplicativeExpression returns Expression: UnaryExpression ({infer BinaryExpression.left=current}
operator=('*'|'/'|'%') right=UnaryExpression)*;
UnaryExpression returns Expression: {infer UnaryExpression} operator=('not'|'-') operand=UnaryExpression
 | PrimaryExpression;
PrimaryExpression returns Expression: Literal | TryExpression | CallExpression | ReferenceExpression |
RecordLiteral | ArrayLiteral | '(' Expression ')';
```

LANGIUM GRAMMAR / CONTINUED

```
84 TryExpression: 'try' '(' expression=Expression (',' fallback=Expression)? ')';
85 CallExpression: target=ValuePath '(' (args+=Expression (',' args+=Expression)*)? ')';
86 ReferenceExpression: path=ValuePath ('[' index=Expression ']')?;
87 Literal: {infer Literal} (number=Numeric | text=STRING | boolean=('true'|'false'));
88 RecordLiteral: 'record' type=[DataType:QualifiedName] '{' fields+=FieldValue* '}';
89 FieldValue: name=Name '=' value=Expression ';';
90 ArrayLiteral: '[' (values+=Expression (',' values+=Expression)*)? ']';
91 Numeric returns number: NUMBER | INT;
92 hidden terminal WS: /\s+/;
93 terminal DOC_COMMENT: /\/\*\*(?:\/|[\s\S]*?\*\/)/;
94 terminal ID: /[_a-zA-Z][\w]*/;
95 terminal NUMBER returns number: /[0-9]+\.[0-9]+/;
96 terminal INT returns number: /[0-9]+/;
97 terminal STRING: /"([^"\\\x00-\x1F]|\\["\\nrt])*"/;
98 hidden terminal ML_COMMENT: /\/\*(?!\*)[\s\S]*?\*\//;
99 hidden terminal SL_COMMENT: /\/\/[^\n\r]*/;
```

NORMATIVE DESIGN INVENTORY

B. Grammar-to-semantics inventory

Every grammar rule/type name is represented. Proposed kernel bases identify the design target; they do not constitute an implemented or validated exporter. Empty bases indicate syntax normalized through its owner.

Rule / kind	Proposed kernel base	Meaning to preserve
Model element	Package	Model: preserve packages and imports, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Package element	Package	Package: preserve packages and imports, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Import element	Package	Import: preserve packages and imports, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Element grouping	Owner representation	Element: preserve component definitions and usages, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
DataType element	DataType	DataType: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Field element	DataType	Field: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
EnumType element	DataType	EnumType: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
TypeRef reference	DataType	TypeRef: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ValueType element	DataType	ValueType: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Argument element	Feature	Argument: preserve attributes and lexical locals, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ComponentDef element	Structure	ComponentDef: preserve component definitions and usages, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ComponentUsage element	Structure	ComponentUsage: preserve component definitions and usages, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ComponentMember grouping	Owner representation	ComponentMember: preserve component definitions and usages, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
Attribute element	Feature	Attribute: preserve attributes and lexical locals, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
PortDef element	Feature	PortDef: preserve directed port members, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
PortUsage element	Feature	PortUsage: preserve directed port members, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
PortMember grouping	Owner representation	PortMember: preserve directed port members, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Connect element	Connector	Connect: preserve policy-bearing connection, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
QosPolicy element	Connector	QosPolicy: preserve policy-bearing connection, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
PolicySetting element	Connector	PolicySetting: preserve policy-bearing connection, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Operation element	Behavior	Operation: preserve operations and nested ownership, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
OperationUsage element	Behavior	OperationUsage: preserve operations and nested ownership, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Executable element	Behavior	Executable: preserve operations and nested ownership, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
OperationBody grouping	Owner representation	OperationBody: preserve operations and nested ownership, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Schedule lowering	Feature	Schedule: preserve schedule entry and successful chaining, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ScheduleEntry lowering	Feature	ScheduleEntry: preserve schedule entry and successful chaining, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Block grouping	Owner representation	Block: preserve statements and decisions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Statement grouping	Owner representation	Statement: preserve statements and decisions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
LocalVariable element	Feature	LocalVariable: preserve attributes and lexical locals, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Assign element	Feature	Assign: preserve statements and decisions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ExpressionStatement value	Expression	ExpressionStatement: preserve expression calls and copied out values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Send element	Feature	Send: preserve directed port members, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Return lowering	Feature	Return: preserve expression calls and copied out values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
If element	Feature	If: preserve statements and decisions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
While lowering	Feature	While: preserve loops and copied iteration, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Until lowering	Feature	Until: preserve loops and copied iteration, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
For lowering	Feature	For: preserve loops and copied iteration, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ForUpdate lowering	Feature	ForUpdate: preserve loops and copied iteration, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ForEach lowering	Feature	ForEach: preserve loops and copied iteration, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Wait lowering	Feature	Wait: preserve suspending wait and receive, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Receive lowering	Feature	Receive: preserve suspending wait and receive, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ReceiveBranch lowering	Feature	ReceiveBranch: preserve suspending wait and receive, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Interruptible lowering	Feature	Interruptible: preserve armed interrupt and cancellation, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
Fail lowering	Feature	Fail: preserve runtime failure and lazy fallback, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Assert lowering	Feature	Assert: preserve runtime failure and lazy fallback, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
State lowering	Feature	State: preserve ordered state transitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
StateAction lowering	Feature	StateAction: preserve ordered state transitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Transition lowering	Feature	Transition: preserve ordered state transitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Deployment element	Feature	Deployment: preserve platform nodes and placement, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Placement element	Feature	Placement: preserve platform nodes and placement, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Requirement element	Feature	Requirement: preserve requirements and intent links, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Documentation element	Feature	Documentation: preserve owned documentation and lexical notes, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Satisfaction element	Feature	Satisfaction: preserve requirements and intent links, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Verification element	Feature	Verification: preserve requirements and intent links, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Scenario element	Feature	Scenario: preserve named routed case steps, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ScenarioStep element	Feature	ScenarioStep: preserve named routed case steps, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ViewDef element	Feature	ViewDef: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
View element	Feature	View: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
ViewSelection element	Feature	ViewSelection: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FilterExpression value	Expression	FilterExpression: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FilterOr element	Feature	FilterOr: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FilterAnd element	Feature	FilterAnd: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FilterPrimary element	Feature	FilterPrimary: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FilterComparison element	Feature	FilterComparison: preserve seven library view definitions, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Duration lexical	Owner representation	Duration: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Name lexical	Owner representation	Name: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
MemberPath reference	Feature	MemberPath: preserve directed port members, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Path lexical	Owner representation	Path: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
QualifiedName reference	Package	QualifiedName: preserve packages and imports, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ValuePath reference	Feature	ValuePath: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Expression value	Expression	Expression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
OrExpression value	Expression	OrExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
AndExpression value	Expression	AndExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
EqualityExpression value	Expression	EqualityExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
RelationalExpression value	Expression	RelationalExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
AdditiveExpression value	Expression	AdditiveExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
MultiplicativeExpression value	Expression	MultiplicativeExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
UnaryExpression value	Expression	UnaryExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
PrimaryExpression value	Expression	PrimaryExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
TryExpression value	Expression	TryExpression: preserve runtime failure and lazy fallback, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
CallExpression value	Expression	CallExpression: preserve expression calls and copied out values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ReferenceExpression value	Expression	ReferenceExpression: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Literal value	Expression	Literal: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
RecordLiteral value	DataType	RecordLiteral: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
FieldValue element	DataType	FieldValue: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ArrayLiteral value	DataType	ArrayLiteral: preserve typed copied values, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
Numeric value	Expression	Numeric: preserve typed expressions and operators, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
WS lexical	Owner representation	WS: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

Rule / kind	Proposed kernel base	Meaning to preserve
DOC_COMMENT lexical	Owner representation	DOC_COMMENT: preserve owned documentation and lexical notes, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ID lexical	Owner representation	ID: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
NUMBER lexical	Owner representation	NUMBER: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
INT lexical	Owner representation	INT: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
STRING lexical	Owner representation	STRING: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
ML_COMMENT lexical	Owner representation	ML_COMMENT: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.
SL_COMMENT lexical	Owner representation	SL_COMMENT: preserve lexical and path spelling, ownership, types, order, occurrence identity and source provenance as defined in its normative chapter.

INFORMATIVE / COMPLETE MODELS

C. Model and library sources

01 hello

SWAML / 01-hello.swaml

```
package Hello {
  port def Messages {
    provides event message(value: string);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      publish output.message("Hello, SWAML");
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: string = "";
    operation accept(value: string) {
      last = value;
    }
    schedule {
      on receive input.message(value) accept(value);
    }
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect (sender.output, receiver.input);
  }
}
```

02 realtime controller

SWAML / 02-realtime-controller.swaml

```
package Control {
  import Swaml::Views;
  port def Temperature {
    provides attribute temperature: real;
  }
  port def Commands {
    provides event set(value: real);
  }
  component def Sensor {
    port reading: Temperature;
    operation sample() {
      wait 1ms;
      send reading.temperature(18.5);
    }
    schedule {
      on every 10ms sample();
    }
  }
  component def Controller {
    reversed port reading: Temperature;
    port command: Commands;
    attribute target: real = 21.0;
    operation regulate(measured: real) {
      var power: real = (target - measured) * 0.1;
      publish command.set(power);
    }
    schedule {
      on receive reading.temperature(t) regulate(t) deadline 5ms;
    }
  }
  component def Heater {
    reversed port command: Commands;
    attribute power: real = 0.0;
    operation apply(value: real) {
      power = value;
    }
    schedule {
      on receive command.set(value) apply(value);
    }
  }
  component Demo {
    component sensor: Sensor;
    component controller: Controller;
    component heater: Heater;
    connect samples (sensor.reading, controller.reading);
    connect commands (controller.command, heater.command);
  }
  requirement Responsive {
    attribute limitMs: uint = 5;
  }
  satisfy Responsive of Demo.controller;
  verificationCase ControlLoop of Demo {
    verify Responsive;
    send measured from sensor.reading.temperature to controller.reading.temperature via samples;
    publish applied from controller.command.set to heater.command.set via commands after measured;
  }
  view overview: General of Demo;
  view intent: Case of ControlLoop;
  view sequence: Sequence of ControlLoop;
  view inventory: Grid of Demo;
  view architecture: Interconnect of Demo;
  view logic: Flow of Demo.controller.regulate;
  view timing: Timing of Demo.sensor.sample;
}
```

03 service pubsub

SWAML / 03-service-pubsub.swaml

```
package Commerce {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

04 stateful function

SWAML / 04-stateful-function.swaml

```
package States {
  import Swaml::Views;
  component Demo {
    attribute lit: bool = false;
    operation lamp() {
      initial Off;
      state Off {
        after 2ms to On {
        }
      }
      state On {
        on entry {
          lit = true;
        } on exit {
          lit = false;
        }
        after 3ms to Done {
        }
      }
      final state Done {
      }
    }
    schedule {
      on start lamp();
    }
  }
  view states: Flow of Demo.lamp;
}
```

05 values and flow

SWAML / 05-values-and-flow.swaml

```
package Values {
  import Swaml::Math;
  datatype Reading {
    value: real;
    unit: string;
  }
  enum Mode {
    idle, active
  }
  component Demo {
    attribute sum: int = 0;
    attribute mode: Mode = Mode.idle;
    attribute reading: Reading = record Reading {
      value = 4.5;
      unit = "C";
    };
    operation add(a: int, b: int) returns int {
      return a + b;
    }
    operation boot() {
      var values: int[3] =[1, 2, 3];
      for (value: int in values) {
        sum = add(sum, value);
      }
      mode = Mode.active;
    }
    schedule {
      on start boot();
    }
  }
}
```

06 mixed gateway

SWAML / 06-mixed-gateway.swaml

```
package Gateway {
  port def Messages {
    provides event message(value: int);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      publish output.message(9);
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: int = 0;
    operation accept(value: int) {
      last = value;
    }
    schedule {
      on receive input.message(value) accept(value);
    }
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect (sender.output, receiver.input);
  }
}
```

07 client server

SWAML / 07-client-server.swaml

```
package ClientServer {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

08 service field

SWAML / 08-service-field.swaml

```
package Settings {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

09 component roots

SWAML / 09-component-roots.swaml

```
package Roots {
  component def Counter {
    attribute count: int = 0;
    operation increment() {
      count = count + 1;
    }
    operation boot() {
      increment();
    }
    schedule {
      on start boot();
    }
  }
  component def Fast specializes Counter {
    redefines operation increment() {
      count = count + 2;
    }
  }
  component Demo {
    component left: Counter {
      redefines attribute count: int = 10;
    }
    component right: Fast;
  }
}
```

10 reusable qos

SWAML / 10-reusable-qos.swaml

```
package Policies {
  import Swaml::Qos;
  qos def Small: Policy {
    delivery = Delivery.atMostOnce;
    ordering = Ordering.fifo;
    capacity = 2;
    overflow = Overflow.reject;
    requestTimeout = 50ms;
  }
  port def Messages {
    provides event message(value: int);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      publish output.message(7);
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: int = 0;
    operation accept(value: int) {
      last = value;
    }
    schedule {
      on receive input.message(value) accept(value);
    }
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect delivery: Small (sender.output, receiver.input);
  }
}
```

11 autosar classic

SWAML / 11-autosar-classic.swaml

```
package Classic {
  import Swaml::Views;
  port def Temperature {
    provides attribute temperature: real;
  }
  port def Commands {
    provides event set(value: real);
  }
  component def Sensor {
    port reading: Temperature;
    operation sample() {
      send reading.temperature(18.5);
    }
    schedule {
      on every 10ms sample();
    }
  }
  component def Controller {
    reversed port reading: Temperature;
    port command: Commands;
    attribute target: real = 21.0;
    operation regulate(measured: real) {
      var power: real = (target - measured) * 0.1;
      publish command.set(power);
    }
    schedule {
      on receive reading.temperature(t) regulate(t) deadline 5ms;
    }
  }
  component def Heater {
    reversed port command: Commands;
    attribute power: real = 0.0;
    operation apply(value: real) {
      power = value;
    }
    schedule {
      on receive command.set(value) apply(value);
    }
  }
  component Demo {
    component sensor: Sensor;
    component controller: Controller;
    component heater: Heater;
    connect samples (sensor.reading, controller.reading);
    connect commands (controller.command, heater.command);
  }
  view architecture: Interconnect of Demo;
  view logic: Flow of Demo.controller.regulate;
  view timing: Timing of Demo.sensor.sample;
}
```

12 autosar adaptive

SWAML / 12-autosar-adaptive.swaml

```
package Adaptive {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

13 web application

SWAML / 13-web-application.swaml

```
package Web {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

14 cloud service

SWAML / 14-cloud-service.swaml

```
package Cloud {
  import Swaml::Qos;
  port def Catalog {
    provides idempotent operation get(key: int) returns real;
    provides event changed(key: int);
  }
  component def Store {
    port api: Catalog;
    attribute value: real = 4.5;
    operation get(key: int) returns real {
      wait 1ms;
      return value;
    }
    operation announce() {
      publish api.changed(1);
    }
    schedule {
      on call api.get(k) get(k);
      on start announce();
    }
  }
  component def Client {
    reversed port api: Catalog;
    attribute last: real = 0.0;
    attribute changedKey: int = 0;
    operation read() {
      last = try(api.get(1), - 1.0);
    }
    operation changed(key: int) {
      changedKey = key;
    }
    schedule {
      on start read();
      on receive api.changed(k) changed(k);
    }
  }
  component Demo {
    component store: Store;
    component client: Client;
    connect catalog: Reliable (store.api, client.api);
  }
}
```

15 component specialization

SWAML / 15-component-specialization.swaml

```
package Reuse {
  component def Counter {
    attribute count: int = 0;
    operation increment() {
      count = count + 1;
    }
    operation boot() {
      increment();
    }
    schedule {
      on start boot();
    }
  }
  component def Fast specializes Counter {
    redefines operation increment() {
      count = count + 2;
    }
  }
  component Demo {
    component left: Counter {
      redefines attribute count: int = 10;
    }
    component right: Fast;
  }
}
```

16 requirements and views

SWAML / 16-requirements-and-views.swaml

```
package Traceability {
  import Swaml::Views;
  port def Messages {
    provides event message(value: string);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      publish output.message("Hello, SWAML");
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: string = "";
    operation accept(value: string) {
      last = value;
    }
    schedule {
      on receive input.message(value) accept(value);
    }
  }
  requirement Delivery {
    /** One named routed message reaches the receiver. */
    attribute description: string = "Deliver startup message";
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect (sender.output, receiver.input);
  }
  satisfy Delivery of Demo.receiver;
  verificationCase Startup of Demo {
    verify Delivery;
    publish greeting from sender.output.message to receiver.input.message;
  }
  view overview: General of Demo;
  view delivery: Sequence of Startup;
}
```

17 function timing

SWAML / 17-function-timing.swaml

```
package Timing {
  import Swaml::Views;
  component Demo {
    attribute count: int = 0;
    operation sample() {
      wait 1ms;
      count = count + 1;
    }
    schedule {
      on every 5ms sample() deadline 3ms;
    }
  }
  view observed: Timing of Demo.sample;
}
```

18 control loops

SWAML / 18-control-loops.swaml

```
package Loops {
  component Demo {
    attribute count: int = 0;
    operation boot() {
      while (count < 2, 3 times) {
        count = count + 1;
      }
      until (count == 4, 5 times) {
        count = count + 1;
      }
      for (var i: int = 0; i < 2; i = i + 1) {
        count = count + i;
      }
      for (x: int in[1, 2]) {
        count = count + x;
      }
    }
    schedule {
      on start boot();
    }
  }
}
```

19 interruptible function

SWAML / 19-interruptible-function.swaml

```
package Interrupts {
  port def Messages {
    provides event message(value: int);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      wait 2ms;
      publish output.message(5);
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: int = 0;
    operation work() {
      interruptible {
        wait 8ms;
        last = 99;
      } on receive input.message(value) {
        last = value;
      }
    }
    schedule {
      on start work();
    }
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect (sender.output, receiver.input);
  }
}
```

20 classes and operations

SWAML / 20-classes-and-operations.swaml

```
package Operations {
  operation def Clamp(v: real, lo: real, hi: real) returns real {
    if (v < lo) {
      return lo;
    }
    if (v > hi) {
      return hi;
    }
    return v;
  }
  component Demo {
    attribute result: real = 0.0;
    operation clamp: Clamp;
    operation low: Clamp {
      return lo;
    }
    operation boot() {
      operation double(v: real) returns real {
        return v * 2.0;
      }
      result = clamp(double(4.0), 0.0, 5.0);
    }
    schedule {
      on start boot();
    }
  }
}
```

21 receive and try

SWAML / 21-receive-and-try.swaml

```
package Receiving {
  port def Messages {
    provides event message(value: int);
  }
  component def Sender {
    port output: Messages;
    operation emit() {
      publish output.message(8);
    }
    schedule {
      on start emit();
    }
  }
  component def Receiver {
    reversed port input: Messages;
    attribute last: int = 0;
    operation work() {
      wait 2ms;
      on receive input.message(value) {
        last = value;
      } or after 1ms {
        last = - 1;
      }
    }
    schedule {
      on start work();
    }
  }
  component Demo {
    component sender: Sender;
    component receiver: Receiver;
    connect (sender.output, receiver.input);
  }
}
```

22 platform deployment

SWAML / 22-platform-deployment.swaml

```
package Placement {
  import Swaml::Platform;
  component def Worker {
    attribute count: int = 0;
    operation boot() {
      count = count + 1;
    }
    schedule {
      on start boot();
    }
  }
  component Demo {
    component workers: Worker[3];
  }
  component def Ecu specializes Node {
    const attribute cores: uint = 2;
    const attribute ref: string = "sysml:Vehicle::ecu";
    component os: Partition;
  }
  component Vehicle: Platform {
    component front: Ecu;
    component rear: Ecu[2];
  }
  deployment OnVehicle of Demo on Vehicle {
    deploy on front;
    deploy workers on rear[0];
    deploy workers[2] on rear[1];
  }
}
```

23 multiplicity

SWAML / 23-multiplicity.swaml

```
package Multiplicity {
  component def Counter {
    attribute count: int = 0;
    operation increment() {
      count = count + 1;
    }
    operation boot() {
      increment();
    }
    schedule {
      on start boot();
    }
  }
  component def Fast specializes Counter {
    redefines operation increment() {
      count = count + 2;
    }
  }
  component Demo {
    component counters: Counter[3];
  }
}
```

24 operation usage

SWAML / 24-operation-usage.swaml

```
package OperationUsage {
  operation def Clamp(v: real, lo: real, hi: real) returns real {
    if (v < lo) {
      return lo;
    }
    if (v > hi) {
      return hi;
    }
    return v;
  }
  component Demo {
    attribute result: real = 0.0;
    operation clamp: Clamp;
    operation low: Clamp {
      return lo;
    }
    operation boot() {
      operation double(v: real) returns real {
        return v * 2.0;
      }
      result = clamp(double(4.0), 0.0, 5.0);
    }
    schedule {
      on start boot();
    }
  }
}
```

25 function completion

SWAML / 25-function-completion.swaml

```
package Chaining {
  component Demo {
    attribute count: int = 0;
    operation load() {
      wait 1ms;
      count = 1;
    }
    operation report() {
      count = count + 1;
    }
    schedule {
      on start load();
      after load report();
    }
  }
}
```

26 boundary forwarding

SWAML / 26-boundary-forwarding.swaml

```
package Boundary {
  port def Temperature {
    provides attribute temperature: real;
  }
  component def Sensor {
    port reading: Temperature;
    operation sample() {
      send reading.temperature(18.5);
    }
    schedule {
      on start sample();
    }
  }
  component def Composite {
    port boundary: Temperature;
    component child: Sensor;
    connect (boundary, child.reading);
  }
  component def Display {
    reversed port reading: Temperature;
    attribute last: real = 0.0;
    operation display(value: real) {
      last = value;
    }
    schedule {
      on receive reading.temperature(value) display(value);
    }
  }
  component Demo {
    component source: Composite;
    component display: Display;
    connect (source.boundary, display.reading);
  }
}
```

27 mixed port

SWAML / 27-mixed-port.swaml

```
package Mixed {
  /** One boundary may combine data, event and operation members. */
  port def Boundary {
    provides attribute value: real;
    requires event reset();
    provides operation read() returns real;
    requires operation acknowledge();
  }
}
```

swaml 2.0

SWAML / swaml-2.0.swaml

```
package Swaml::Qos {
  enum Delivery { bestEffort, atMostOnce, atLeastOnce }
  enum Ordering { fifo, unordered }
  enum Overflow { dropOldest, dropNewest, reject }
  qos def Policy { }
  qos def Default: Policy { delivery = Delivery.atMostOnce; ordering = Ordering.fifo; capacity = 16; overflow = Overflow.reject; requestTimeout = 1s; }
  qos def Latest: Policy { delivery = Delivery.atMostOnce; ordering = Ordering.fifo; capacity = 1; overflow = Overflow.dropOldest; requestTimeout = 1s; }
  qos def Reliable: Policy { delivery = Delivery.atLeastOnce; ordering = Ordering.fifo; capacity = 16; overflow = Overflow.reject; requestTimeout = 1s; }
  qos def BestEffort: Policy { delivery = Delivery.bestEffort; ordering = Ordering.unordered; capacity = 16; overflow = Overflow.dropNewest; requestTimeout = 1s; }
}
package Swaml::Views {
  view def General; view def Interconnect; view def Case; view def Sequence;
  view def Grid; view def Flow; view def Timing;
}
package Swaml::Platform {
  component def Node { }
  component def Platform specializes Node { }
  component def Partition specializes Node { }
  component def Process specializes Node { }
}
package Swaml::Calls { enum CallFailure { timeout, unavailable, rejected, transport, remoteFailure, interrupted } }
package Swaml::Math {
  operation def abs(v: real) returns real { if (v < 0.0) { return -v; } return v; }
  operation def min(a: real, b: real) returns real { if (a < b) { return a; } return b; }
  operation def max(a: real, b: real) returns real { if (a > b) { return a; } return b; }
  operation def floor(v: real) returns int { /** Host intrinsic: checked floor conversion. */ }
  operation def round(v: real) returns int { /** Host intrinsic: ties toward positive infinity. */ }
  operation def trunc(v: real) returns int { /** Host intrinsic: truncate toward zero. */ }
  operation def length(v: string) returns uint { /** Host intrinsic: Unicode scalar count. */ }
  operation def concat(a: string, b: string) returns string { return a + b; }
}
```

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Quick start: SWAML 2.0

Read a model in this order: components, attributes, ports, operations, schedule, connections. A def defines a reusable type/body; a usage creates a distinct occurrence. Every value has an explicit type. Every cross-component message has an owned port and explicit route.

SWAML

```
package HelloGuide {
  port def Greeting { provides event message(text: string); }
  component def Speaker {
    port output: Greeting;
    operation greet() { publish output.message("Hello, SWAML"); }
    schedule { on start greet(); }
  }
  component def Listener {
    reversed port input: Greeting;
    attribute last: string = "";
    operation accept(text: string) { last = text; }
    schedule { on receive input.message(text) accept(text); }
  }
  component Demo {
    component speaker: Speaker;
    component listener: Listener;
    connect (speaker.output, listener.input);
  }
}
```

Install dependencies with npm ci, build with npm run build, then validate the saved file using node out/src/cli/main.js --json hello.swaml. To execute, create options.json with {"untilNs": 1000000} and run node out/src/cli/simulate.js HelloGuide::Demo options.json hello.swaml. Listener.last becomes Hello, SWAML through a real routed event. Bare connect uses the pinned Default policy.

Continue with the usage guide, complete examples, AI guide and support matrix. Version 2.0 accepts only the new syntax; published 1.x references remain historical artifacts.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0 usage guide

Start with a component, give each persistent value a typed attribute, and write named operations for behavior. Add a schedule to say when those operations run. Add directed ports and explicit connections when behavior crosses component boundaries.

This guide targets the 2.0 parser only. Install Node.js 22+ and npm 10.2.3+, then run npm ci and npm run build in this repository. Validate with node out/src/cli/main.js --language-version 2.0 model.swaml. Files are headerless; the host and package revision select the edition.

A first model

SWAML

```
package Counter {
  component Demo {
    /** A small autonomous counter. */
    attribute count: int = 0;
    operation increment() {
      count = count + 1;
    }
    schedule {
      on every 10ms increment();
    }
  }
}
```

Demo is the execution root. count belongs to its occurrence for the run; a var inside increment would belong to that activation/block. The schedule releases at 10ms, 20ms and so on, with a default 10ms deadline. Operations without a schedule entry are local helpers. Unscheduled behavior is not automatically executed.

Create options.json containing {"untilNs": 25000000}, then run:

SH

```
node out/src/cli/simulate.js Counter::Demo options.json model.swaml
```

The final Demo.count is 2. The JSON records sources, logical trace, limits and final stores. It is bounded reference evidence with zero physical CPU cost. It does not prove production timing.

Reuse structure without sharing state

SWAML

```
package ReuseGuide {
  component def Counter {
    attribute count: int = 0;
    operation increment() { count = count + 1; }
    schedule { on start increment(); }
  }
  component Demo {
    component left: Counter {
      redefines attribute count: int = 10;
    }
    component right: Counter;
    component bank: Counter[3];
  }
}
```

left starts with its own count at 10, then reaches 11. right and every bank index reach 1 independently. A usage body adds members or explicitly redefines an inherited whole member. Single specializes inheritance follows the same rule. Definition identity is shared; occurrence state is separate. Use bank[0] in connector/deployment/view paths. Port arrays and connector loops are absent.

Choose the value's lifetime

Use const attribute for a fixed per-occurrence configuration value, attribute for mutable component state, var for lexical activation data and typed parameters/results for copied calls. Requirements own immutable attributes. Never read another component's private state by a dotted path.

Primitive types are bool, byte, uint, int, real and string; arrays have fixed extents. Define nominal records with datatype and enums with enum. Construct records with every field, as in record Reading { value = 2.5; unit = "C"; }. Access an enum with Mode.active, or a qualified package name if a local value shadows Mode. Check the values example for a complete model. Numeric overflow, division by zero and dynamic invalid indexes fail at runtime; narrowing real to int uses the checked Math conversions.

Compute with operations

SWAML

```
package OperationGuide {
  operation def Clamp(v: real, lo: real, hi: real) returns real {
    if (v < lo) { return lo; }
    if (v > hi) { return hi; }
    return v;
  }
  component Demo {
    attribute result: real = 0.0;
    operation clamp: Clamp;
    operation calculate() {
      operation double(v: real) returns real { return v * 2.0; }
      result = clamp(double(4.0), 0.0, 5.0);
    }
    schedule { on start calculate(); }
  }
}
```

A declaration has parentheses and optional returns; a usage has a type and inherits its signature. Its optional body replaces the whole definition body. Nested operations come first and never capture the outer operation's parameters/locals. Pass values explicitly. Component operations may read/write their own attributes. Package definitions can use only parameters/nested operations. Calls are expressions; void calls are statements; a value result cannot be discarded. Out arguments name mutable compatible cells and copy back only after success. Recursion and overloading are absent.

Import Swaml::Math explicitly for abs/min/max/floor/round/trunc/length/concat. View, QoS, CallFailure and Platform libraries are also loaded and explicitly imported when using their short names. Do not invent host built-ins or assume an arbitrary library exists.

Connect components

The hello model is the smallest complete routed example. A port definition declares provides event message(value: string). Sender owns a port of that type and emits with publish. Receiver uses reversed port, making that member requires, and binds it once in on receive. The root connects the two child ports.

Use provides/requires on each member, rather than assigning a role to the whole port. send updates a provided attribute; publish emits a provided event; p.operation(args) calls a required operation. Port definitions may differ if their members match structurally by name and compatible type. Records/enums remain nominal. A boundary-to-child same-direction connection forwards the boundary without queue/latency; it cannot have a policy or local handler.

A bare connect uses Swaml::Qos::Default: atMostOnce, fifo, capacity 16, reject, request timeout 1s. Select a named complete policy with connect link: Reliable (a.p, b.p). Reliable requests need an idempotent provided operation. Attributes retain the latest pending value regardless of event/request buffering. Nonoptional ports of every concrete root must resolve to a connection. optional only waives that port's closure requirement.

Serve and request a value

See the service example. The provider binds its provided operation through on call api.get(k) get(k). The consumer calls its own required member and usually chooses an explicit fallback:

```text
last = try(api.get(1), -1.0);
```

Timeout comes from the route policy. A bare failing call fails the activation; try catches runtime expression failures and evaluates fallback lazily. Static errors, interruption, deadlines and resource violations cannot be caught. Committed effects remain committed. The reason is trace evidence and is not an implicit model variable. A suspending call must be the whole initializer, assignment value or statement, optionally wrapped in try; avoid api.get(1) + 2.

Schedule and suspend deliberately

on start runs once; on every D runs periodically; on receive binds an incoming event/attribute; on call serves a provided operation; after op runs after successful whole same-owner completion. One operation may have several entries, but the same trigger/operation pair cannot appear twice in one block. After dependencies are acyclic and release fresh parameterless void activations, with no forwarded arguments or result. The chaining example shows this distinction.

wait, requests and receive statements release the component's serial lock. Another activation can change attributes before resumption. Recheck conditions after resuming before writing state. A concurrent entry cannot write state, even through a helper or out argument, and always reads its dispatch snapshot. Entry deadlines include suspension/backlog; an every entry defaults to its period. WCET belongs to target binding/evidence.

while tests first; until tests after its body. An optional n times limit evaluates once and exits normally. Counted for and copied-array for in have lexical loop scopes. The loop model covers every form. Unbounded loops are permitted but a finite run has a step budget/deadline.

Receive, interrupt and model state

Receive statements wait for one of several explicitly bound inputs or a timeout, keeping early input queued. An interruptible region is armed only during its body/callees and transfers within the same activation, preserving committed effects. Inactive region input is discarded observably; heed SW2012.

For a state machine, write initial and state directly in an operation. Put transitions inside each source state, in the order they should be tested. Use on entry/on exit, on receive with optional when, after D to Target, and final state. A stateful operation has no parameters/result and exactly one on start schedule entry. The state example completes at its final state; idle reactions do not release after dependents.

Describe deployment, intent and views

Use a separate deployment declaration over software and platform roots. Targets are component occurrences specializing Swaml::Platform::Node. Const attributes may retain external ref identities; task/core scheduling stays in target adapters. The deployment example shows family placement with an indexed override.

Requirements, satisfy and verificationCase/verify record intent. The traceability example includes a routed named case and views. A case does not inject events or prove a witness. Use from/to/via/after to describe exchange dependencies.

Import Swaml::Views and type views by General, Interconnect, Case, Sequence, Grid, Flow or Timing. Compact of selects a subject; include/recursive/where builds a stable union. Filter kinds are component and operation for the structural/executable families. Browser navigates ownership; Flow combines statements/states; Timing separates declared schedule/deadline information from identified observed traces. Presentation layout belongs in sidecars, not language source.

Validate and repair

```sh
node out/src/cli/main.js --json model.swaml
node out/src/cli/main.js --format model.swaml
node out/src/cli/main.js --check-format model.swaml
npm run check
```

Load every application dependency together. Fix syntax/names first, then ownership/routes, types/effects and schedules. JSON diagnostics expose stable codes/ranges and identity metadata. Always revalidate after a repair; formatting is not a semantic repair. Read AI authoring, the keyword reference and the support matrix for precise boundaries.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

AI authoring with SWAML 2.0

Treat SWAML as typed source that must be validated, rather than prose to approximate. Start with the usage guide, inspect a relevant complete example, and read the affected normative chapter. Pin parser 2.0 and the exact package revision. Use the portable repository skills swaml-author and swaml-validate; they are versioned with the language and may be installed in the user's agent skill directory. They do not authorize publication or external messages.

Build a model from intent

Identify execution roots and reusable component definitions. Assign each persistent value to one component attribute, configuration to const attribute and scratch data to var. Identify every exchange before writing behavior: give its two sides owned port members with provides/requires, then an explicit connect route. Decide which inputs are queued events and which are latest attributes. Bind each input exactly once.

Write operations with explicit parameter/result types. Use return for values, expressions for calls and statements for void calls. Nested operations take explicit inputs; they do not capture enclosing locals. A package operation def is parameter-only reusable behavior. Use a usage to reuse its signature/body. No class/function/calc/interface, role suffixes, call/as syntax, profile or old transition syntax is accepted.

Put activation in one schedule. Use on start/every/receive/call or after same-owner operation. Select complete policy definitions where Default is unsuitable. Keep remote calls as whole initializer/assignment/statement forms, optionally in try. Recheck live attributes after suspension; use concurrent only when the transitive operation graph cannot write. Copy-back of out parameters occurs only on success.

Multiplicity is literal and paths are indexed explicitly. Deployment targets are platform Node occurrences. Requirements and cases express intent; they are not proof or input drivers. View definitions select existing identities, with geometry separate.

Validate-fix loop

SH

```sh
npm run build
node out/src/cli/main.js --json model.swaml dependency.swaml
node out/src/cli/main.js --format model.swaml
node out/src/cli/main.js --json model.swaml dependency.swaml
```

Inspect errorCount and diagnostic code/range/message/data. Resolve parser/name errors before reasoning from a recovered AST. Fix the smallest relevant declaration, keeping private scopes and explicit routes intact. A suggested edit is advice to revalidate, never permission to apply a stale edit. Validate the complete source workspace after changes, including negative lookups made ambiguous by newly loaded files. Never invent a library, untyped value, implicit port, hidden event or direct destination injection to silence diagnostics.

Run a bounded simulation only after validation. Record source bytes/hashes, root, profile, options, deployment and trace. Inspect final occurrence stores and relevant failure/timeout/discard/interrupt observations. A caught failure is still evidence of that runtime failure; no-failure-observed is a finite-run statement. Show declared timing separately from observed intervals and never label zero-cost logical steps as measured WCET.

Common diagnostics

Group	Repair direction
SW2001	Load/import the correct package, disambiguate a name, remove shadowing
SW2002	Correct explicit types, record shape, fallback/result or array bounds
SW2003	Add explicit redefines only for a real compatible inherited member; break cycles
SW2004/2005	Repair member direction/type/route/closure, emission provenance or exclusive input owner
SW2006	Correct nested scope, return/out use, effects or whole-expression suspension
SW2007/2008	Repair schedule identity/signature/after graph or ordered state ownership
SW2009/2010	Resolve occurrence/Node paths, case route/correlation or view subject
SW2011/2012	Recheck live state after resumption; account for inactive interrupt input
SW2013	Supply complete typed policy values and safe positive durations

Keep diagnostics/evidence honest. Consult implementation status before claiming full V/E/G/K support. Publishing a page or release is a separate user-authorized action; this project requires a design preview before publication.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Architecture patterns

SWAML 2.0.

Start each pattern from owned component state and explicit typed boundary members. Components may be leaves or composites and both may own operations. A composite forwards a port to an immediate child with same-direction connect; peer connections use opposite-compatible directions and own queues.

A pipeline sends latest attributes when freshness matters and publishes events when every occurrence matters. The controller model samples temperature, computes power and publishes an actuator command. Each receiver owns its input once; no private data is read through a sibling path.

A service uses required/provided operation members, on call schedule bindings and a policy request timeout. try supplies a lazy fallback. Suspension unlocks the caller; recheck live state before committing a decision. Service model.

Publish/subscribe fanout uses event-only required ports and several explicit connections. Each connection occurrence has an independent policy queue; partial fanout may commit before a failed offer. Adding an attribute or operation requirement changes cardinality to exactly one peer.

Reusable components use single specializes inheritance and explicit compatible whole-member redefines. Usage bodies configure per-occurrence const/attribute values without sharing stores. Reuse and multiplicity.

Classic, Adaptive, web and cloud example filenames illustrate these software patterns. They do not claim generated RTE/task, middleware, broker, infrastructure or production timing bindings. Deployment places software on platform Node occurrences; target translation is separate.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Ontology and views

SWAML 2.0.

The semantic families are package, nominal values/types, component definition/usage, attribute/local/parameter, port/member, operation definition/declaration/usage, schedule/entry, connection/forwarding, state/transition, deployment, requirement, case and view. A nested operation has no closure capture; an operation body has no independent executable identity. An indexed occurrence is distinct from its shared definition.

Browser navigates ownership. Seven imported library definitions provide source views: General, Interconnect, Case, Sequence, Grid, Flow and Timing. General and Grid can select named model identities; Interconnect selects components; Case/Sequence select roots/cases; Flow/Timing select operations. General Box in box and Tree retain the same selected graph. Flow contains statements, loops, receive/interrupt regions and states.

Compact of selects one subject. Include/recursive/where selects a stable union with Boolean name/kind/path predicates. Structural/executable kinds are component and operation. Filters do not call behavior. Timing declared mode shows schedules/deadlines/static waits; observed mode requires a run identity and preserves open intervals. Geometry/routing/camera live in sidecars. See normative view rules and traceability example.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0 reference simulation

The portable interpreter selects **swaml-reference/2.0** and requires linked, validated source. All activation comes from ordinary source schedules/ports/routes, including authored test drivers. Execution semantics is normative; this guide covers the reference host.

SH

```
npm ci
npm run build
node out/src/cli/main.js --json examples/01-hello.swaml
node out/src/cli/simulate.js Hello::Demo examples/simulation/01-hello.json examples/01-hello.swaml
node scripts/check-simulations.mjs
```

Options include required untilNs and optional maxSteps (100000), maxPending (1024), maxConcurrent (64), latencyNs (0), deployment (qualified name) and explicit offer-indexed faults. Unknown options are rejected. faults contains an offer number, kind drop/duplicate/transport and optional delayNs. Faults operate on real routed source emissions/requests, never destination injection. The deterministic backend chooses insertion order for unordered queues and records its delivery assumptions.

Serial wait/request/receive suspensions release the component lock; resumption reads live state. Concurrent activations retain a dispatch snapshot and cannot write. Default connection policy is atMostOnce/fifo/16/reject/requestTimeout 1s. Attribute queues keep latest values independently. try catches runtime expression failures with lazy fallback and retained effects; deadlines/resources/interrupts bypass it. Requests retain route correlation, timeout wins ties and late replies are suppressed. After edges release fresh work after successful whole completion/unlock.

The JSON result records profile, root, options, logical trace ordinals/identities, sources, final occurrence stores, state machines and selected placements. CLI adds SHA-256 source digests. Status is quiescent, horizon, violation or limit. Failure observations remain evidence even when a caller catches them. Horizon retains live/open intervals. An event left pending without possible progress may be quiescent; that is not proof of useful behavior.

The examples' expected final stores are checked independently, and deterministic reruns check reproducibility. Tests cover adversarial requests, interruptions, failure effects and resource limits. Physical CPU/WCET, production transport/retry protocols, target scheduling and universal analysis remain outside this logical interpreter. Exporting a trace does not establish those claims.

Outcome failure-observed includes caught expression failures as well as failed activations; status separately records whether execution reached quiescence, a horizon or a resource/contract stop. The CLI returns 1 when a failure was observed, including a successfully handled fallback, so automation must inspect the trace and final stores when exercising expected failure scenarios.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Requirements, documentation and intent

SWAML 2.0.

A requirement owns immutable typed attributes and at most one formal doc. Use /** ... */ *first in a body or immediately before the semicolon of a leaf declaration. Ordinary // and /* ... */ comments remain separate lexical annotations. Formal documentation is owned source, not an executable statement or proof.

satisfy Requirement of designPath links intent from a design identity. Only verificationCase owns verify. Use cases describe an intended interaction without asserting verification. A case selects a concrete root with of and uses named send/publish/request/reply steps from source to target, optionally via a named connection and after a predecessor.

Routes, member kinds/directions and request/reply correlation must resolve to the root topology. Cases do not create channels or inject events. A route-valid case needs an identified witness execution before realization can be claimed. A requirement needs associated test/analysis/review evidence before fulfillment can be claimed.

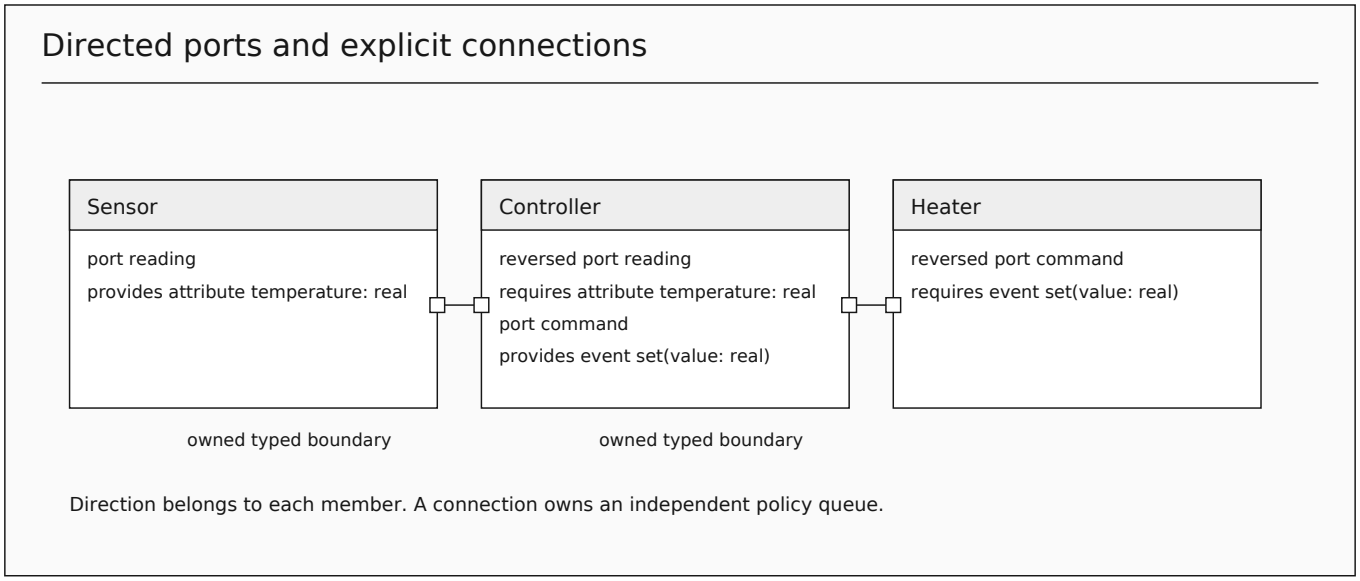
The complete traceability model contains a documented requirement, satisfaction, verification case and selected views. General/Grid can inspect intent relations; Case/Sequence present cases and causality. All preserve occurrence identity and retain unproven status where evidence is absent.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0 worked diagrams

Generated from original scene geometry; source subjects remain authoritative.

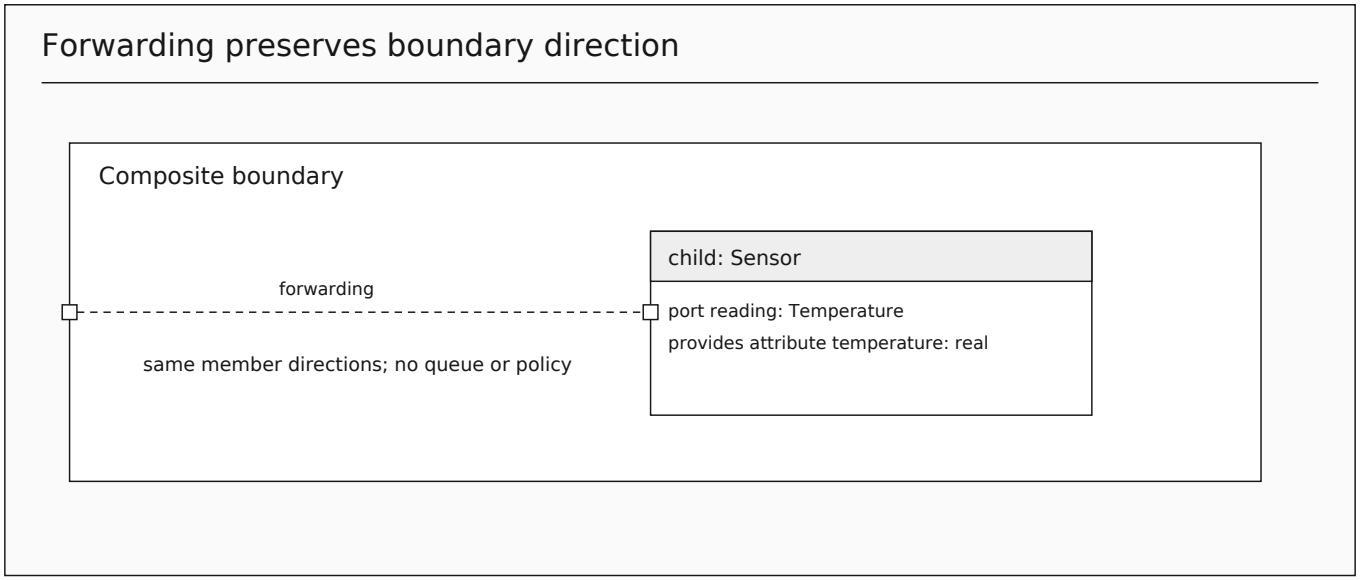
Directed ports and explicit connections



Directed ports and explicit connections

Source: Control::Demo.

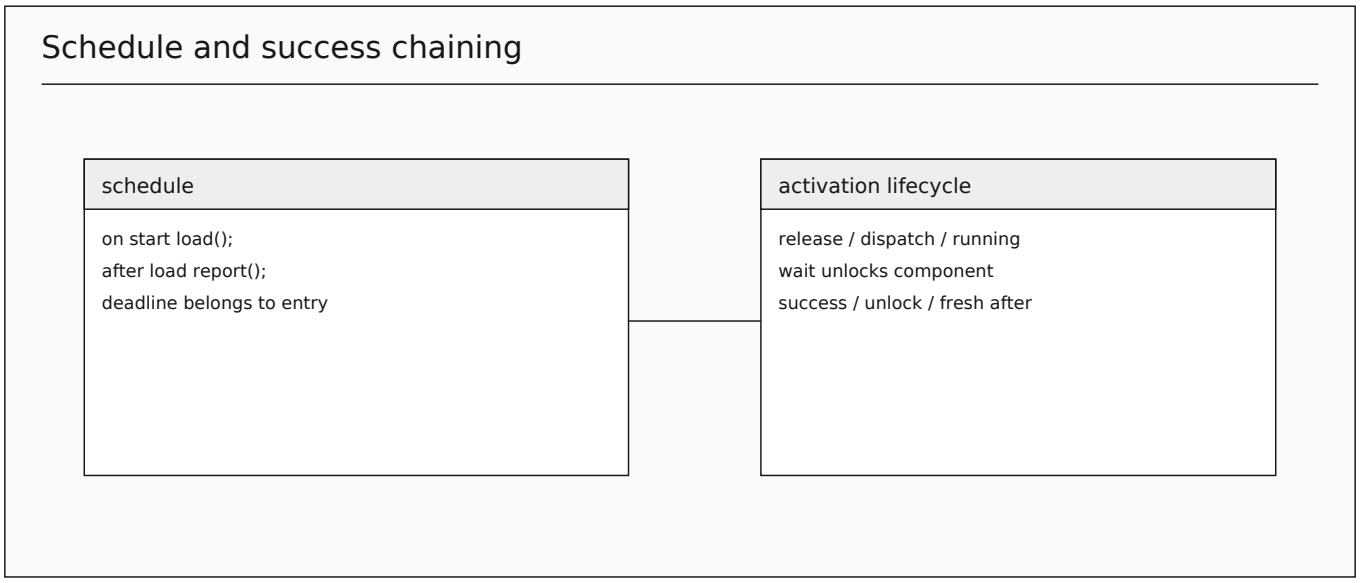
Forwarding preserves boundary direction



Forwarding preserves boundary direction

Source: Boundary::Demo.

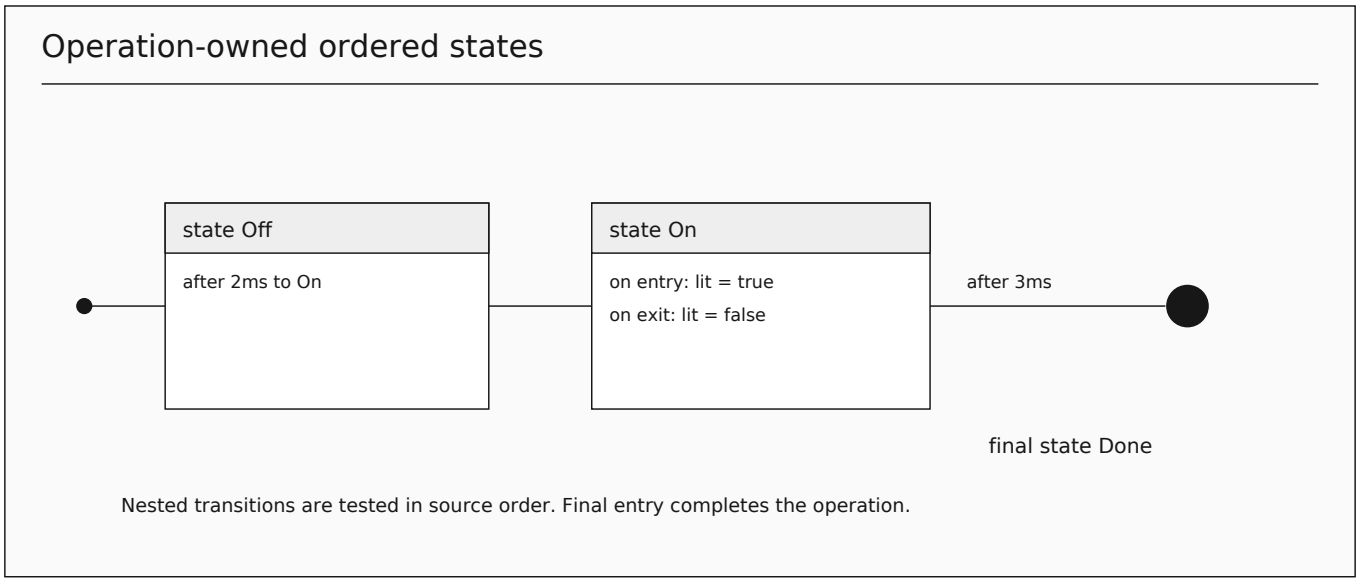
Schedule and success chaining



Schedule and success chaining

Source: Chaining::Demo.

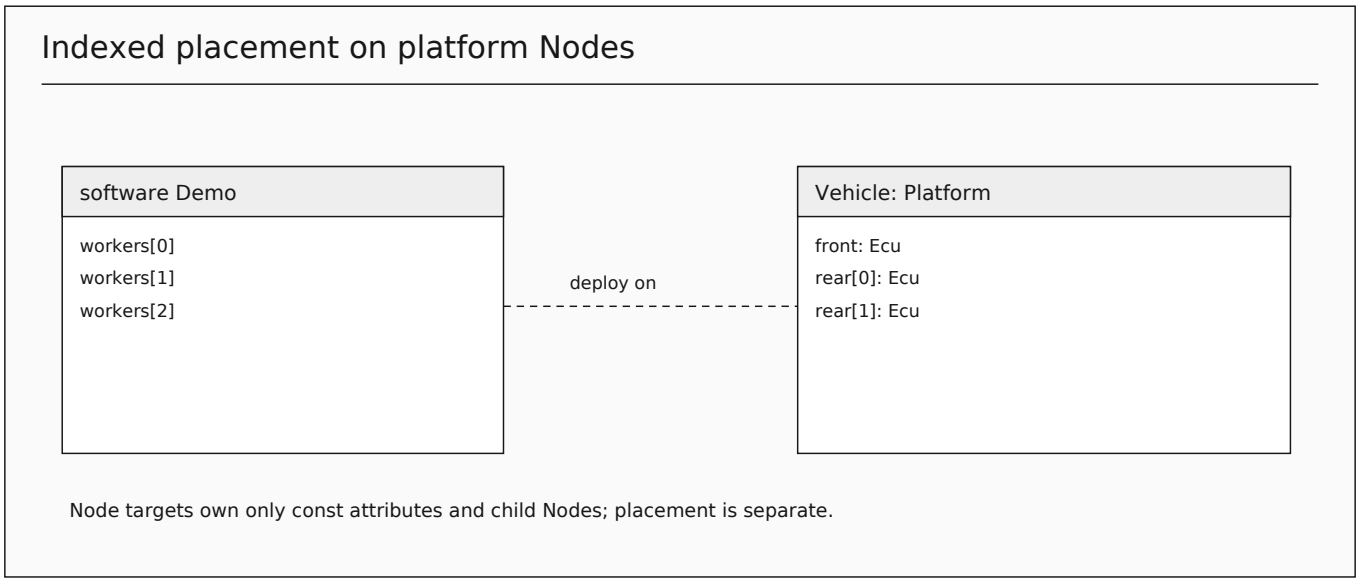
Operation-owned ordered states



Operation-owned ordered states

Source: States::Demo.lamp.

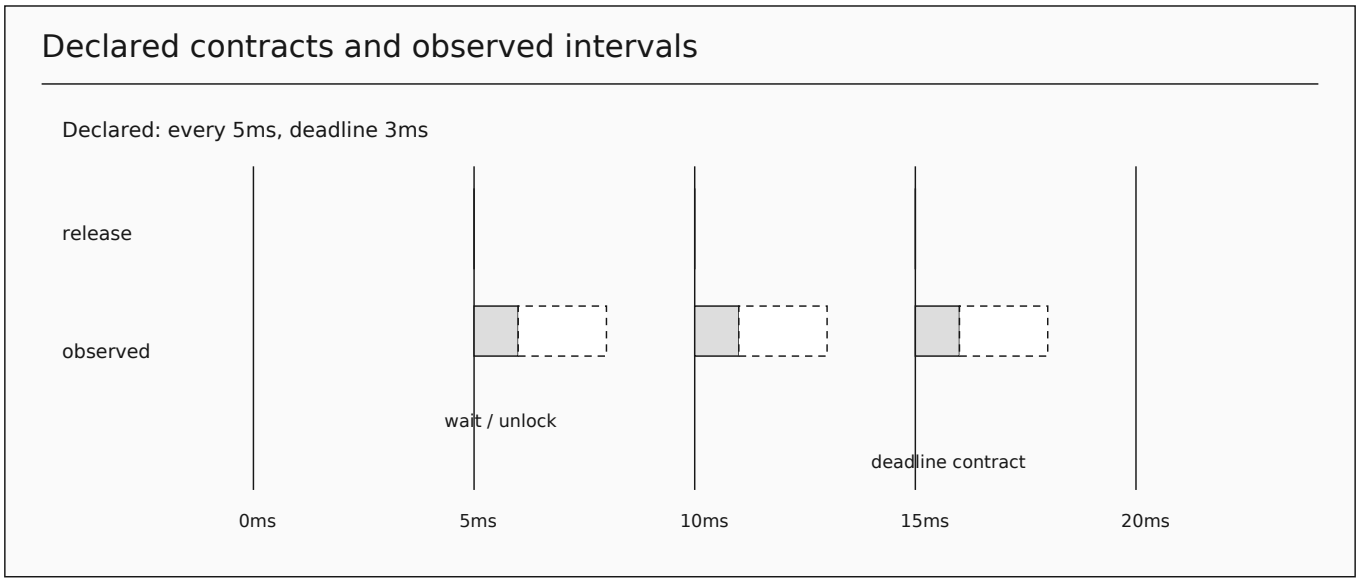
Indexed placement on platform Nodes



Indexed placement on platform Nodes

Source: Placement::Demo.

Declared contracts and observed intervals



Declared contracts and observed intervals

Source: Timing::Demo.sample.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0 implementation status

Edition/package: 2.0.0 release · **Parser:** 2.0 only · **Reference profile:** swaml-reference/2.0 · **Publication:** versioned reference edition.

The 2.0 core replaces the 1.x grammar, AST, resolver, validator and interpreter. It does not accept historical source. Published 1.x books, PDFs, manifests/checksums and historical figure files remain unchanged. ADR 0016 defines the restart; conformance obligations separate syntax, validation, execution, analysis, kernel and graphical claims.

Reference implementation

Capability	Implemented evidence and boundary
Syntax and recovery	Normative Langium grammar, generated AST/TextMate, malformed input and retired-syntax rejection tests
Components and values	Single specialization, whole redefinition, usage bodies, typed roots, indexed multiplicity, private copied stores, nominal records/enums/arrays and checked domains
Operations	Definitions/usages, explicit returns, nested no-capture scope, expression calls, in/out, success-only copy-back, definite out assignment and acyclic calls
Effects and failures	Transitive write/read/communication/suspension flags, pure initializers, read-only guards, non-writing concurrency, lazy try and allowed suspension placement
Ports and routes	Per-member direction/reversal, structural compatibility with variance and individual diagnostics, connection/forwarding, closed roots, resolved cardinality and ownership/emission provenance
Policies	Versioned complete typed QoS definitions, Default/Latest/Reliable/BestEffort, per-use idempotency; occurrence/direction/member queues and latest attributes
Schedule and runtime	Start/every/receive/call/after entries, inherited entry replacement, dispatch copying, deadlines, lock release/reacquisition, live serial reads and concurrent snapshots
Flow and state	Loops and copied iteration, wait/receive, interruption across callees, ordered nested transitions, final completion and queued startup input
Correlation and adverse behavior	Routed results/out values, failures, timeout ties, terminal cancellation, late-reply suppression, policy overflow, age at eligibility and explicit resource limits
Deployment	Separate software/platform roots, Node-only contents/targets, indexed/family placements and most-specific selection
Requirements/cases/views	Immutable intent, routed named steps with causal replies, seven library definitions, package/member subjects and stable occurrence unions
AI authoring	Usage and AI guides, portable author/validate skills, --json diagnostics with code/range/URI/declaration/occurrence metadata and selected safe repair edits
Authoring services	Canonical token-preserving full-document formatting; semantic index, definition/references and versioned multi-file rename over indexed paths, port members, nested operations, schedules, states, placements, cases and view subjects/eligible filters
Host behavior	CLI exit/JSON/edition checks, LSP 2.0 negotiation, complete dependency enforcement, standard-library validation and workspace negative-lookup invalidation

Capability	Implemented evidence and boundary
Diagrams and page	Exhaustive notation/mapping inventories; original 2.0 SVG/PNG/scenes and source-subject checks; generated source/trace site data, seven interactive views, light/dark/system themes and responsive preview

The unit suite exercises observable positive/negative language behavior, malformed recovery, final stores/traces and validated authoring edits. The example gate validates every current model, checks explicit expected stores and deterministic reruns. These are reference implementation tests, not an independent formal certification of all possible executions.

Deliberate implementation limits

The reference validator expands at most 10000 component occurrences per concrete root and reports SW2013 above that host limit. This is a reference resource boundary, not a new mathematical language multiplicity restriction. Stale-read analysis is conservative and may warn after a safe recheck. Definite-return/out assignment is conservative. Formatter range/on-type operations are not implemented. A rename that introduces an error, changes any indexed reference binding, or cannot preserve selected view identities is refused; persistent editor sidecar identity/native undo belongs to the editor integration.

Math abs/min/max/concat are ordinary source; floor/round/trunc/length are explicitly documented host intrinsics with checked semantics. The interpreter uses zero physical CPU cost and one deterministic permitted order, with finite resources and recorded fault assumptions. It does not implement a production scheduler, real network reliability protocol or universal analysis.

No validated KerML exporter or executable KerML semantic library exists; chapter 5 and the mapping inventory remain a proposed contract. No XMI adapter, generic UML/SysML importer or graphical conformance certification is claimed. The adjacent editor has its own pin, source-edit, desktop/browser and package gates; its completion must be reported separately from a passing core suite.

The 2.0.0 release includes the production website and a separately versioned PDF under the publication gate. Draft mockups are excluded. See the review resolution and release notes; archived editions are never rebuilt from current sources.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Tool integration

SWAML 2.0.

The language core is portable TypeScript over Langium 4.4.0 and the normative grammar. Node CLI/server adapters own file access and source hashing. Build with npm run build; validate with node out/src/cli/main.js --json and explicitly load all application dependencies. The host selects parser 2.0; unsupported edition flags are errors.

JSON diagnostics expose stable SW2xxx codes, range, URI and identity metadata. --format applies canonical token-preserving whitespace; --check-format checks it without writing. The LSP exposes full-document formatting. Source repairs must be validated against the complete workspace before application.

The simulator requires validated models and records swaml-reference/2.0, sources, options, trace, final stores and deployment. Source digests are added by the CLI. Its finite logical run does not supply production timing or universal verification.

The adjacent editor repository has separate acceptance, diagram, refactoring and packaging gates. Paths with multiplicity, port members, schedules, nested operations, deployments, cases and views need semantic path-aware navigation/rename; generic Langium references alone are insufficient. Preserve occurrence identity and validated source transactions, with geometry in sidecars. See implementation status for the current evidence boundary.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0 review resolution

The 22 October 2026 source review of revision 46044b5 identified the following defects. Each row records implemented corrections and evidence; pending rows are not acceptance claims. The reference interpreter remains bounded evidence under its recorded assumptions.

Finding	Resolution and regression evidence
F01 Package operation out writes bypass concurrency validation	Fixed: resolve package callees and preserve the caller's out-target write effect; imported/transitive negative cases and serial/local positive cases in review-validation.test.ts
F02 Package calls capture component helpers	Fixed: Lexical package calls and reused bodies cannot capture occurrence helpers; inheritance still dispatches effective component members. Regression evidence in review-runtime.test.ts
F03 Rename changes reference binding	Fixed: Trial rename compares every indexed binding by stable AST identity and translated token ranges, rejecting capture even when validation passes. Regression evidence in review-authoring.test.ts
F04 Request out variance crashes	Fixed: Request out cells use the provider type before successful caller copy-back. Regression evidence in review-runtime.test.ts
F05 Port payloads bypass numeric domains	Fixed: Outgoing attribute/event/request values check the own port domain before offers, including wider peers. Regression evidence in review-runtime.test.ts
F06 Negative computed loop limits succeed	Fixed: Computed limits check uint once; negative counts fail and can be caught. Regression evidence in review-runtime.test.ts
F07 Initializer failure escapes evidence	Fixed: Initializer faults yield source-attributed failure evidence before startup; CLI returns JSON and exit 1. Regression evidence in review-runtime.test.ts
F08 QoS capacity accepts duration	Fixed: capacity is a positive integer count; duration/boolean/string/fraction/zero rejection in review-validation.test.ts
F09 Enum token indexing is ambiguous	Fixed: Enum heads use the first path segment with concrete token ranges, including qualified names and comments. Regression evidence in review-authoring.test.ts
F10 Inherited case via references are missing	Fixed: Case navigation shares effective occurrence/alias route resolution with validation, including inherited repeated occurrences. Regression evidence in review-authoring.test.ts
F11 Projection drops package containment	Fixed: Projection records package ownership without inventing namespace-prefix containment. Regression evidence in review-authoring.test.ts
F12 Ownerless redefines is accepted	Fixed: replacement requires a containing component's inherited member; package/nested negative and valid replacement tests
F13 Duplicate delivery exceeds queue capacity	Fixed: Every duplicate copy observes capacity/reject/dropNewest/dropOldest; committed copies remain observable. Regression evidence in review-runtime.test.ts
F14 Linking diagnostics have missing/wrong code	Fixed: shared DocumentValidator assigns SW2001 to linking and SW2000 to syntax; CLI and malformed-recovery regression checks

All fourteen findings have implemented corrections and regression coverage. The validation, runtime and authoring groups passed npm run check with 185, 197 and 202 tests respectively, including example, simulation, documentation and inventory gates. Production site model data reflects corrected ownership. Final publication is gated separately on the versioned PDF, visual inspection and cross-platform checks. These results do not establish universal verification, production scheduling or kernel conformance.

Timers begin at state entry before entry actions, including suspension. Elapsed timers precede queued input after entry; equal expiry uses source order, deadline ties win, and final entry completes once. Tests cover each ordering and successful after dependencies.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 2.0.0 release

22 October 2026. Parser 2.0, package 2.0.0, reference profile swaml-reference/2.0.

The clean 2.0 language replaces the 1.x grammar and runtime with components, private attributes, reusable operations, mixed directed port members, explicit routes, schedules and operation-owned states. It accepts only 2.0 source. Published 1.x editions retain their original artifacts.

All fourteen findings from the source review have corrections and regression coverage in the resolution record. The language gate passes 202 tests plus example, simulation, source/diagram inventory and documentation checks. These checks are bounded implementation evidence, not universal verification.

State timer epochs begin at state entry before entry actions. Time spent suspended in entry counts toward the delay. Entry finishes before a transition runs; an elapsed timer precedes queued input, equal-expiry timers follow declaration order, and deadlines win ties. Final entry completes once and releases successful after dependencies.

The port symbol is one square boundary with member-level glyphs: arrows for attribute/event direction and ball/socket for provided/required operations. Mixed members retain their individual names, types and directions.

The release includes the reference book, PDF, source manifest and checksum. The manifest identifies the source base commit, package revision and exact canonical source bundle, fonts and dependency versions. Embedded sources use repository LF text bytes; the final publication commit also contains the generated PDF itself.

The production website publishes the validated model views, current release downloads and mixed-port notation. Draft mockups are excluded. A bounded run does not establish production scheduling, real transport reliability, universal correctness or KerML export conformance; implementation limits remain explicit.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0016: SWAML 2.0 restart

Status: accepted for implementation, 2026-10-21.

The accepted design is the 2.0 proposal, sections 1-11 and 13-15. This is a clean restart: no compatibility parser, aliases, conversion tool or new migration guide. Published 1.x books, PDFs and their provenance remain immutable.

SWAML has one structural kind, component, and one executable kind, operation. Attributes belong to occurrences; locals belong to activations. Nested operations never capture enclosing locals. Operation definitions supply reusable signatures and bodies. Usage bodies replace whole operations or explicitly redefine component members. Inherited calls resolve against effective occurrence members.

Ports own directed attribute, event and operation members. Compatibility is structural by name, with nominal records/enums. Reversal flips every direction. Connect derives connection or forwarding from boundary ownership and direction. Only connections own queues and policies; omission selects the pinned Default. Attributes retain the latest value independently of buffering.

One effective schedule binds triggers to operations. Suspension releases the serial lock. Serial resumptions read live attributes; concurrent activations retain their dispatch snapshot and cannot write state through any callee. After dependencies create new activations only after successful whole completion and unlock. Out parameters copy back only on success. Try catches expression failures, preserving committed effects; it cannot catch interruption or contract/resource violations.

State transitions are nested, ordered and unnamed. Stateful operations have exactly one on start entry. Receive statements exclusively own their members. Interrupt regions are armed only during their protected body, including suspended callees; inactive input is discarded with trace evidence. Concurrent operations may receive/interrupt if their transitive effects do not write state. Each input is consumed once, in activation order in the reference interpreter.

Deployment is separate and targets platform Node occurrences. Multiplicity expands separate stores. Views use General, Interconnect, Case, Sequence, Grid, Flow and Timing library definitions. Browser remains the editor navigation surface. Timing separates declared contracts from identified observations. Geometry stays in sidecars.

Libraries are versioned source. Any Math host intrinsics must be explicitly identified and implemented by a backend. KerML remains a proposed mapping until an exporter, semantic library and kernel validation exist.

Stable diagnostics: SW2001 names/imports; SW2002 types/values; SW2003 structure/redefinition; SW2004 ports/routes; SW2005 ownership/provenance; SW2006 operations/effects; SW2007 schedules; SW2008 states; SW2009 deployment; SW2010 cases/views; SW2011 stale reads; SW2012 inactive interrupt input; SW2013 policies/durations. Diagnostics retain ranges and identity; attach edits only when unambiguous.

Supersedes ADRs 0007, 0008, 0011, 0014 and 0015 for 2.0. Amends ADR 0010 with from/to and returns; ADR 0012 with library views and declared/observed timing; ADR 0013 with source operations and schedules. Their published-edition decisions remain historical evidence. ADRs 0001, 0002, 0006 and 0009 retain the boundary, evidence, queue, requirement and identity constraints.

Acceptance requires positive/negative language tests, malformed recovery, runtime traces/stores, synchronized inventories/diagrams, executable guides and an honest support matrix. Preview the site before publication. A release additionally requires the PDF and visual publication gate.

The proposal’s deployment illustration spelling component def Ecu: Node is resolved consistently with its unchanged single-inheritance rule: definitions use specializes Node; colon types usages. No alias is accepted. The reference host reports an explicit 10000-expanded-occurrence validation resource limit, without changing the language’s finite multiplicity domain.

D. License

MIT License

Copyright (c) 2026 Catalin Ilie

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.